



Universidad Nacional Autónoma de México

Programa de Maestría en Ciencia e Ingeniería de la
Computación

Detección del ataque Software Fault Injection mediante el análisis
topológico de datos dentro de la Infraestructura como Servicio
OpenStack.

T E S I S

que para optar por el grado de
Maestro en Ciencia e Ingeniería de la Computación

PRESENTA:
Andrés Martínez López

Tutor:
Dr. Luis Francisco García Jiménez

México, CDMX. Junio 2026

Agradecimientos

A mi familia

Por su apoyo incondicional durante cada etapa de mi vida. Gracias por acompañarme en los momentos difíciles y celebrar conmigo cada logro alcanzado. Su esfuerzo, cariño y motivación desde el inicio han sido fundamentales para transitar y disfrutar esta etapa.

A mis amigos

Por su compañía, apoyo y ánimo a lo largo de esta etapa. Gracias por los consejos, las conversaciones, los momentos de distracción y hasta los retos compartidos que han logrado enriquecer mi perspectiva ante la vida. Su amistad hace que cada desafío sea mucho más fácil de afrontar.

A la Universidad Nacional Autónoma de México

Quiero expresar mi más sincero agradecimiento a la **Máxima Casa de Estudios de México**, la UNAM. Desde mis primeros años de formación y que a través de mi familia, esta noble institución me ha brindado conocimiento, amistades invaluable y un profundo sentido de responsabilidad social. Gracias a ello, he podido convertirme en el profesionista y ser humano que hoy soy.

Deseo expresar mi más sincero agradecimiento a todos los profesores que formaron parte de mi trayectoria académica. A través de sus enseñanzas, consejos y ejemplo, me han impulsado a superarme tanto en el ámbito académico como profesional, motivándome a aspirar siempre a más y a descubrir el potencial de lo que puedo llegar a ser. En especial, deseo agradecer profundamente al **Dr. Luis Francisco García Jiménez**, cuyo apoyo constante ha sido invaluable y pieza clave en este camino. Su guía, paciencia y confianza en mi trabajo han sido piezas clave de esta etapa.

Finalmente, agradezco el apoyo dado por la beca **SECIHTI** con CVU 1318796 y al proyecto **PAPIIT IN109624**.

Índice general

Resumen	1
1. Introducción	2
1.1. Definición del problema	4
1.2. Alcance de la tesis	4
1.3. Objetivo	4
1.4. Hipótesis	4
1.5. Metodología	4
1.6. Contribución y resultados	5
1.7. Estructura de la tesis	5
1.8. Anexos	6
2. Cómputo en la nube	7
2.1. Computación en la nube	7
2.2. OpenStack	8
2.2.1. Estructura de OpenStack	8
3. Marco teórico (Topología algebraica)	10
3.1. Análisis de datos de alta dimensión	10
3.1.1. Datos de alta dimensión	10
3.1.2. Técnicas de reducción dimensional	10
3.2. Teoría de grupos	11
3.2.1. Grupo	11
3.2.2. Subgrupos	12
3.2.3. Homomorfismos	13
3.3. Topología algebraica	14
3.3.1. Espacios topológicos	14
3.3.2. Simplex o Simplex	14
3.3.3. Complejos simpliciales	15
3.3.4. Complejos de Vietoris-Rips	15
3.3.5. Homología	16
3.3.5.1. Grupos de Homología	16
3.3.5.2. Números de Betti	17
3.3.6. Homotopía	18

4. Análisis Topológico de Datos	19
4.1. Algoritmo de Mapper	19
4.2. Homología persistente	20
4.3. Similitud entre conjunto de puntos	22
4.3.1. Distancia de Wasserstein	22
5. Implementación del ataque Software Fault Injection	24
5.1. Ataques cibernéticos	24
5.2. Herramientas de detección	25
5.3. Ataque Software Fault Injection	26
5.4. Ejecución del programa FIT4Python	27
5.5. Recolección de bitácoras del sistema	29
6. Modelo para la detección del ataque SFI	33
6.1. Estandarización de datos	33
6.2. Reducción de dimensionalidad de las bitácoras	35
6.3. Representación en R^2 de las bitácoras usando Mapper	37
6.4. Cómputo de características topológicas	42
6.4.1. Comparación de características topológicas	44
6.5. Comentarios finales	49
7. Análisis y resultados	51
7.1. Métricas de evaluación	51
7.2. Pruebas de evaluación	52
7.3. Comentarios finales	56
8. Conclusiones	57
8.1. Desafíos del modelo	58
8.2. Puntos destacados en el desarrollo	58
8.3. Trabajo futuro	59
A. Creación de Máquinas Virtuales de la IaaS	60
B. Instalación y Configuración de OpenStack	61
B.1. Configuración de Red	61
B.1.1. Nodo Controller	61
B.1.2. Nodo Compute	62
B.2. Resolución de Nombres	62
B.2.1. Verificación de Conectividad	63
B.3. Sincronización de Tiempo	63
B.3.1. Nodo Controller	63
B.3.2. Nodo Compute	63
B.3.3. Repositorio Oficial de OpenStack	63
B.4. Servicios Base - Nodo Controller	64
B.4.1. Configuración de MariaDB	64
B.4.2. Configuración de RabbitMQ, memcached y etcd	64
B.5. Servicio de Identidad (Keystone)	65

B.5.1. Nodo Controller	65
B.6. Servicio de Gestión de imágenes (Glance)	66
B.6.1. Nodo Controller	66
B.7. Servicio de Gestión de recursos disponibles (Placement)	67
B.7.1. Nodo Controller	67
B.8. Servicio de Gestión de recursos de las instancias de máquinas virtuales dentro de OpenStack (Nova)	68
B.8.1. Nodo Controller	68
B.8.2. Nodo Compute	70
B.9. Servicio de administración de redes virtuales (Neutron)	72
B.9.1. Nodo Controller	72
B.9.2. Nodo Controller	75
B.10. Servicio de administración vía interfaz Web (Horizon)	76
B.10.1. Nodo Controller	76
C. Modificación de parámetros dentro de FIT4Python	80
D. Archivo main.py de FIT4Python	83
E. Archivo injection_manager.py de FIT4Python	84
F. Archivo wlec_fault.py de FIT4Python	91
G. Archivo para la segmentación de registros por tipo de ataque SFI	94
H. Archivo para la automatización de pruebas con los archivos API modificados	95
I. Archivo para la limpieza de registros	98
J. Archivo para segmentación por cardinalidad porcentual	100
K. Archivo para la estandarización de los datos	102
L. Archivo de uso de algoritmos de reducción de dimensionalidad PCA, UMAP, t-SNE	107
M. Archivo para unión de archivos estandarizados	109
N. Archivo para la construcción de grafo global mediante TDA	111
Ñ. Archivo para la generación de diagramas de persistencia mediante complejos de Vietori-Rips	115
O. Archivo para el cálculo de distancias de Wasserstein entre diagramas de persistencia	121
P. Programa para la generación de gráficas comparativas de distancias de Wasserstein	124
Q. Programa para la construcción de una matriz global de distancias entre tipos de ataque	127

R. Programa para la generación de un dendrograma jerárquico a partir de la matriz global de distancias	130
S. Programa para la clasificación de nuevos experimentos y generación de reportes comparativos	133
T. Programa que aplica los criterios de clasificación final y muestra falsos positivos	147
U. Muestra de resultados obtenidos del Anexo T ante un nuevo experimento	154
V. Generación de métricas de evaluación de modelo propuesto	165
Bibliografía	173

Índice de figuras

2.1. Nodos esenciales de OpenStack.	9
2.2. Módulos de OpenStack.	9
3.1. Acciones del grupo - ejemplo del cuadrado.	12
3.2. Subgrupo - ejemplo del cuadrado.	12
3.3. Homomorfismo - ejemplo del cuadrado.	13
3.4. Nucleo e imagen del ejemplo del cuadrado.	13
3.5. Algunos símlices.	15
3.6. Ejemplo de Vietoris-Rips.	16
3.7. Figuras geométricas.	16
3.8. Seis objetos básicos y su número de Betti.	18
3.9. Ejemplo de homotopía.	18
4.1. Pasos de Mapper.	20
4.2. Ejemplo de código de barras de homología persistente [23].	21
4.3. Ejemplo de Diagrama de persistencia.	22
4.4. Ejemplo de las bolas de plastilina.	23
5.1. Diagrama de red - OpenStack.	27
5.2. Agente malicioso y configuración de parámetros.	27
5.3. Ejecución del ataque hacia el nodo Controller.	29
5.4. Cambio de archivos de configuración en Nova y recolección de bitácoras.	31
6.1. Estructura de directorio después de la estandarización.	35
6.2. Reducción de dimensionalidad con el algoritmo PCA.	35
6.3. Reducción de dimensionalidad con el algoritmo UMAP.	36
6.4. Reducción de dimensionalidad con el algoritmo t-SNE.	36
6.5. Generación de grafo global con Mapper.	37
6.6. Extracción de la estructura topológica de los datos recolectados de los ataques. En color azul se muestran grafos pequeños (uno o dos nodos), considerados como ruido debido a su baja conectividad. En color morado se observan componentes conexas formadas por registros de un único tipo de ataque SFI. En color verde se presentan grafos compuestos por múltiples tipos de ataque, evidenciando mayor heterogeneidad en su estructura y composición.	39
6.7. Nube de puntos de grafo global.	40
6.8. Nube de puntos de grafo global formado por más de un tipo de ataque.	40

6.9. Nube de puntos de grafo global formado por más de un tipo de ataque.	41
6.10. Generación de grafos para cada tipo de ataque usando Mapper.	42
6.11. Generación de grafos para cada tipo de ataque usando Mapper.	42
6.12. Ejemplo de Diagrama de persistencia para el vector MCEFL.	43
6.13. Generación de Diagrams de persistencia para cada vector de ataque usando Mapper y los subconjuntos de porcentajes desde el 10 % hasta el 100 %.	43
6.14. Cálculo de la distancia de Wasserstein.	44
6.15. Comportamiento por tipo de ataque - Dimensión 0.	45
6.16. Comportamiento por tipo de ataque - Dimensión 1.	45
6.17. Comportamiento por tipo de ataque - Dimensión 2.	45
6.18. Matriz global de distancias de Wasserstein por cada tipo de ataque SFI.	47
6.19. Jerarquización de los ataques conforme a sus distancia de Wasserstein.	48
7.1. Métricas promedio de <i>precision</i> , <i>recall</i> y <i>F1-score</i> por tipo de ataque SFI para el modelo de clasificación basado en características topológicas.	52
7.2. Distribución del error cuadrático medio por tipo de ataque SFI utilizando diagramas de caja.	53
7.3. Matriz de confusión global del modelo.	54
7.4. Mapa de calor con las métricas globales.	55
7.5. Ranking de la métrica F1-score de los ataques SFI.	56

Acrónimos

1. IaaS: *Infrastructure as a Service*, infraestructura como servicio.
2. PaaS: *Platform as a Service*, plataforma como servicio.
3. SaaS: *Software as a Service*, software como servicio.
4. SDN: *Software-Defined Networking*, redes definidas por software.
5. TDA: *Topological Data Analysis*, análisis topológico de datos.
6. SFI: *Software Fault Injection*, inyección de fallos en software.
7. IDS: *Intrusion Detection System*, sistema de detección de intrusos.
8. IPS: *Intrusion Prevention System*, sistema de prevención de intrusos.
9. HIDS: *Host-based Intrusion Detection System*, sistema de detección de intrusos basado en host.
10. HIPS: *Host-based Intrusion Prevention System*, sistema de prevención de intrusos basado en host.
11. VM: *Virtual Machine*, máquina virtual.
12. CPU: *Central Processing Unit*, unidad central de procesamiento.
13. RAM: *Random Access Memory*, memoria de acceso aleatorio.
14. I/O: *Input/Output*, entrada/salida.
15. PCA: *Principal Component Analysis*, análisis de componentes principales.
16. t-SNE: *t-distributed Stochastic Neighbor Embedding*, técnica de reducción de dimensionalidad no lineal.
17. UMAP: *Uniform Manifold Approximation and Projection*, aproximación y proyección de variedades uniformes.
18. DBSCAN: *Density-Based Spatial Clustering of Applications with Noise*, algoritmo de agrupamiento basado en densidad.
19. MSE: *Mean Squared Error*, error cuadrático medio.
20. CSV: *Comma-Separated Values*, valores separados por comas.
21. API: *Application Programming Interface*, interfaz de programación de aplicaciones.
22. TCP: *Transmission Control Protocol*, protocolo de control de transmisión.
23. IP: *Internet Protocol*, protocolo de Internet.

Resumen

El crecimiento del cómputo en la nube y el uso del paradigma Infraestructura como Servicio (IaaS) han permitido tanto a las organizaciones empresariales como a las académicas acceder a recursos de procesamiento, almacenamiento y redes virtualizadas de manera flexible. Sin embargo, este paradigma introduce nuevos retos en materia de seguridad, particularmente en entornos virtualizados donde las técnicas tradicionales de detección presentan limitaciones, al no considerar que uno de los propósitos más importantes es saltar al hipervisor y tomar control de todo el entorno. En este contexto, ataques como *Software Fault Injection* (SFI) resultan especialmente difícil de detectar, ya que modifican el comportamiento del sistema sin generar indicadores de compromiso a nivel de red o alteración de huellas digitales previamente conocidas.

Ante esta problemática, esta tesis propone un enfoque basado en Análisis Topológico de Datos (TDA) para la caracterización y clasificación de ataques SFI en una nube que provee infraestructura como servicio basada en Red Hat OpenStack, específicamente el ataque se realiza en el servicio de aprovisionamiento de instancias de máquinas virtuales Nova. El desarrollo del trabajo se estructura en distintas etapas. En la primera fase, se ejecutan ataques de tipo SFI sobre el sistema; posteriormente, se realiza la recolección de registros bajo diferentes escenarios de ataque, conformando el conjunto de datos de análisis. En la tercera etapa, se aplican técnicas de reducción dimensional y el algoritmo Mapper para construir representaciones topológicas de los datos utilizando complejos Vietoris-Rips. Con ello, se crean diagramas de persistencia (DP), los cuales permiten extraer las características topológicas principales de los ataques. Subsecuentemente, se construyen perfiles representativos por tipo de ataque y se implementa un esquema de clasificación basado en el error cuadrático medio (MSE) de la distancia de *Wasserstein* como criterio de selección.

Finalmente, a partir de este proceso, se evalúa la efectividad del modelo en la clasificación de ataques SFI, para esto, se emplean diversas métricas y herramientas de análisis que permiten caracterizar de manera integral el desempeño del modelo frente a los distintos tipos de ataques, considerando además el cálculo de la distancia de *Wasserstein* como métrica de similitud. Los resultados obtenidos muestran que el enfoque propuesto permite identificar estructuras topológicas relevantes dentro de los conjuntos de datos analizados, alcanzando una clasificación correcta promedio del 80 % entre los 13 vectores de ataque evaluados, e incluso logrando una eficacia del 100 % en varios de ellos. En este contexto, el uso del *F1-score* junto a otras métricas de evaluación, evidencian la dificultad de clasificación en ciertos tipos de ataques SFI debido a la dispersión y similitud de sus patrones topológicos. En conjunto, estos resultados validan la efectividad del enfoque propuesto y aportan una comprensión más profunda del comportamiento del sistema. Con base en los resultados obtenidos, se proponen líneas de trabajo futuro orientadas a mejorar la capacidad de discriminación y robustez del enfoque propuesto.

Capítulo 1

Introducción

Actualmente, el poder de cómputo en la nube se ha incrementado exponencialmente, esto gracias al uso de redes definidas por software (SDN) que ha permitido que grandes compañías tecnológicas puedan utilizar servicios de procesamiento, redes virtualizadas y respaldo de datos bajo demanda. Existen diversos modelos de computación en la nube, entre los cuales destacan: infraestructura como servicio (IaaS), plataforma como servicio (PaaS) y software como servicio (SaaS).

En el paradigma del cómputo en la nube, la infraestructura se ejecuta mediante instancias gestionadas por un hipervisor. En este entorno, los procesos operan a través de llamadas al *kernel* del hipervisor en lugar de interactuar directamente con los recursos físicos del *hardware*. Esta capa de abstracción transforma la naturaleza operativa del sistema y, en consecuencia, exige que las técnicas de seguridad tradicionales sean adaptadas para abordar las vulnerabilidades específicas de los entornos virtualizados.

Un ejemplo de los sistemas tradicionales son los sistemas de detección de intrusos (IDS) o los sistemas de prevención de intrusos (IPS), que tradicionalmente se implementan como dispositivos dedicados o integrados en un *Firewall* para monitorizar el tráfico de red. Así mismo, tanto en entornos físicos como virtualizados también existen variantes orientadas al host, como los Host-based Intrusion Detection System (HIDS) y los Host-based Intrusion Prevention System (HIPS). A diferencia de los IDS/IPS, estos sistemas analizan eventos internos del sistema, como cambios en archivos críticos, registros del sistema o patrones específicos en el uso de recursos, con el objetivo de identificar actividad maliciosa dentro del *host* sin considerar parámetros de red. Incluso han surgido conceptos como IPS basado en SDN (SDNIPS) que adicional a los sistemas de detección tradicional se agregan métricas de configuración de redes virtuales a través de programas de código abierto y protocolos estándar como **Open vSwitch** y **OpenFlow** [1]. No obstante, a pesar de la robustez de estas herramientas, la infraestructura de IaaS sigue siendo vulnerable, ya que esta debilidad radica en la capa de virtualización: el hipervisor y las APIs de gestión se convierten en nuevos vectores de entrada que los IDS/IPS tradicionales no siempre pueden monitorizar. Además, el dinamismo de las IaaS permite la creación y destrucción de instancias en segundos, lo que a menudo genera brechas de visibilidad y errores de configuración humana que los sistemas de detección no logran mitigar de forma proactiva. En este contexto, la seguridad ya no es solo una cuestión de analizar paquetes o archivos, sino de garantizar la integridad de la orquestación y el aislamiento entre máquinas virtuales.

En los últimos años, el uso de IaaS se ha incrementado considerablemente. Un ejemplo de ello es **OpenStack** que es una solución de código abierto ampliamente utilizada a nivel mundial que facilita la gestión de almacenamiento, procesamiento, creación de máquinas virtuales y conexión de redes virtuales. Además, ha sido adoptada por importantes compañías como **IBM**, **Dell**, **Oracle**, **Ericsson**, entre muchas otras. Gracias a su arquitectura modular, **OpenStack** permite escalabilidad y la capacidad de ajustarse a

diversos entornos, ya sea empresariales o educativos. Sin embargo, también han surgido nuevas amenazas cibernéticas propias de este paradigma. Por ejemplo, ataques de denegación de servicio, que pueden afectar no solo al servicio objetivo sino también a otros servicios alojados en la misma infraestructura física; la explotación de vulnerabilidades de software en redes virtualizadas, como desbordamientos de búfer o fallos en mecanismos de filtrado; ataques contra la capa de virtualización, que pueden permitir la ejecución de código en el hipervisor, el escalamiento de privilegios o el acceso no autorizado a la memoria de otras máquinas virtuales mediante el aprovechamiento de vulnerabilidades propios de los hipervisores más usados: QEMU-KVM, Virtualbox o Xen. Asimismo, existen ataques orientados al robo de información y monopolización de recursos, donde un atacante puede extraer datos sensibles o consumir excesivamente recursos de CPU para degradar el rendimiento de otros servicios. También, se han identificado vulnerabilidades en los componentes de gestión y orquestación de recursos informáticos, las cuales pueden permitir la obtención de datos de otras instancias o el control indebido de la infraestructura virtualizada [2].

Aunado a esto, un ataque poco estudiado y que no es alojado en un *host* mediante el uso de la red dentro de un paradigma IaaS, es el ataque **Software Fault Injection**, que tiene como objetivo principal inyectar fallos dentro de un software que dé como resultado un comportamiento inesperado del sistema abriendo brechas de seguridad que pueden ser aprovechadas por los atacantes, en donde las técnicas de detección previamente mencionadas presentan limitaciones significativas, ya que operan sobre vectores de ataque muy específicos, no monitorizan los cambios en el comportamiento dinámico del sistema o solo basan su detección mediante métricas de red.

Con el fin de minimizar los ataques orientados a inyectar comportamientos maliciosos dentro del software, existen técnicas como: la Aleatorización del Espacio de Direcciones (ASLR), que coloca áreas clave del programa de manera aleatoria con el fin de que un atacante no sepa dónde añadir código malicioso; las *cookies* de pila, que colocan valores de control antes de la dirección de retorno en la pila para detectar si se ha producido un desbordamiento de búfer; la Prevención de Ejecución de Datos (DEP), que marca ciertas regiones de memoria como no ejecutables para impedir que se ejecute código inyectado en áreas destinadas a solo datos, y el Control del Flujo de Ejecución (CFG), que supervisa que el programa solo realice saltos y llamadas a ubicaciones válidas previamente definidas. Si bien estas técnicas no son propias de entornos virtualizados o de entornos SDN, sí son aplicables al software que se ejecuta dentro de las máquinas virtuales. No obstante, se trata principalmente de mecanismos preventivos que supervisan parámetros específicos del sistema, por lo que no siempre proporcionan una visión integral. Por lo tanto, es necesario utilizar técnicas más robustas que permitan analizar variaciones de manera integral en las métricas críticas del estado del sistema, como la cantidad de procesos en ejecución, la memoria usada en *swap*, bloques de disco, uso de CPU, tiempo de operaciones de I/O, tiempo para levantar módulos virtualizados, etc.

Para reducir este problema, este trabajo de tesis propone un algoritmo que permite la detección y clasificación de ataques SFI en la plataforma **OpenStack** mediante el uso de TDA (Topological Data Analysis), un área de la topología algebraica que identifica de manera muy robusta las diferencias entre espacios topológicos. Para ello, se realizan múltiples ataques del tipo Software Fault Injection en el servicio de aprovisionamiento de instancias de cómputo (Nova), componente de **OpenStack** que instancia las máquinas virtuales. Estas afectaciones modifican el comportamiento de las máquinas virtuales, y por ende el funcionamiento de la infraestructura informática.

1.1. Definición del problema

En el contexto de las SDN e infraestructuras virtualizadas, ataques como la ejecución de código dentro de un hipervisor, el escalamiento de privilegios o el acceso no autorizado a la memoria de otras máquinas virtuales son difíciles de prevenir pese a existir técnicas como ASLR, DEP, CFG o **Stack Canaries**, las cuales buscan restringir la ejecución de código malicioso y proteger la integridad de los recursos de memoria tanto en entornos físicos como virtualizados. En este panorama, un ataque de tipo SFI dentro de una IaaS es particularmente difícil detectarlo, ya que la inyección de fallos dentro del programa que instancia máquinas virtuales no necesariamente genera cambios visibles en algún registro particular del sistema ni produce indicadores de compromiso detectables por los sistemas tradicionales de detección de intrusos. Por lo anterior, resulta fundamental analizar el comportamiento global del sistema y detectar una posible actividad maliciosa sin la necesidad de firmas reconocidas, uso de la red o cambios en registros específicos del sistema operativo. Por ello, un enfoque integral del comportamiento del sistema, apoyado de un estudio multidimensional a través de análisis de estructuras topológicas puede proveer una herramienta que permita proteger infraestructuras de cómputo en la nube.

1.2. Alcance de la tesis

Implementación de un sistema de detección de anomalías que permita clasificar mediante el uso de Análisis Topológico de Datos un conjunto de ataque **Software Fault Injection** a través de toma de bitácoras del sistema dentro del servicio de aprovisionamiento de instancias de cómputo Nova de OpenStack.

1.3. Objetivo

Diseñar e implementar un algoritmo capaz de detectar un conjunto de ataques del tipo SFI en progreso con base en las características topológicas de los datos tomados de las bitácoras al instanciar máquinas virtuales dentro de una IaaS.

1.4. Hipótesis

El uso de TDA para la detección de anomalías dentro de una IaaS permite identificar un ataque SFI en el servicio de aprovisionamiento de instancias, y con ello mandar estas alertas a un controlador inteligente que tome acciones.

1.5. Metodología

Para alcanzar los objetivos planteados en este trabajo se sigue la siguiente metodología:

- Implementar una infraestructura de nube basada en **OpenStack**, la cual será utilizada como entorno de pruebas para la generación y recolección de datos del sistema.
- Con base en el artículo [3], generar un conjunto de ataques de tipo **Software Fault Injection** dirigidos al módulo *Nova* de **OpenStack**. Para ello, se emplea un agente externo que ejecuta los ataques sobre el nodo *controller* de la IaaS.

- Durante la ejecución de los ataques, instanciar máquinas virtuales dentro de **OpenStack** y recolectar registros del estado del sistema relacionados con el uso de recursos computacionales, tales como: cantidad de procesos en ejecución, memoria usada en *swap*, memoria libre, memoria utilizada como *buffers*, cantidad de memoria movida entre RAM y *swap*, bloques de disco leídos y escritos, cantidad de interrupciones, cambios de contexto por segundo, porcentaje de CPU usado por procesos de usuario o por el sistema, tiempo de ocio, tiempo en operaciones de I/O y tiempo para instanciar la virtualización.
- Proponer un algoritmo de detección y clasificación de ataque SFI que aproveche la robustez del Análisis Topológico de Datos, utilizando algoritmos como Mapper para transformar los registros del sistema en representaciones topológicas que permitan analizar patrones en los datos.
- A partir del grafo construido con Mapper, calcular invariantes topológicas mediante complejos simpliciales de *Vietoris-Rips* y generar los diagramas de persistencia.
- Dividir los datos obtenidos en conjuntos de entrenamiento y prueba.
- Comparar las estructuras topológicas obtenidas mediante el cálculo de la distancia de Wasserstein entre diagramas de persistencia, con el objetivo de medir la similitud entre los distintos tipos de ataques.
- Finalmente, evaluar la generación de falsos positivos del algoritmo propuesto para clasificar un ataque SFI.

1.6. Contribución y resultados

Esta investigación aborda el problema de detección de ataques SFI desde una perspectiva basada en Análisis Topológico de Datos, utilizando teoría de grupos y topología algebraica como conceptos matemáticos fundamentales orientados al estudio de la estructura y conectividad de los datos generados por las bitácoras del sistema. En otras palabras, este trabajo explora la capacidad que tiene TDA para identificar patrones asociados a distintos vectores de ataques SFI.

A lo largo del desarrollo experimental se observa que la implementación de ataques de Software Fault Injection genera patrones estructurales diferenciables en las bitácoras recolectadas. En este sentido, la aplicación del algoritmo Mapper y de la Homología Persistente proporcionan una herramienta para crear relaciones estructurales entre los eventos registrados, facilitando la identificación de estructuras relevantes dentro de los datos. Asimismo, el uso de la Distancia de Wasserstein permite cuantificar la similitud entre diferentes vectores de ataque, evidenciando que ciertos tipos de inyección de fallos comparten características topológicas comunes. Esto demuestra que TDA no solo permiten analizar la estructura interna de los datos, sino también establece características para su clasificación.

Finalmente, los resultados muestran que el enfoque propuesto provee una alternativa confiable para la detección de ataques SFI, lo que proporciona una metodología capaz de clasificar vectores de ataque SFI dentro de una Infraestructura como Servicio (OpenStack). Además, este trabajo abre la posibilidad de extender estas técnicas hacia modelos automatizados de detección y categorización de ataques.

1.7. Estructura de la tesis

El contenido de esta tesis se organiza de la siguiente manera:

- En el capítulo 2, se presentan los conceptos relacionados a la computación en la nube.
- En el capítulo 3, se presentan los conceptos matemáticos fundamentales de topología algebraica.
- En el capítulo 4, se ofrece la definición de las técnicas utilizadas para el Análisis Topológico de Datos como Mapper, diagramas de Persistencia y distancia de Wasserstein.
- En el capítulo 5, se detalla la implementación del ataque **Software Fault Injection**.
- En el capítulo 6, se explica el algoritmo propuesto para la detección del ataque **SFI**.
- En el capítulo 7, se realiza un análisis de las pruebas bajo diferentes condiciones.
- Finalmente, el capítulo 8, presenta las conclusiones generales, así como las posibles mejoras del método propuesto y trabajos futuros.

1.8. Anexos

El contenido de los anexos adicionados en este trabajo de tesis se organizan de la siguiente manera:

- El Anexo A y B muestran las configuraciones y comandos ejecutados para el escenario base de las pruebas, en específico el servicio de **IaaS**.
- Del Anexo C al F se detallan configuraciones del programa **FIT4Python**, utilizado para la ejecución del ataque **SFI**.
- Del Anexo G al I se muestran las implementaciones de los programas para la recolección, segregación y limpieza de los registros recolectados de las máquinas virtuales con inyección de fallos.
- Del Anexo J al M se muestran los programas para el preprocesamiento de los registros recolectados, como la división porcentual de los datos y su estandarización.
- El Anexo N muestra el programa para la construcción del grafo global de los datos mediante el algoritmo Mapper.
- El Anexo Ñ muestra el programa para la generación de diagramas de persistencia mediante complejos de Vietori-Rips.
- El Anexo O y P muestran el programa para el cálculo de la distancia de Wasserstein entre diagramas de persistencia.
- El Anexo Q y R muestran los programas para el agrupamiento de los tipos de ataque conforme a sus distancia de Wasserstein.
- El Anexo S muestra el programa para la clasificación de los ataques **SFI**.
- El Anexo T y U muestran los programas para la identificación de falsos positivos y los resultados obtenidos para cada tipo de ataque.
- El Anexo V muestra el programa para la generación de métricas de evaluación final.

Capítulo 2

Cómputo en la nube

En este capítulo se abordan los conceptos necesarios para la comprensión de una infraestructura de nube, una infraestructura como servicio (IaaS), así como la plataforma de código abierto **OpenStack** utilizada en este proyecto de tesis.

2.1. Computación en la nube

El cómputo en la nube, de acuerdo a NIST [4], es un modelo que permite el acceso bajo demanda a una red con recursos informáticos, tales como el procesamiento, almacenamiento, ejecución de aplicaciones, etc., que pueden ser administrados con una mínima interacción, teniendo las siguientes características esenciales:

- **Autoservicio bajo demanda:** El cliente puede aprovisionar recursos informáticos sin la necesidad de la interacción con un tercero.
- **Acceso mediante red:** La infraestructura informática y sus componentes deben ser accesibles mediante el uso de Internet.
- **Agrupación de recursos:** Los recursos informáticos se agrupan para servir a múltiples consumidores, con la capacidad de reasignarse dinámicamente según su demanda.
- **Escalabilidad:** Permite expandir de forma rápida las capacidades técnicas del sistema.
- **Servicio medido:** El uso de los recursos de cómputo se monitorizan, controlan y reportan de forma continua, con el fin de tener un modelo de pago por uso.

De esta forma, se otorgan servicios específicos a las necesidades del cliente, dejando una gran parte de la gestión a las empresas tecnológicas que cuentan con esta capacidad. Existen diversos modelos de computación en la nube; sin embargo, los más destacados son:

- **Software como servicio (SaaS):** Modelo con la capacidad de proveer aplicaciones. Estas puede ser accesibles desde dispositivos clientes, ya sea a través de un navegador web o una interfaz de programa sin la necesidad de administrar componentes de red, procesamiento o almacenamiento [4]. Algunos ejemplos populares de SaaS son **Slack**, **Canva** o **Zoom**.

- **Plataforma como servicio (PaaS):** Este modelo permite a desarrolladores elegir bajo demanda los componentes necesarios para ejecutar, desarrollar y gestionar aplicaciones sin la complejidad de mantener los servicios de red, almacenamiento y bases de datos de forma local [5]. Plataformas como GitHub, Docker o Microsoft Azure App Service son ejemplos de PaaS.
- **Infraestructura como servicio (IaaS):** Modelo que proporciona acceso bajo demanda de procesamiento, almacenamiento, redes o cualquier recurso computacional fundamental que permita al cliente escalar los recursos con base a sus necesidades sin la adquisición de *hardware* o inversión inicial en una infraestructura informática de forma local [5]. Ejemplos que se destacan en IaaS son Amazon Web Services (AWS), Google Compute Engine (GCE) y OpenStack.

2.2. OpenStack

OpenStack es una plataforma IaaS que permite aprovisionamiento de recursos de cómputo dentro de un centro de datos a través del uso de API [6]. Esta plataforma permite a las grandes empresas tecnológicas que cuentan con una inversión grande de infraestructura, construir y administrar nubes privadas para sus clientes, con la flexibilidad de gestionar los recursos acorde a sus necesidades. Los componentes principales de OpenStack son:

- **Keystone:** Servicio de autenticación y autorización. Gestiona la identidad y permisos de los usuarios.
- **Glance:** Servicio de gestión de imágenes. Permite almacenar, recuperar y gestionar las imágenes de los sistemas operativos usados para crear instancias de máquinas virtuales.
- **Neutron:** Servicio de gestión de redes. Proporciona redes virtuales, direcciones IP, encaminamiento, balanceo de carga, creación de *switches* y *routers* virtuales, entre otras.
- **Nova:** Servicio de cómputo. Se encarga de aprovisionar, gestionar y automatizar la creación de máquinas virtuales (instancias). Para ello, utiliza hipervisores como XEN y KVM.
- **Horizon:** Interfaz gráfica web de *OpenStack*. Proporciona una interfaz de usuario para gestionar la infraestructura de *OpenStack* de manera visual.

OpenStack ofrece una implementación de código abierto, respaldada principalmente por Red Hat Linux, que permite trabajar con cualquier proveedor [7]. Además, de acuerdo a compañías líderes en ciberseguridad como Fortinet [8], OpenStack ofrece a las empresas y usuarios: agilidad empresarial, optimización de procesos, disponibilidad de recursos y adaptabilidad a la infraestructura informática. Beneficios que aprovechan diversas compañías de distinta índole, como: Walmart, American Airlines, AT&T, NASA, Volkswagen, entre muchas más [9].

2.2.1. Estructura de OpenStack

OpenStack requiere de al menos dos nodos esenciales, los cuales se denominan *Controller* y *Compute*. Para esta tesis, la creación de estos nodos y la instalación de OpenStack se detallan en el Anexo A. El nodo *Controller* es el encargado de ejecutar y administrar servicios de identidad, imágenes, recursos de cómputo y redes dentro de la infraestructura informática proveyendo gran parte de la administración de la IaaS. Mientras que el nodo *Compute* adopta las funciones de un hipervisor, encargado de operar

los recursos para la creación de máquinas virtuales, también ejecuta agentes de red para conectar las máquinas virtuales entre sí. La figura 2.1 muestra la arquitectura básica de OpenStack, donde KVM es el hipervisor utilizado por OpenStack que funge como el *host* principal donde se alojarán las máquinas virtuales (VM).

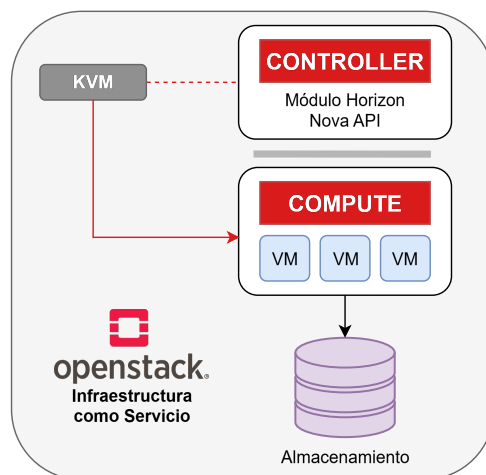


Figura 2.1: Nodos esenciales de OpenStack.

OpenStack segmenta su arquitectura mediante la implementación de módulos que se encargan de un aspecto específico, de entre los cuales destacan Nova, Neutron, Glance, Keystone y Horizon. Cada uno de estos expone una API que les permite comunicarse entre sí. La figura 2.2, muestra los servicios principales y otros servicios adicionales como Swift y Manila. Sobre esta base operan módulos adicionales de orquestación como infraestructura como código (Heat), manejo de contenedores (Magnum) o gestión de alta disponibilidad (Masakari). Además, existen módulos de monitorización, optimización y herramientas de despliegue como Kolla-Ansible, OpenStack-Ansible, con el fin de escalar la solución de infraestructura y ajustarse a los requerimientos del cliente.

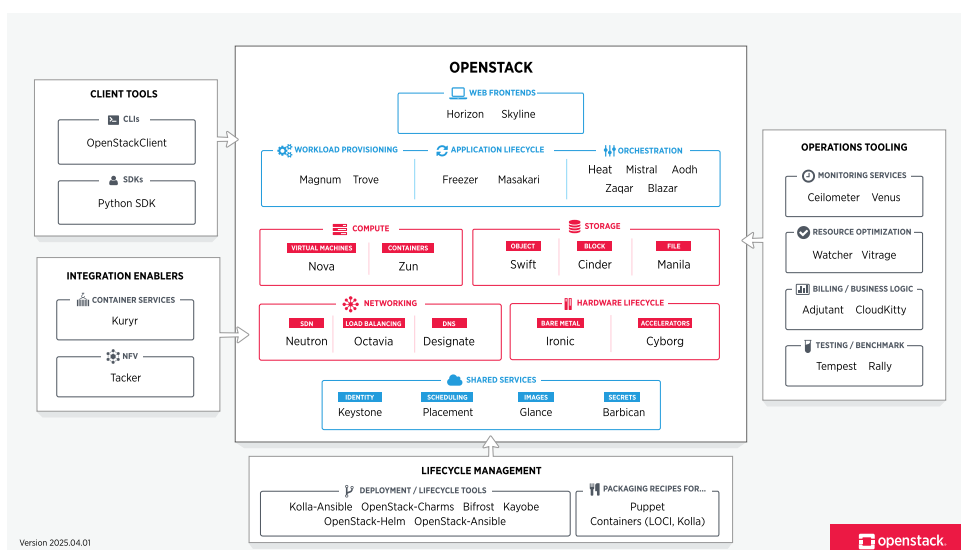


Figura 2.2: Módulos de OpenStack.

Capítulo 3

Marco teórico (Topología algebraica)

Este capítulo establece los fundamentos matemáticos para el desarrollo de la metodología empleada en este trabajo de tesis. Específicamente, se abordan algunas técnicas asociadas al análisis de datos de alta dimensión. Posteriormente, se introducen conceptos fundamentales de teoría de grupos, los cuales permiten comprender las estructuras algebraicas utilizadas en distintos procesos topológicos. Finalmente, se presentan los principios de la topología algebraica, incluyendo nociones de homología, homotopía y complejos simpliciales, las cuales permiten estudiar propiedades globales y relaciones estructurales dentro de los datos.

3.1. Análisis de datos de alta dimensión

3.1.1. Datos de alta dimensión

Se considera que un conjunto de datos es de alta dimensión cuando cada instancia se describe mediante un número elevado de variables o características, generando espacios multidimensionales difíciles de representar, interpretar y procesar computacionalmente. Desde una perspectiva geométrica, en espacios de alta dimensión, los datos suelen encontrarse muy concentrados, provocando que las métricas de distancia pierdan capacidad para diferenciar adecuadamente puntos cercanos y lejanos. Esto afecta directamente a numerosos algoritmos de clasificación, agrupamiento y reducción dimensional, los cuales dependen de relaciones de vecindad o similitud. Asimismo, el incremento en el número de variables puede introducir correlaciones poco significativas que dificultan la interpretación de la información. Debido a estas limitaciones, se han desarrollado técnicas que permitan representar los datos en espacios de menor dimensión, conservando en la medida de lo posible, sus propiedades estructurales más relevantes.

3.1.2. Técnicas de reducción dimensional

La reducción dimensional (RD) puede ser aplicada a cualquier conjunto de datos que tenga una dimensión alta con el fin de visualizar y modelar los datos de manera más sencilla y comprensible [10]. Estas técnicas han sido mayormente utilizadas en disciplinas como *Machine Learning*, reconocida por el uso de datos extensivos con alta dimensión, que a través de este tipo de técnicas, encuentran una forma viable para analizar los datos con la menor alteración posible. A continuación, se mencionan algunos ejemplos de estas técnicas [11]:

- **Integración de vecinos estocásticos distribuidos en t - TSEN:** Es un algoritmo de reducción

de dimensionalidad no lineal, diseñado para la visualización de datos de alta dimensión. El método opera convirtiendo las distancias euclidianas entre puntos en el espacio original en probabilidades de similitud, utilizando una distribución normal para el espacio de alta dimensión y una distribución t-Student en el espacio de baja dimensión. Su objetivo principal es preservar la estructura local de los datos, garantizando que observaciones cercanas en el espacio original mantengan dicha proximidad en la representación reducida, lo que facilita la identificación de agrupamientos (clusters).

- **Análisis de Componentes Principales - PCA:** Es una técnica de transformación lineal utilizada para la reducción de dimensionalidad mediante la proyección de un conjunto de datos multivariados hacia un nuevo sistema de coordenadas ortogonales. Este método se fundamenta en la descomposición de la matriz de covarianza (o de correlación) para extraer los autovectores y autovalores del sistema; los autovectores resultantes constituyen los componentes principales, los cuales representan direcciones ortogonales en las que la varianza de los datos es máxima.
- **Aproximación y Proyección de Variedad Uniforme - UMAP:** El algoritmo parte de la premisa de que los datos se distribuyen a lo largo de una estructura geométrica subyacente de alta dimensión, denominada *manifold*. UMAP estima la topología de esta estructura calculando las relaciones de vecindad local y las extiende para capturar la forma global de los datos. A través de un proceso de optimización, el método proyecta esta estructura en un espacio de baja dimensión, buscando preservar la conectividad y las distancias relativas entre los puntos. Esto permite que, a diferencia de otras técnicas, UMAP sea muy eficiente en mantener la separación entre grupos (clusters) distantes.

Para este trabajo de tesis se utiliza PCA, esta técnica demostró, en nuestras simulaciones, una mejor reconstrucción de los datos ante otros métodos [12].

3.2. Teoría de grupos

La teoría de grupos proporciona el marco algebraico fundamental para el estudio de las simetrías y de las estructuras que surgen en diversas áreas de las matemáticas, en particular para este trabajo en la topología algebraica. A partir de la noción de grupo como un conjunto dotado de una operación binaria que satisface las propiedades de cerradura, asociatividad, existencia de elemento neutro e inversos, es posible formalizar conceptos clave como subgrupos, homomorfismos, núcleos e imágenes. Estos conceptos permiten describir de manera rigurosa cómo las estructuras algebraicas se relacionan entre sí y cómo ciertas propiedades se preservan bajo ciertos parámetros.

3.2.1. Grupo

Un **Grupo** es una estructura algebraica definida por la pareja $(G, \cdot)$, la cual consiste en un conjunto no vacío G y una operación binaria (o ley de composición interna) $\cdot : G \times G \rightarrow G$ que satisface los siguientes axiomas:

$$a \cdot (b \cdot c) = (a \cdot b) \cdot c \quad \forall a, b, c \in G \quad (\text{Asociatividad}) \quad (3.1)$$

$$\exists e \in G \text{ tal que } e \cdot g = g = g \cdot e, \quad \forall g \in G \quad (\text{Existencia del elemento identidad}) \quad (3.2)$$

$$\forall g \in G, \exists h \in G \text{ tal que } g \cdot h = e = h \cdot g \quad (\text{Existencia del elemento inverso}). \quad (3.3)$$

Si $(G, \cdot)$ es un grupo con la propiedad de que $g \cdot h = h \cdot g$ para todo par de elementos $g, h \in G$, diremos que $(G, \cdot)$ es un grupo abeliano o conmutativo [13].

De manera intuitiva, supóngase que se tiene un cuadrado dibujado en una hoja con el cual puede realizar las siguientes acciones: No hacer nada, girarlo 90° , girarlo 180° , girarlo 270° y reflejarlo.

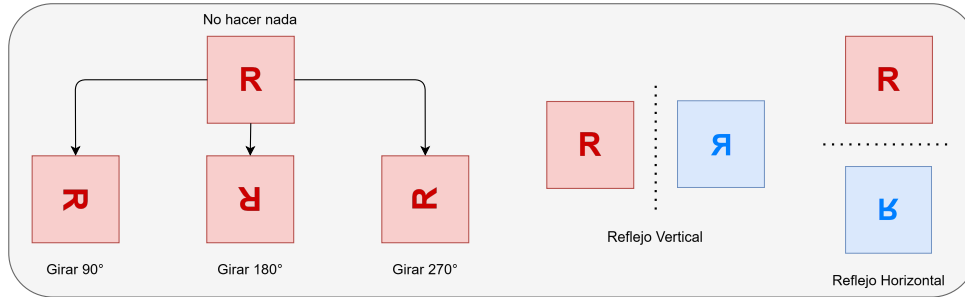


Figura 3.1: Acciones del grupo - ejemplo del cuadrado.

Este conjunto de acciones forma un grupo ya que cumple las propiedades antes mencionadas, por ejemplo:

- Cerradura: Si giras 90° y reflejas $\implies$ otra acción de la lista.
- Elemento neutro: No hacer nada.
- Inverso: Un reflejo se deshace con el mismo reflejo.
- Asociatividad: $(\text{girar } 90^\circ + \text{girar } 90^\circ) + \text{girar } 90^\circ = \text{girar } 90^\circ + (\text{girar } 90^\circ + \text{girar } 90^\circ)$.

3.2.2. Subgrupos

Sean $(G, \cdot)$ un grupo y H un subconjunto de G . Diremos que H es un subgrupo de G (con respecto a la operación $\cdot$), y escribiremos $H \leq G$, si H es grupo bajo (la restricción a $H \times H$ de) la operación $\cdot$, es decir, si las siguientes condiciones son satisfechas simultáneamente [13]:

$$\text{para cualesquier elemento } a, b \in H, \text{ el producto } a \cdot b \text{ es un elemento de } H \quad (3.4)$$

$$\text{existe un elemento } e_H \in H \text{ tal que } e_H \cdot h = h = h \cdot e_H \text{ para todo } h \in H \quad (3.5)$$

$$\text{para todo } h \in H \text{ existe } h' \in H \text{ con la propiedad de que } h \cdot h' = e_H = h' \cdot h. \quad (3.6)$$

Retomando el ejemplo anterior del cuadrado dibujado, un subgrupo puede ser entonces las siguientes acciones: girarlo 180° y no hacer nada.

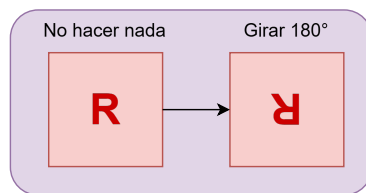


Figura 3.2: Subgrupo - ejemplo del cuadrado.

3.2.3. Homomorfismos

Sean G y H dos grupos. Un homomorfismo de grupos de G a H es una función $\varphi : G \rightarrow H$ que satisface $\varphi(xy) = \varphi(x)\varphi(y)$ para todo par de elementos $x, y \in G$. Adicionalmente [13],

se define como **kernel** de φ al conjunto $\ker(\varphi) = \{g \in G \mid \varphi(g) = e\} \subseteq G$,

se define como **imagen** de φ al conjunto $\text{Im}(\varphi) = \{\varphi(g) \mid g \in G\} = \{h \in H \mid \exists g \in G \text{ tal que } \varphi(g) = h\} \subseteq H$.

Retomando el ejemplo del cuadrado, supongase que G es el grupo de simetrías de un cuadrado (rotaciones y reflexiones) y que H es el grupo $\{1, -1\}$, donde 1 significa que mantiene la orientación y -1 significa que se invierte la orientación.

Si se tiene una función φ que a cada simetría del cuadrado le asigne 1 si es una rotación (conserva orientación) y -1 si es una reflexión (invierte la orientación). Entonces, esta función es un homomorfismo, ya que si se hacen dos movimientos seguidos, el efecto sobre la orientación se comporta multiplicativamente:

- Rotación con rotación sigue siendo rotación ($1 \cdot 1 = 1$).
- Reflexión con reflexión termina siendo rotación ($-1 \cdot -1 = 1$).
- Rotación con reflexión da como resultado reflexión ($1 \cdot -1 = -1$).

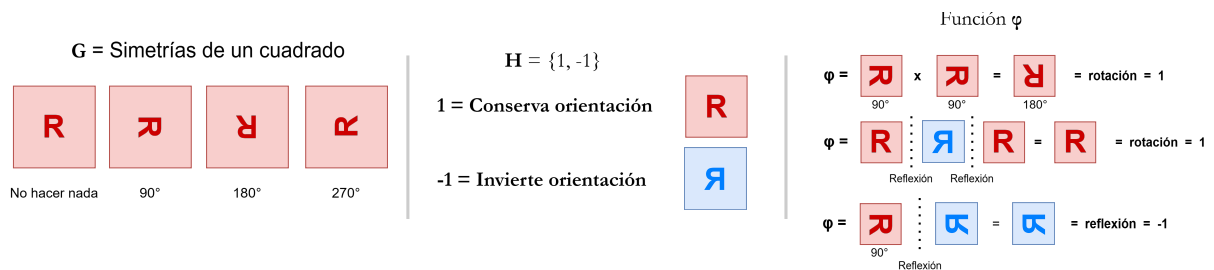


Figura 3.3: Homomorfismo - ejemplo del cuadrado.

De esta forma, el núcleo de φ son todas las rotaciones del cuadrado (acciones que mantienen la orientación), y la imagen es $\{1, -1\}$, porque solo hay dos posibles resultados: conservar o invertir la orientación.

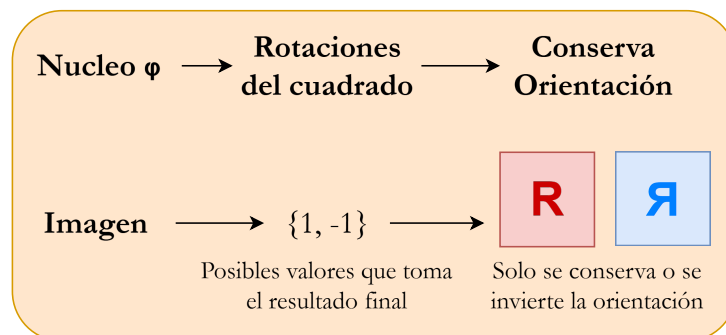


Figura 3.4: Nucleo e imagen del ejemplo del cuadrado.

De forma intuitiva, el homomorfismo no considera el detalle exacto del movimiento, solo conserva la información esencial sobre si se invierte o no la figura.

3.3. Topología algebraica

La topología algebraica es la rama de las matemáticas que emplea herramientas del álgebra abstracta para estudiar, clasificar y cuantificar las propiedades globales de los espacios geométricos que permanecen invariantes ante deformaciones continuas. Su objetivo central es asociar estructuras algebraicas, como grupos u homomorfismos, a dichos espacios para transformar problemas geométricos complejos en sistemas algebraicos [14].

3.3.1. Espacios topológicos

Un espacio topológico se describe como el par (X, T) donde X es un conjunto y T es una colección de subconjuntos de X tal que [15]:

$$\emptyset \in T \text{ y } X \in T.$$

Para cada colección infinita $\{O_\alpha\}_{\alpha \in A} \subset T$, se tiene que $\bigcup_{\alpha \in A} O_\alpha \in T$.

Para cada colección finita $\{O_i\}_{1 \leq i \leq n} \subset T$, se tiene que $\bigcap_{1 \leq i \leq n} O_i \in T$.

Con lo anterior se entiende que X es un conjunto abierto, en el que tanto la unión y la intersección con cualquiera otra colección de conjuntos abiertos, seguirá siendo un conjunto abierto. Este concepto permite adentrar nociones de continuidad, convergencia y cercanía sin necesidad de una estructura métrica.

A pesar de que un espacio topológico convencional (X, T) provee el marco teórico idóneo para analizar propiedades invariantes, su naturaleza abstracta y continua lo vuelve poco manejable para el cómputo digital. Dado que las computadoras exigen representaciones discretas y finitas para almacenar y procesar información. Por lo tanto, surge la necesidad de transitar hacia la topología combinatoria mediante el uso de complejos simpliciales. Estas estructuras actúan como un puente metodológico fundamental, ya que preservan las propiedades topológicas globales del espacio continuo original pero las codifican en objetos finitos discretizados. La necesidad de transitar hacia los complejos simpliciales trasciende la dicotomía entre espacios continuos y discretos, fundamentándose rigurosamente en la finitud computacional. Incluso si un espacio topológico (X, T) posee una naturaleza discreta, como ocurre con los conjuntos infinitos numerables, su infinitud estructural lo vuelve algebraicamente inmanejable para la memoria y capacidad de procesamiento de un ordenador actual. Por lo tanto, el uso de complejos simpliciales resulta indispensable como un mecanismo de truncamiento y modelado combinatorio finito; estas estructuras permiten aproximar y codificar las invariantes topológicas globales de un espacio infinito mediante un número finito de bloques de construcción (simplices). Al hacerlo, transforman la topología teórica en un problema de complejidad computacional acotada, y resoluble.

3.3.2. Símpex o Símpice

El n -símpice se define como la envoltura convexa de un conjunto de $n + 1$ puntos linealmente independientes en un espacio euclidiano $\mathbb{R}^k$ (donde $k \geq n$). Estos puntos constituyen los vértices del símpice.

Asimismo, una cara de Δ^n es cualquier sub-símpice de dimensión inferior formado por un subconjunto de sus vértices [16]. Ejemplos de símpices son (ver figura 3.5) :

- Un 0-simplex es un punto.
- Un 1-simplex es un segmento de recta (dos puntos unidos).

- Un 2-simplex es un triángulo (tres puntos no colineales).
- Un 3-simplex es un tetraedro (cuatro puntos no coplanarios).

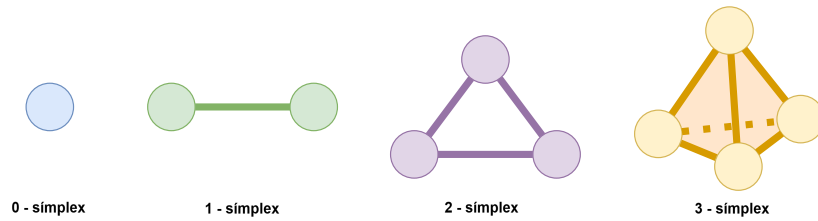


Figura 3.5: Algunos símplexes.

3.3.3. Complejos simpliciales

Un complejo simplicial K es un conjunto finito de símplexes en $\mathbb{R}^n$ que garantiza la integridad estructural mediante dos condiciones fundamentales [16]:

1. **Inclusión de caras:** Si un símplex σ pertenece a K , entonces cada una de sus caras $\alpha \subseteq \sigma$ también es un elemento de K .
2. **Intersección:** Para cualquier par de símplexes $\sigma_1, \sigma_2 \in K$, su intersección $\sigma_1 \cap \sigma_2$ es vacía o es una cara común para ambos.

En términos combinatorios, un complejo simplicial se representa como una familia de subconjuntos no vacíos de un conjunto de vértices $V = \{v_0, \dots, v_k\}$, donde cada subconjunto representa un símplex y la estructura preserva todas las relaciones de pertenencia entre ellos.

3.3.4. Complejos de Vietoris-Rips

Dado un conjunto finito de puntos X en un espacio métrico (X, d) , y un escalar $\epsilon \geq 0$, el complejo de Vietoris-Rips denotado como $\mathcal{R}_\epsilon(X)$, es el complejo simplicial cuyas caras se definen bajo la siguiente condición: Un subconjunto de puntos $\{x_0, x_1, \dots, x_k\}$ forma un k -símplex si y solo si la distancia entre cada par de puntos es menor o igual a ϵ :

$$x_0, \dots, x_k \in \mathcal{R}_\epsilon(X) \iff d(x_i, x_j) \leq \epsilon, \quad \forall i, j \in 0, \dots, k. \quad (3.7)$$

Desde una perspectiva geométrica, el complejo de Rips es una estructura de proximidad que aproxima la topología subyacente de la nube de puntos. El parámetro ϵ actúa como un umbral de conectividad: valores pequeños de ϵ revelan detalles locales y ruido, mientras que valores mayores permiten la formación de símplexes de dimensiones superiores, permitiendo identificar características globales como agujeros o cavidades. No obstante, un incremento excesivo en ϵ conlleva un aumento exponencial en la complejidad computacional, debido al crecimiento del número de símplexes formados. La figura 3.6 muestra un ejemplo de cómo evolucionan la conexión de los complejos simpliciales conforme aumenta el escalar ϵ .

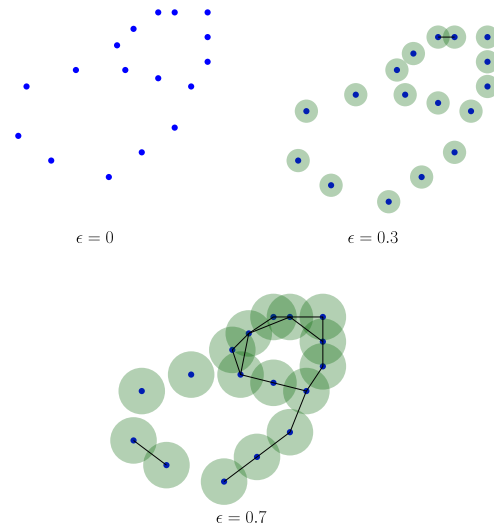


Figura 3.6: Ejemplo de Vietoris-Rips.

3.3.5. Homología

Considere una superficie como una hoja de papel, un aro o una esfera. Desde un punto de vista intuitivo, cada una puede ser distinguida por la presencia de agujeros o cavidades: una hoja no contiene ningún agujero, un aro presenta una cavidad cerrada o agujero a lo largo del aro, y una esfera cuenta con una cavidad interna (ver figura 3.7). Haciendo una analogía con la homología, esta proporciona bases matemáticas para identificar y clasificar estas características estructurales, permitiendo distinguir objetos no por su forma o representación en tres dimensiones, sino por la cantidad y tipo de componentes (conexas, ciclos, cavidades, etc) que se presentan en distintas dimensiones.

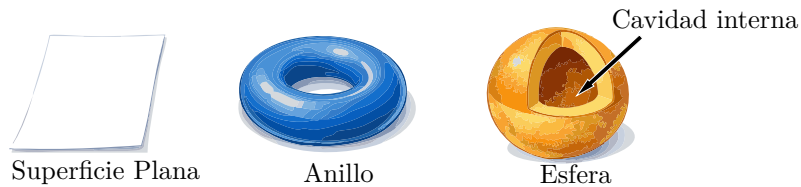


Figura 3.7: Figuras geométricas.

3.3.5.1. Grupos de Homología

La homología puede verse como el lenguaje algebraico utilizado para identificar y clasificar agujeros en un complejo simplicial K . Este proceso se puede formalizar mediante los siguientes conceptos [16]:

1. Grupo de n -cadenas (C_n): Representa la suma formal de todos los n -símplices presentes en el complejo K , ponderados por coeficientes (g_i) en un grupo abeliano (generalmente $\mathbb{Z}$). Una n -cadena $c = \sum g_i \sigma_i$ puede entenderse como una sub-estructura del complejo donde se seleccionan ciertos símplices y se les asigna una orientación.
2. Operador de frontera (∂_n): Se define el homomorfismo $\partial_n : C_n \rightarrow C_{n-1}$, donde este operador descompone un objeto de dimensión n en los elementos de dimensión $n - 1$ que lo limitan. Para un

n -símplice orientado, la frontera se calcula como:

$$\partial_n([v_0, \dots, v_n]) = \sum_{i=0}^n (-1)^i [v_0, \dots, \hat{v}_i, \dots, v_n], \quad (3.8)$$

donde el término $(-1)^i$ garantiza que las caras mantengan una orientación coherente. Una propiedad fundamental en topología es que la frontera de una frontera es siempre nula ($\partial_n \circ \partial_{n+1} = 0$), lo que significa que las estructuras que delimitan un objeto sólido no tienen, a su vez, una frontera propia.

3. Ciclos (Z_n) y Fronteras (B_n): A través del operador ∂ , se clasifican las cadenas en dos categorías fundamentales:

- **Ciclos ($Z_n = \ker(\partial_n)$):** Cadenas cuya frontera es cero. Representan estructuras cerradas o lazos que no tienen principio ni fin.
- **Fronteras ($B_n = \text{im}(\partial_{n+1})$):** Ciclos que son el borde de una estructura de dimensión superior.

La esencia de la homología reside en que todo borde es un ciclo, pero no todo ciclo es un borde. Un ciclo que no delimita un objeto de dimensión superior representa un agujero real en los datos. Así, el n -ésimo grupo de homología se define como el grupo cociente:

$$H_n(K) = Z_n(K) / B_n(K). \quad (3.9)$$

3.3.5.2. Números de Betti

A partir de la definición de ciclo en un complejo simplicial K , se establece que dos n -ciclos, z_1 y z_2 , son homólogos si su diferencia es la frontera de una estructura de dimensión superior ($n+1$). Geométricamente, esto implica que ambos ciclos delimitan una misma cavidad en el espacio topológico. El conjunto de todas las clases de equivalencia de estos ciclos conforma el n -ésimo grupo de homología, $H_n(K)$. De acuerdo con Edelsbrunner y Harer [17], la estructura de estos grupos se puede resumir mediante un valor numérico denominado el n -ésimo número de Betti (β_n). Este se define formalmente como el rango del grupo de homología correspondiente [18]:

$$\beta_n = \text{rank}(H_n). \quad (3.10)$$

En términos de análisis de datos, β_n representa el número de agujeros n -dimensionales linealmente independientes; por ejemplo, β_0 indica el número de componentes conexas, β_1 el número de túneles o ciclos unidimensionales, y β_2 el número de vacíos o cavidades volumétricas. La figura 3.8 muestra ejemplos del rango de los grupos de homología.

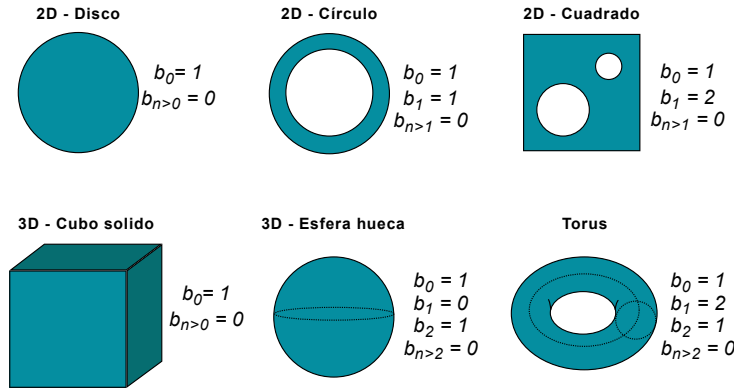


Figura 3.8: Seis objetos básicos y su número de Betti.

3.3.6. Homotopía

Sean f y g dos funciones continuas entre los espacios topológicos X e Y . Se dice que f es homotópica a g (denotado como $f \simeq g$) si existe una familia de funciones continuas que transforma gradualmente a f en g . Formalmente, una homotopía es una aplicación continua $H : X \times [0, 1] \rightarrow Y$ tal que para todo $x \in X$ se satisfacen las condiciones de contorno:

$$H(x, 0) = f(x) \quad \text{y} \quad H(x, 1) = g(x).$$

Intuitivamente, el parámetro $t \in [0, 1]$ puede interpretarse como el tiempo, donde $H(x, t)$ representa una deformación continua que, al transcurrir de $t = 0$ a $t = 1$, deforma la función f hasta convertirla en g sin romper la estructura de los espacios [19].

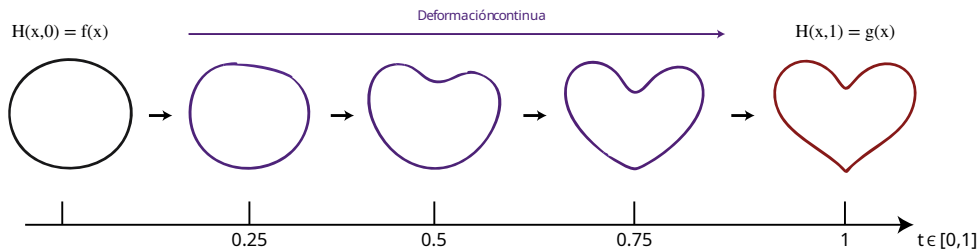


Figura 3.9: Ejemplo de homotopía.

En resumen, en topología algebraica, dos funciones o espacios se consideran homotópicos cuando es posible transformar uno en otro de manera continua, sin realizar cortes ni uniones. Por ejemplo, en la figura 3.9, se muestra una circunferencia inicial que progresivamente se va deformado. Cada una de estas etapas representa un estado continuo de la deformación, preservando la conectividad de la figura durante todo el proceso. Con lo anterior se hace notar que la Homología y la Homotopía integran dos conceptos centrales de la topología algebraica orientados al estudio de las invariantes topológicas. La homotopía se ocupa del análisis de equivalencias topológicas definidas mediante deformaciones continuas entre espacios, permitiendo determinar cuándo dos estructuras pueden considerarse equivalentes desde un punto de vista topológico. En contraste, la homología proporciona un enfoque algebraico para la caracterización de un espacio mediante la identificación de propiedades como componentes conexas, ciclos y cavidades de distintas dimensiones, asociando dichas estructuras a grupos algebraicos que facilitan su estudio y clasificación.

Capítulo 4

Análisis Topológico de Datos

El enfoque de TDA se basa en la idea de que estructuras topológicas de un conjunto de datos pueden revelar patrones, agrupamientos y relaciones intrínsecas que permanecen ocultas para otros métodos de análisis. Entre las técnicas más relevantes de esta área destaca el algoritmo Mapper, el cual permite construir representaciones gráficas simplificadas de datos de alta dimensionalidad, genera un grafo que facilita la visualización y exploración de la estructura global de los datos, lo que permite identificar regiones con alta densidad y conexiones significativas.

Otra herramienta fundamental dentro este campo es la Homología Persistente, una metodología que estudia la aparición y desaparición de característica topológicas como componentes conexas, ciclos y cavidades (grupos de homología) en diferentes dimensiones. Así, esta técnica proporciona una descripción robusta de la información topológica presente en los datos. La comparación entre estas estructuras topológicas requiere métricas capaces de cuantificar diferencias entre representaciones derivadas de conjuntos de datos, en este contexto, la distancia de Wasserstein es una herramienta esencial para medir la similitud entre conjuntos de puntos y diagramas de persistencia.

En este capítulo se presentan los fundamentos del Análisis Topológico de Datos y se estudian tres de sus herramientas más representativas: el algoritmo Mapper, la Homología Persistente y la distancia de Wasserstein. Se describen sus fundamentos teóricos, procedimientos de construcción y aplicaciones en el análisis de datos complejos, destacando cómo estas herramientas permiten extraer información relevante y su vez aportan nuevas perspectivas para la comprensión de datos de alta dimensión.

4.1. Algoritmo de Mapper

El algoritmo Mapper se ha consolidado como una de las herramientas más potentes para la exploración y visualización de estructuras complejas en conjuntos de datos de alta dimensión. A diferencia de las técnicas de reducción de dimensionalidad tradicionales, Mapper genera una representación simplificada en forma de grafo que captura la forma de los datos, preservando patrones críticos de conectividad, agrupamiento y ramificación. Esta metodología permite obtener una abstracción global que revela estructuras no lineales, como lazos o bifurcaciones, que a menudo resultan invisibles para los métodos estadísticos convencionales. Mapper opera sobre un conjunto de datos X , y su salida se determina mediante los siguientes pasos [20]:

- **Función de filtro (lente):** Se define una función continua $f : X \rightarrow Y$, donde Y representa un espacio de referencia de baja dimensión. Esta función actúa como una lente que proyecta los datos

de alta dimensión, y puede basarse en medidas estadísticas, geométricas o topológicas como la densidad, excentricidad, centralidad o proyecciones tipo PCA.

- **Recubrimiento del espacio de imagen:** Se selecciona una colección finita de conjuntos abiertos $\mathcal{U} = \{U_\alpha\}_\alpha \subseteq Y$ tal que cubren la imagen del filtro, $f(X) \subseteq \bigcup_\alpha U_\alpha$. Es fundamental que estos conjuntos presenten traslapes entre sí, ya que este solapamiento es el que permitirá capturar la continuidad de los datos en el grafo final.
- **Preimagen y descomposición local:** Para cada conjunto abierto U_α , se considera su preimagen bajo el filtro, $f^{-1}(U_\alpha)$. Debido a que f es continua, esta preimagen agrupa puntos de X que tienen valores de filtro similares.
- **Clustering y refinamiento:** Se aplica un algoritmo de agrupamiento (como DBSCAN o clustering jerárquico) a cada preimagen $f^{-1}(U_\alpha)$ de forma independiente. Esto particiona cada preimagen en componentes disjuntos o clústeres locales, denotados como $\{C_{\alpha,\beta}\}_\beta$. La colección total de estos clústeres, $\mathcal{C} = \{C_{\alpha,\beta}\}_{\alpha,\beta}$, constituye el *recubrimiento refinado* del espacio original X .
- **Construcción del Grafo (Complejo de Nervio):** El algoritmo Mapper construye un grafo no dirigido $G = (V, E)$ que representa el nervio del recubrimiento refinado:
 - El conjunto de vértices V contiene un nodo por cada clúster $C_{\alpha,\beta}$.
 - El conjunto de aristas E conecta dos vértices si y solo si sus clústers correspondientes tienen una intersección no vacía en el espacio original: $C_{\alpha_1,\beta_1} \cap C_{\alpha_2,\beta_2} \neq \emptyset$ [21]. La figura 4.1 muestra los pasos de Mapper en un conjunto de puntos X , primero se proyectan los puntos en un espacio de baja dimensión, posteriormente se elige el recubrimiento, se concentran los datos mediante un algoritmo de clustering y finalmente se construye un grafo $G = (V, E)$.

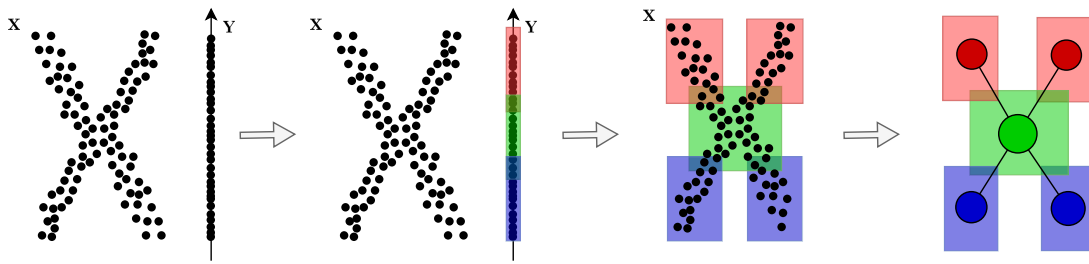


Figura 4.1: Pasos de Mapper.

4.2. Homología persistente

La homología persistente estudia la evolución y estabilidad de las características homológicas a través de una jerarquía de escalas. La idea principal es capturar qué rasgos topológicos (invariantes topológicas) son intrínsecos a la estructura de los datos y cuáles son producto del ruido o de la discretización.

Dado un complejo simplicial K , una filtración se define como una secuencia ordenada y anidada de subcomplejos $\{K_r\}_{r \in \mathbb{R}}$, estructurados de la siguiente manera:

$$\emptyset = K_0 \subseteq K_1 \subseteq K_2 \subseteq \cdots \subseteq K_n = K, \quad (4.1)$$

donde cada elemento K_i constituye, por sí mismo, un complejo simplicial contenido en el complejo total. La relación de inclusión ($\subseteq$) exige una propiedad de herencia combinatoria: si un simplejo σ pertenece a un subcomplejo K_i , entonces todas las caras de σ deben pertenecer a K_i , y éste coexistirá en todos los subcomplejos subsecuentes de la secuencia $(K_{i+1}, K_{i+2}, \dots)$. Esta familia de complejos se construye de manera sistemática a partir del filtro $f : K \rightarrow \mathbb{R}$. Bajo este esquema, cada componente indexado de la filtración está determinado por la preimagen de la función para un umbral real r [22]:

$$K_r = f^{-1}((-\infty, r]) = \{\sigma \in K \mid f(\sigma) \leq r\}. \quad (4.2)$$

Esto garantiza que, a medida que el parámetro de control r varía de $-\infty$ a $+\infty$, los simplejos se incorporen de forma indexada. La unión indexada de todos los elementos finitos que conforman la filtración recupera en su totalidad al complejo simplicial global, es decir, $\bigcup_r K_r = K$. Este crecimiento monótono permite observar y registrar la conectividad del espacio topológico que se ensambla gradualmente, y de esta manera se puede evaluar la persistencia de sus características.

Una clase de homología o característica topológica $\gamma \in H_n(K_p)$ se dice que *nace* en el subcomplejo K_p (asociada a un valor de filtro o tiempo de nacimiento $b = f(\sigma)$ para algún simplejo σ) si no se encuentra en la imagen del homomorfismo inducido por la inclusión previa, es decir, $\gamma \notin \text{im}(f_n^{p-1,p})$. Por otro lado, se dice que dicha característica *muerde* al entrar al subcomplejo K_q (asociada a un valor de filtro o tiempo de muerte $d = f(\tau)$) si su imagen bajo el homomorfismo inducido se vuelve nula al convertirse en la frontera de una estructura de dimensión superior $(n+1)$, o bien, si se fusiona con una clase homológica más antigua en $H_n(K_q)$. Toda evolución de estas estructuras a lo largo de la filtración se resume de manera indexada en el conjunto de intervalos semiabiertos $\mathcal{B} = \{[b_i, d_i)\}$, el cual puede representarse mediante:

- **Código de barras:** Donde cada intervalo se grafica como una línea horizontal. Las barras de mayor longitud representan la topología intrínseca de K .

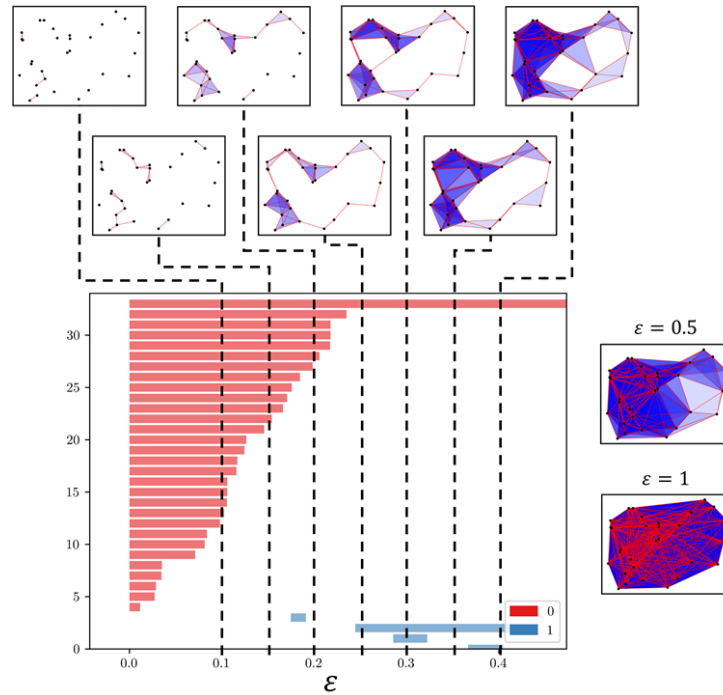


Figura 4.2: Ejemplo de código de barras de homología persistente [23].

En la figura 4.2 se muestra un ejemplo de código de barras persistente, en este se señalan con líneas punteadas verticales los intervalos de ϵ , mostrando las características topológicas presentes al variar su valor dentro del complejo simplicial durante la filtración. Las barras rojas representan los intervalos de persistencia para β_0 , y las azules, para β_1 . Se puede ver que persisten en el tiempo dos barras, una componente conexa $\beta_0 = 1$ (roja) y un agujero ($\beta_1 = 1$, azul).

- **Diagrama de persistencia:** Donde cada par (b_i, d_i) es un punto en el plano. La estabilidad de estos diagramas garantiza que pequeñas variaciones en la nube de puntos X no alteren significativamente la firma topográfica resultante.

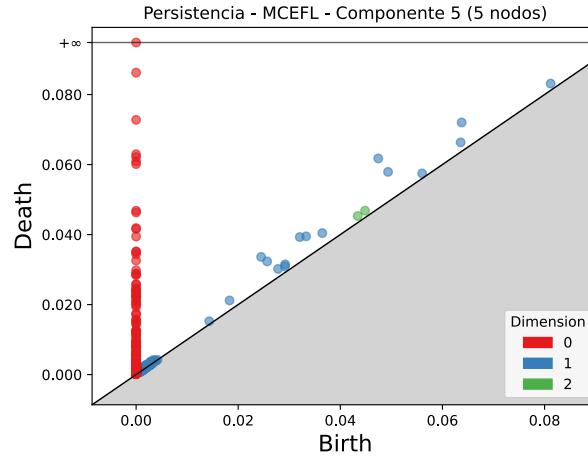


Figura 4.3: Ejemplo de Diagrama de persistencia.

La figura 4.3 muestra un ejemplo de diagrama de persistencia. En esta representación, ϵ_{min} se traza en el eje x y ϵ_{max} en el eje y , así, las características topológicas son representadas por los puntos (b_i, d_i) , donde la distancia vertical con respecto a la línea $y = x$ es análoga a la longitud del intervalo; es decir, cuanto más alejado esté un punto de la línea $y = x$, más persistente será la característica.

4.3. Similitud entre conjunto de puntos

4.3.1. Distancia de Wasserstein

Teniendo dos diagramas de persistencia con puntos $D_1 = \{(b_1^i, d_1^i)\}_i$ y $D_2 = \{(b_2^j, d_2^j)\}_j$ donde b_n^i codifica el índice donde nace una característica topológica, y d_n^j el índice donde dicha característica muere. Se dice que, la distancia q-Wasserstein, W_q , está definida como el costo de correspondencia óptimo entre los dos conjuntos de puntos de los diagramas de persistencia [24]:

$$W_q(D_1, D_2) := \inf_{\gamma: D_1^* \rightarrow D_2^*} \left(\sum_{x \in D_1^*} \|x - \gamma(x)\|_\infty^q \right)^{1/q},$$

donde $D_i^* := D_i \cup \{(x, x) : x \in [0, \infty)\}$, y $\gamma: D_1^* \rightarrow D_2^*$ son los rangos ínfimos sobre todas las aplicaciones biyectivas. En otras palabras, la distancia de Wasserstein permite cuantificar la estabilidad entre dos diagramas de persistencia, es decir, que pequeñas perturbaciones en el sistema original generará pequeñas variaciones en las posiciones de sus puntos [25]. De este modo, esta métrica proporciona un criterio robusto

para comparar las firmas topológicas de los datos, garantizando que el ruido no altere significativamente los resultados del análisis.

De manera intuitiva, supóngase que se tienen dos tableros que representan dos diagramas de persistencia, donde cada bola de plastilina colocada en ellos corresponde a una característica topológica (un punto con coordenadas de nacimiento y muerte). El objetivo es transformar la configuración del primer tablero para replicar exactamente la del segundo. Para lograrlo, se debe tomar cada bola del primer tablero y desplazarla hasta la posición de una bola en el segundo tablero. Además, debido a que la cantidad de puntos entre ambos diagramas puede diferir, se permite la opción de hacer desaparecer una bola aplastándola directamente contra la línea diagonal del tablero ($y = x$), o bien, hacer nacer una nueva bola desde dicha diagonal.

Bajo este esquema, cuanto mayor sea la distancia que se deba mover o deformar cada bola de plastilina, mayor será el esfuerzo o trabajo realizado. Así, la distancia de Wasserstein cuantifica el costo total mínimo necesario para reorganizar por completo el primer conjunto de bolas hasta hacer que coincida con el segundo, encontrando el emparejamiento óptimo y más eficiente entre las estructuras topológicas de ambos sistemas. La figura 4.4 muestra un ejemplo físico del cómputo de Wasserstein.

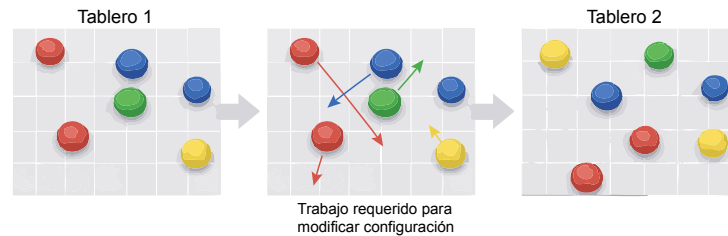


Figura 4.4: Ejemplo de las bolas de plastilina.

En esta tesis, el uso de la distancia de Wasserstein permitirá obtener el grado de similitud entre diferentes diagramas de persistencia mediante [13]. Esta métrica identifica formalmente el costo computacional que implica transformar las características topológicas representadas en un diagrama hacia otro; por lo tanto, a menor distancia de Wasserstein, mayor será la homología entre ambos diagramas, lo que indica que las estructuras subyacentes de sus datos son topológicamente idénticas. Lo anterior, nos permite tener un criterio de clasificación robusto para el comportamiento de un sistema informático.

Capítulo 5

Implementación del ataque Software Fault Injection

En este capítulo se introducen los tipos de ataques cibernéticos, así como las herramientas más usadas para su detección. Se describe el proceso de implementación del ataque de Software Fault Injection. También, se detallan las etapas necesarias para su ejecución y evaluación. Asimismo, se explica la preparación del entorno de trabajo, y el proceso de recolección y análisis de bitácoras generadas durante la ejecución del ataque.

5.1. Ataques cibernéticos

Dentro de la ciberseguridad cualquier tipo de actividad maliciosa que intente recopilar, interrumpir, denegar, degradar o destruir los recursos del sistema de información se contempla como un ataque [6]. En algunos casos, los agentes maliciosos hacen uso de un conjunto de diversas técnicas para poder perpetrar un activo informático de manera más sofisticada, dificultando su detección por métodos tradicionales. A continuación, se presentan algunos tipos de ataques [26]:

- **Ataques a contraseñas:** Uso de diversas técnicas y herramientas para atacar credenciales como son los ataques por fuerza bruta o ataques de diccionario.
- **Ataques por ingeniería social:** Conjunto de técnicas dirigidas a los usuarios con el objetivo de revelar información personal o de acceso que permita al atacante hacer mal uso de esta información o tomar el control de activos informativos, ejemplos de este tipo de ataque son: *phishing*, *spam* o *baiting*.
- **Ataques a las conexiones:** Uso de diversas herramientas de software para evadir las medidas de seguridad implementadas dentro de una infraestructura informática a nivel de red que permita tomar el control o infectar los activos dentro de la red interna, ejemplos populares de ataques a las conexiones son: *DNS spoofing*, Denegación de Servicios (DoS), escaneo de puertos o inyección SQL.
- **Ataques por *malware*:** Programas o archivos maliciosos cuya finalidad consiste en dañar o alterar el comportamiento habitual de un sistema informático, así como la información contenida en este, atentando contra la privacidad del usuario con el fin de que el atacante obtenga el control de los activos o algún beneficio económico. Dentro de este tipo de ataque se destacan los virus, troyanos,

keyloggers, *ransomware* e inclusive *Software Fault Injection*. Un ejemplo crítico de esta vulnerabilidad son los ataques de inyección de *prompts* dirigidos a Modelos de Lenguaje de Gran Escala (LLMs). Al no existir una distinción robusta entre el *system prompt* y el *input* del usuario, un atacante puede inyectar comandos que alteran la lógica de ejecución del modelo. Esto puede provocar un cambio de las restricciones de seguridad y la ejecución de tareas imprevistas. Este fenómeno se clasifica técnicamente como un ataque de tipo SFI, ya que aprovecha vulnerabilidades en la lógica de validación para forzar estados de error o comportamientos anómalos, afectando directamente la confiabilidad e integridad del sistema [27].

En esta tesis, nos enfocamos específicamente en el ataque SFI, que tiene como objetivo alterar el funcionamiento del sistema sin dejarlo inutilizable; además, no busca propagarse mediante la red y no cuenta con firmas o *hashes* que permitan identificar el *malware* de forma automática como lo hacen los antivirus. De esta manera, debido a la dificultad para identificar este tipo de ataque y que su comportamiento varía de acuerdo con el entorno en el que se encuentre, se vuelve una amenaza crucial para entornos como las IaaS, ya que puede ocasionar una degradación en los recursos informáticos, impactando en el rendimiento, disponibilidad y confiabilidad del usuario, sin mencionar que da origen a brechas de seguridad que pueden ser aprovechadas por un agente malicioso y aprovecharse de la infraestructura y la información contenida en esta.

5.2. Herramientas de detección

Las grandes compañías tecnológicas que desarrollan equipos de seguridad perimetral o **Firewalls** como **Fortinet** basan su detección de amenazas mediante el uso de herramientas de detección de intrusos catalogados en dos grandes vertientes: Sistemas de Detección de Intrusos (IDS) y Sistemas de Prevención de Intrusos (IPS). Los IDS ayudan a la detección de amenazas a través del tráfico de red, mientras que los IPS permiten actuar de forma automática para prevenir un incidente mayor [28]. Ambas basan su funcionamiento en la definición de reglas que, mediante el ajuste de ciertos parámetros como el contenido del paquete, banderas *tcp*, filtros de IP, entre otros, permiten analizar los paquetes de red que atraviesan el sistema. De esta forma, cuando un paquete cumple con los parámetros establecidos, se emite una alerta en el caso de los IDS o se bloquea en el caso de los IPS. Este tipo de herramientas forman parte de equipos de seguridad perimetral especializados, ejemplos de ellos son **snort** y **suricata**. El funcionamiento de este tipo de sistemas se basan en un umbral estadístico que se define con base en la experiencia del administrador o de umbrales predefinidos por instituciones u organizaciones en ciberseguridad. Sin embargo, esto suele ocasionar la generación de falsos positivos (FP). Estudios revelan que alrededor del 90 % de los FP no tienen relación alguna con eventos de seguridad en la red [29].

La mayor parte de la literatura sobre ataques de tipo Software Fault Injection se realizan a nivel de programación de bajo nivel, es decir, se enfocan primordialmente en la alteración sobre la lógica y el comportamiento en el *hardware*, ejemplos de ello son: Intel SGX [30], ataques a dispositivos IoT [31] y *software* embebido [32] diseñado para sistemas de *hardware* específico. También, existen ataques SFI enfocados en aplicaciones basadas en cifrado homomórfico (FHE) [33] o *daemons* del sistema como *kfingerd*, *httpd*, *wu-ftp* y *pop3d* [34]. Sin embargo, no existe una contramedida que pueda aplicarse de forma general, dada las particularidades de cada sistema o software, ya que se vuelve muy complejo identificar cuando una falla es ocasionada por un error en la programación o por un ataque.

Otra forma de ataque SFI consta en alterar funciones de visualización que no tienen algún efecto en la salida del programa. No obstante, un atacante mediante el análisis de las entradas del usuario puede

violar la seguridad de la aplicación o materializar una amenaza como la saturación de búfer [34]. Por todo lo anterior, detectar y abordar contramedidas que puedan dar solución a este tipo de ataques es vital en la seguridad de los sistemas de cómputo.

5.3. Ataque Software Fault Injection

Comúnmente, este ataque es utilizado para probar la tolerancia a fallos mediante la introducción deliberada de sentencias no propias dentro de un sistema [35]. Sin embargo, esta técnica puede ser aprovechada por agentes maliciosos para inyectar fallos, dando como resultado un comportamiento anormal del software. Como base para este trabajo se toma el ataque `FIT4Python` tomado de [3]. El código se encuentra abierto y es utilizado para crear un ataque SFI dentro de la IaaS de `OpenStack` en su versión `Ussuri`, específicamente en el componente `Nova`. El ataque `FIT4Python` genera 15 diferentes tipos de SFI con la siguiente clasificación:

- MCEFL: Missing caught exceptions for function call.
- MIA: Missing if construct around statements.
- MPFC: Missing parameter in function call.
- MRS: Missing return statement.
- MSC: Missing Super Class constructor call.
- MSFC: Missing Synchronization calls.
- MVIE: Missing variable initialization using an expression.
- MVIV: Missing variable initialization using a value.
- WIDS: Wrong string in initial data.
- WIDSL: Wrong string in initial data - missing one char.
- WLEC: Wrong logical expression used as branch condition.
- WSUT: Wrong data types or conversion used.
- WVAE: Wrong arithmetic expression used in assignment.
- WVAL: Wrong logical expression used in assignment.
- WVIV: Wrong value used in variable initialization.

La ejecución del ataque SFI dentro de este trabajo de investigación se fundamentó en la herramienta de código abierto `FIT4Python` [3]. Este software permite crear un entorno de inyección de fallas (SFI) sobre el servicio de cómputo `Nova` dentro de la infraestructura como servicio (IaaS) de `OpenStack` (versión `Ussuri`). Los detalles del proceso de despliegue de los componentes, así como la configuración de red, asignación de puertos y topología de interconexión, se detallan en el Anexo B. La figura 5.1 muestra un resumen de la configuración de la IaaS utilizada.

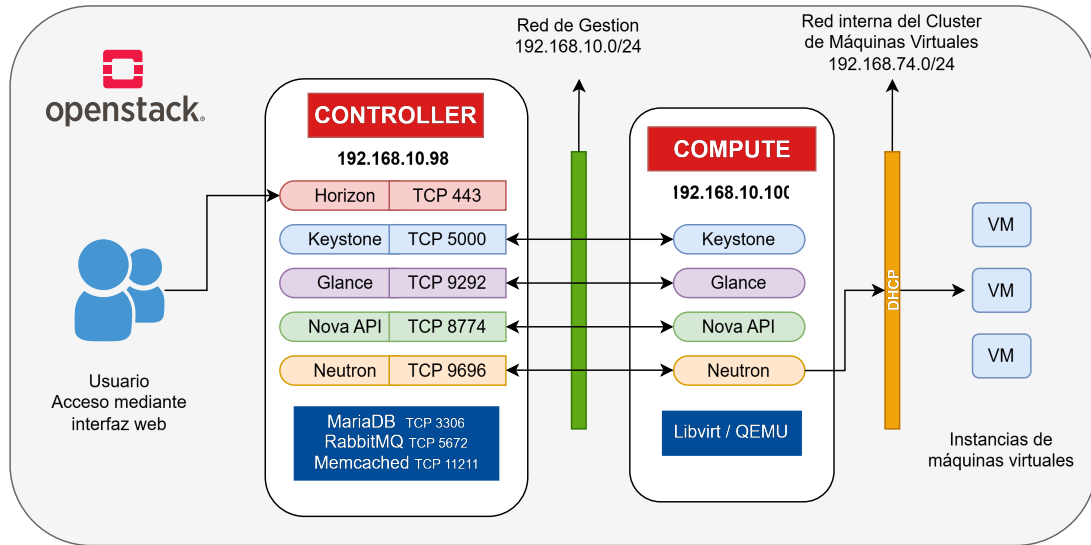


Figura 5.1: Diagrama de red - OpenStack.

5.4. Ejecución del programa FIT4Python

Con el objetivo de detallar de manera clara las actividades involucradas en la ejecución del ataque SFI, se desarrollaron pseudocódigos que proporcionan una representación algorítmica y estructurada del procedimiento. Como parte de la preparación en la herramienta FIT4Python, se modificaron variables críticas en sus archivos de configuración, integrando los parámetros operativos y las credenciales de autenticación correspondientes al nodo **Controller**. La especificación detallada de estos parámetros y sus valores se incluye en el Anexo C.

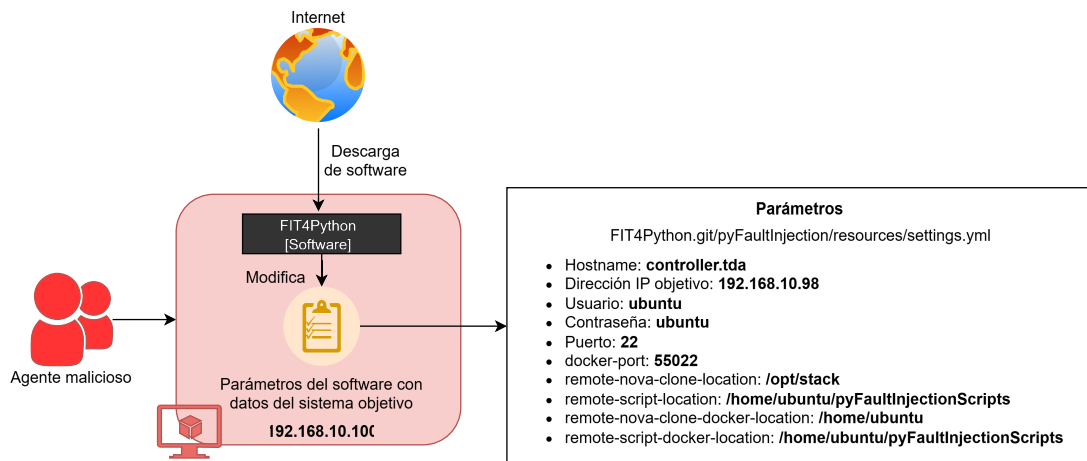


Figura 5.2: Agente malicioso y configuración de parámetros.

La figura 5.2 ilustra el bloque de inicialización del Agente malicioso descrito en las líneas 1 a 3 del algoritmo 1, donde se descarga y configura el software FIT4Python con los parámetros del sistema objetivo, en este caso, el módulo Nova del nodo **Controller**.

Algorithm 1 Inicio de ataque SFI**Entrada:** Configuración del sistema objetivo, nodo controller**Salida:** Archivos API del módulo Nova modificados

- 1: Crear máquina virtual adicional como agente malicioso
- 2: Dentro del agente, descargar y configurar el software FIT4Python para el ataque SFI
- 3: Ejecutar FIT4Python
- 4: **Si** acceso a la máquina objetivo **Entonces**
- 5: Dentro del agente malicioso
- 6: FIT4Python Instala Docker
- 7: FIT4Python Crea un contenedor
- 8: FIT4Python Obtiene el archivo API del módulo Nova original
- 9: FIT4Python Realiza modificaciones al archivo API respecto a las clasificaciones definidas
- 10: **Fin Si**
- 11: Generación de archivos API modificados
- 12: Se insertan los archivos modificados en el nodo controller

Dentro de la modificación al archivo FIT4Python.git/pyFaultInjection/resources/settings.yml se destacan los siguientes parámetros:

- **User y password:** Se parte de la suposición que se conocen las credenciales de un usuario al sistema operativo del nodo **Controller**, mismas que pudieron ser filtradas u obtenidas mediante otro tipo de ataque, los cuales no se detallan en este trabajo de tesis.
- **host:** Dirección IP del objetivo, en este caso, el nodo **Controller**.
- **remote-script-location:** Ruta donde se insertarán los archivos modificados de la API de Nova dentro del nodo **Controller**.
- **remote-nova-clone-docker-location y remote-script-docker-location:** Parámetros necesarios para la correcta ejecución y manejo de variables dentro del contenedor de Docker para la ejecución del software FIT4Python.

La figura 5.3 ilustra el flujo de ejecución de la herramienta FIT4Python, en ella se ilustra la ejecución de la línea 4 a la 12 del Algoritmo 1. Al iniciar el ataque, el agente malicioso realiza el despliegue de un contenedor **Docker**. Este entorno aislado permite descargar el archivo fuente principal del módulo Nova con el fin de crear las modificaciones en el API. Como resultado, se generan múltiples archivos alterados con base en la taxonomía de fallas propuesta por [3] (detallada en el Capítulo 2). Finalmente, estos componentes se inyectan en el nodo **Controller** de **OpenStack** a través de una conexión **SSH**.

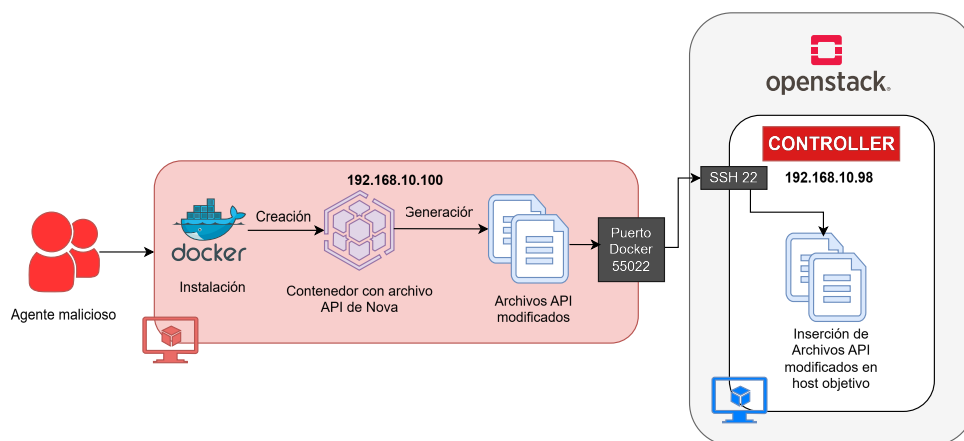


Figura 5.3: Ejecución del ataque hacia el nodo Controller.

Al ejecutar la herramienta FIT4Python, el flujo principal inicia dentro del archivo `__main__.py`, donde se inicializan las configuraciones previamente establecidas para el nodo **Controller**. Posteriormente, la ejecución formal del ataque SFI se desencadena mediante la llamada al método `InjectionManager(settings).run(args.resume)`, tal como se documenta en el Anexo D.

En el manejador de inyecciones (`injection_manager`), documentado en el Anexo E, se observa que en la línea 257 se invoca la función `FaultFactory.create_fault_injector`. Este método mapea la ejecución de los vectores de ataque localizados en el directorio `FIT4Python.git/faults/`, el cual aloja los scripts en Python encargados de realizar las modificaciones específicas sobre las funciones del código objetivo.

A modo de ejemplo, el Anexo F detalla las modificaciones para el ataque de tipo WLEC dentro del archivo `wlec_fault.py`. Específicamente, entre las líneas 36 y 45 se implementa la alteración de las estructuras condicionales (`in`, `not in`, `==`, `!=`, `>`, `<`, `<=` y `>=`), sustituyéndolas por su contraparte lógica o por valores aleatorios. Estos cambios representan las inyecciones de código que caracterizan al ataque SFI bajo la clasificación WLEC.

De este modo, las modificaciones se ejecutan de manera dinámica con base en los vectores de ataque definidos en los archivos de configuración inicial (Anexo C). Al habilitar la totalidad de las clasificaciones disponibles, correspondientes a los ataques MPFC, MIA, WLEC, MVIE, MRS, MCEFL, MVIV, MSFC, MVAV, WVIV, WVAL, WIDS, WVAE, WIDSL, WSUT y MSC, el entorno genera un total de 2,774 archivos fuentes modificados para la simulación de fallas.

La documentación del programa FIT4Python se encuentra en el sitio: <https://git.dei.uc.pt/henmar/FIT4Python/tree/master> y el uso detallado del mismo en el artículo “Injecting software faults in Python applications. The OpenStack case study” [3].

5.5. Recolección de bitácoras del sistema

A partir de los 2,774 archivos fuentes modificados en el nodo **Controller**, se efectúa una segmentación en función de la clasificación del ataque. Para este propósito, se implementó un *script* desarrollado en *bash* (documentado en el Anexo G), el cual automatiza y organiza las bitácoras en directorios independientes asociados a cada vector de ataque SFI. Como resultado, se obtiene una estructura jerárquica de directorios que permite la identificación, gestión y posteriormente la ejecución cronológica de las pruebas experimentales.

Para la fase de experimentación, se desarrolló un *script* que automatiza la evaluación masiva y secuencial de las variantes del archivo `api.py` (por cada vector de ataque), perteneciente al servicio `nova-api`. El flujo operativo completo de esta herramienta se documenta en el Anexo H. El programa procesa de manera cíclica cada una de las versiones alteradas, ejecutando de forma sistemática la siguiente serie de acciones:

- **Sustitución del archivo.** Se reemplaza el archivo `api.py` original del servicio por la versión modificada correspondiente. Este proceso se detalla en las líneas 41 a 52 del programa.
- **Reinicio del servicio.** Una vez realizado el reemplazo del archivo, se reinicia el servicio `nova-api` con el fin de aplicar los cambios al *daemon* de Linux. Este proceso se describe en las líneas 54 a 57.
- **Creación de una máquina virtual de prueba.** Posteriormente, se crea una máquina virtual con fines de validación utilizando la imagen del sistema operativo `Cirros`. Este paso permite registrar alteraciones en el comportamiento del sistema ante la modificación realizada. Este proceso se detalla en las líneas 61 a 69.
- **Medición del rendimiento del sistema.** Durante la ejecución de la máquina virtual se realiza la monitorización del rendimiento del sistema mediante el comando `vmstat`, como se describe en las líneas 71 a 75.
 - En este punto se define el intervalo de tiempo durante el cual se recopilan las métricas del sistema. Para el presente trabajo de tesis se consideró un tiempo mínimo de tres minutos, el cual se estimó suficiente para que la máquina virtual completara su proceso de arranque y alcanzara un estado de funcionamiento estable.
- **Eliminación de la máquina virtual.** Una vez concluida la recolección de métricas, la máquina virtual creada para la prueba es eliminada con el fin de preparar el entorno para la siguiente iteración. Este proceso se describe en las líneas 77 a 84.
- **Clasificación automática de resultados.** Finalmente, el archivo evaluado se clasifica automáticamente. Si el proceso se ejecuta correctamente, el archivo se mueve a la carpeta denominada “*ya vistos*”; en caso de presentarse algún fallo, el archivo se mueve a la carpeta “*extraños*”. Este proceso se detalla en las líneas 87 a 91.

Este procedimiento se repite para cada uno de los vectores de ataques `SFI`. De manera general, el flujo completo del proceso automatizado se representa en el Algoritmo 2.

Cada ejecución del proceso permite recolectar métricas del sistema mediante el comando `vmstat`. Entre los datos obtenidos se encuentran la cantidad de procesos en ejecución, la memoria utilizada en *swap*, la memoria libre disponible, la memoria utilizada como *buffers*, la cantidad de memoria transferida entre la memoria RAM y *swap*. Asimismo, registros de métricas relacionadas con las operaciones de entrada y salida, como los bloques de disco leídos y escritos; indicadores del comportamiento del procesador, incluyendo el número de interrupciones y cambios de contexto por segundo, el porcentaje de uso de CPU por procesos de usuario y del sistema, el tiempo de inactividad del procesador, el tiempo de espera por operaciones de entrada/salida y el tiempo destinado a la virtualización.

Como se muestra en el Anexo H, en caso de que ocurra algún error durante la creación de la máquina virtual o si esta tarda más de cinco minutos en generarse, el archivo correspondiente se descarta automáticamente y se clasifica dentro de la categoría de “*extraños*”. Esta clasificación indica que la modificación

realizada provoca una alteración significativa en el sistema, impidiendo que el servicio funcione de manera adecuada. A partir de este proceso, se identificó que los ataques MPFC y MVIE resultan inviables, ya que la modificación a los archivos son tan grandes que las VM no son capaces de inicializarse. Por lo tanto, estos vectores de ataque no se tomaron en cuenta dentro de este trabajo de tesis.

De manera general, el proceso de sustitución del archivo de configuración, ejecución de pruebas y recolección de métricas se ilustra en la figura 5.4.

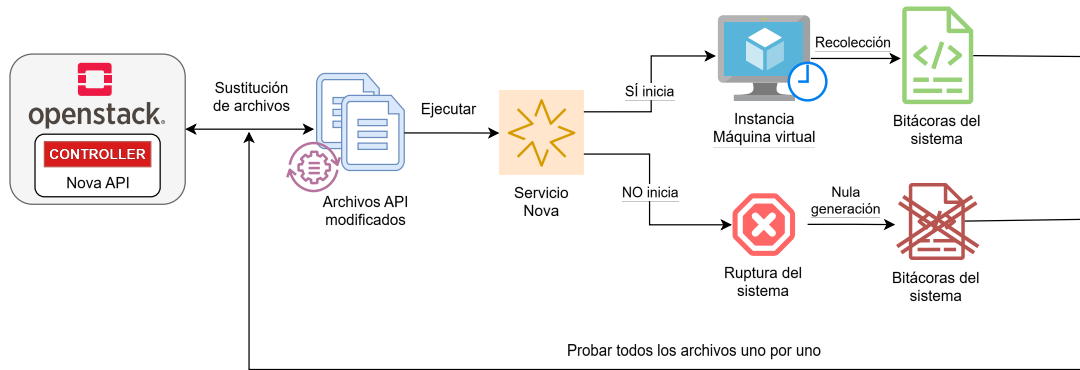


Figura 5.4: Cambio de archivos de configuración en Nova y recolección de bitácoras.

Algorithm 2 Procesamiento automatizado de archivos API modificados

Entrada: Directorio con archivos API organizados por prefijos

Salida: Archivos clasificados como procesados exitosos o fallidos y métricas registradas

```

1: Detener el servicio Nova-API
2: Si error al detener el servicio Entonces
3:   Registrar error global
4:   Finalizar ejecución
5: Fin Si
6: Para cada carpeta de prefijo (tipo de ataque SFI) en el directorio de modificados Hacer
7:   Para cada archivo.py en la carpeta actual Hacer
8:     Copiar archivo a la ruta del servicio Nova
9:     Si error en la copia Entonces
10:      Registrar error
11:      Mover archivo a carpeta de extraños
12:      Continuar con el siguiente archivo
13:   Fin Si
14:   Reiniciar servicio Nova
15:   Si error en el reinicio Entonces
16:     Registrar error
17:     Mover archivo a carpeta de extraños
18:     Continuar
19:   Fin Si
20:   Esperar 15 segundos
21:   Crear máquina virtual de prueba con timeout de 5 minutos
22:   Si error o timeout en la creación Entonces
23:     Registrar error
24:     Mover archivo a carpeta de extraños
25:     Continuar
26:   Fin Si
27:   Ejecutar monitoreo de recursos por al menos 180 segundos, usando el comando vmstat, y
   guardar resultados
28:   Si error en monitoreo Entonces
29:     Registrar error
30:     Mover archivo a carpeta de extraños
31:     Continuar
32:   Fin Si
33:   Obtener lista de instancias activas
34:   Si existen instancias Entonces
35:     Eliminar instancias
36:     Si error al eliminar Entonces
37:       Registrar error
38:       Mover archivo a carpeta de extraños
39:       Continuar
40:     Fin Si
41:   Fin Si
42:   Mover archivo a carpeta de ya vistos (Caso de éxito)
43:   Si error al mover Entonces
44:     Registrar error
45:     Mover archivo a carpeta de extraños
46:     Continuar
47:   Fin Si
48: Fin Para
49: Fin Para
50: Mostrar mensaje de finalización del proceso

```

Capítulo 6

Modelo para la detección del ataque SFI

En el presente capítulo se describe el modelo propuesto para la detección del ataque *SFI*, detallando de manera sistemática las etapas metodológicas que conforman el proceso de análisis y clasificación. Inicialmente, se aborda la estandarización de los datos, con el propósito de homogenizar las características presentes en las bitácoras y garantizar la consistencia en su tratamiento. Posteriormente, se presenta un análisis comparativo entre distintas técnicas de reducción de dimensionalidad y el enfoque basado en Análisis Topológico de Datos, evidenciando una mayor capacidad de este último para preservar estructuras relevantes y representar patrones complejos presentes en las bitácoras analizadas. A continuación, se desarrolla la representación de las bitácoras en el espacio (R^2) mediante el algoritmo *Mapper*, herramienta fundamental dentro de TDA que permite modelar y visualizar la estructura topológica de los datos. Finalmente, se realiza el cómputo y la comparación de características topológicas derivadas de dichas representaciones, con el objetivo de identificar patrones distintivos asociados a los diferentes vectores de ataque y evaluar la capacidad del modelo propuesto.

6.1. Estandarización de datos

Con el fin de homologar los datos obtenidos, en primer lugar se realiza una etapa de limpieza de los registros, eliminando los encabezados que se generan de forma intermitente durante la recolección de las métricas del sistema. Para ello se utiliza el programa descrito en el Anexo I. Adicionalmente, los registros se dividen en subconjuntos con diferentes niveles de cardinalidad porcentual, en incrementos del 10%, comenzando desde el 10 % hasta el 100 % del total de los datos. Este proceso permite analizar el comportamiento de los ataques considerando distintas cantidades de datos, lo cual resulta útil para evaluar la estabilidad y consistencia de los mismos en cada uno de los experimentos. Esta segmentación se realiza con el programa descrito en el Anexo J.

Una vez completada la etapa de limpieza y organización de los registros, se estandarizan los datos con el objetivo de mantener la consistencia entre las diferentes variables recolectadas. Este paso es necesario debido a que las métricas obtenidas presentan distintas escalas y magnitudes, lo cual podría afectar el análisis de datos. A continuación se describe el proceso de estandarización.

Para llevar a cabo la homogeneización de la información, el proceso de estandarización se despliega de manera secuencial. En primer lugar, los registros recolectados se agrupan en función del tipo de ataque

SFI, garantizando la integridad contextual de cada muestra. Posteriormente, se estandarizan las variables numéricas extraídas de las distintas bitácoras del sistema. Para ello se calcula la media aritmética de todo el conjunto de datos mediante la expresión:

$$\mu_g = \frac{1}{N} \sum_{i=1}^N x_i, \quad (6.1)$$

donde N representa el número total de observaciones y x_i denota el valor de cada registro individual. A partir de este parámetro, se determina la dispersión de los datos a través del cálculo de la desviación estándar global, definida por:

$$\sigma_g = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu_g)^2}. \quad (6.2)$$

Con ambos valores establecidos, cada observación original x es transformada de manera individual mediante el cálculo del puntaje estandarizado (*Z-score*), aplicando la ecuación:

$$z = \frac{x - \mu_g}{\sigma_g}. \quad (6.3)$$

Este proceso garantiza que la totalidad de los valores se expresen bajo una misma escala relativa con respecto a la distribución global de los datos. Esta homogeneización es un paso crítico en la metodología, ya que elimina las disparidades de magnitud entre las diferentes métricas al tomar las bitácoras, asegurando que el análisis topológico no se vea sesgado por variables de escalas numéricas dominantes.

Una vez estandarizados los datos, se realiza una partición del conjunto global para generar dos conjuntos independientes: 90 % de los datos destinados para **entrenamiento** y el 10 % para **prueba**, destinados al desarrollo y la evaluación del clasificador propuesto. Con la finalidad de mitigar el sesgo y garantizar la capacidad de generalización del modelo, se implementa una estrategia de validación cruzada basada en un esquema rotacional. Bajo este enfoque, de manera iterativa, un subconjunto específico se reserva exclusivamente para la fase de prueba, mientras que las fracciones restantes se consolidan para el entrenamiento [36]. Esta metodología asegura que ambos subconjuntos sean mutuamente excluyentes en cada ejecución, garantizando que ningún registro analítico pertenezca simultáneamente a ambos entornos durante un mismo ciclo experimental. Este proceso se realiza mediante el programa descrito en el Anexo K. Posteriormente, se generan archivos agrupados por tipo de ataque, en los cuales se concatenan todos los registros estandarizados asociados al tipo de ataque SFI. Este proceso se lleva a cabo mediante el programa descrito en el Anexo M. Como resultado, se obtiene una estructura final de archivos como se muestra en la figura 6.1.

Este proceso se realiza mediante el programa descrito en el Anexo K. Posteriormente, se generan archivos agrupados por tipo de ataque, en los cuales se concatenan todos los registros estandarizados asociados al tipo de ataque SFI correspondiente. Este procedimiento se lleva a cabo mediante el programa descrito en el Anexo M. La figura 6.1 muestra la estructura de directorio de archivos final una vez concluido el proceso de estandarización y partición de los datos. Se pueden observar diferentes directorios identificados como `split50_50`, `split60_40`, `split70_30`, `split80_20` y `split100`. Por ejemplo, el directorio `split60_40` indica que los registros han sido divididos en dos subconjuntos: uno que contiene el 60 % del total de los registros y otro que contiene el 40 %. De manera similar, el directorio `split70_30` contiene un subconjunto con el 70 % de los datos y otro con el 30 %.

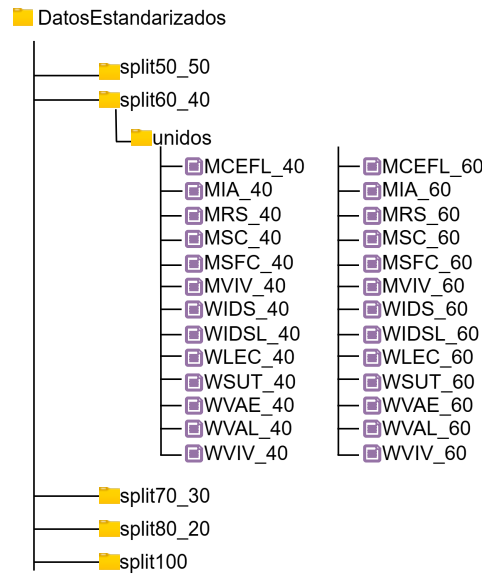


Figura 6.1: Estructura de directorio después de la estandarización.

6.2. Reducción de dimensionalidad de las bitácoras

Como se mencionó en la sección 3.1.2, en una primera etapa del análisis se decidió emplear técnicas tradicionales de reducción de dimensionalidad, específicamente PCA, UMAP y t -SNE. El objetivo de utilizar estas metodologías fue explorar si, mediante herramientas ampliamente utilizadas en el análisis de datos era posible obtener una representación de baja dimensionalidad que permitiera identificar patrones entre los distintos tipos de ataques SFI. Para ello, se realizó un proceso de experimentación en el cual se evaluaron distintos parámetros de cada algoritmo con el fin de identificar configuraciones que facilitaran la visualización y posible separación de las clases de ataque. Los resultados obtenidos mediante estas configuraciones se muestran en las figuras 6.2, 6.3 y 6.4, generadas a través del programa descrito en el Anexo L.

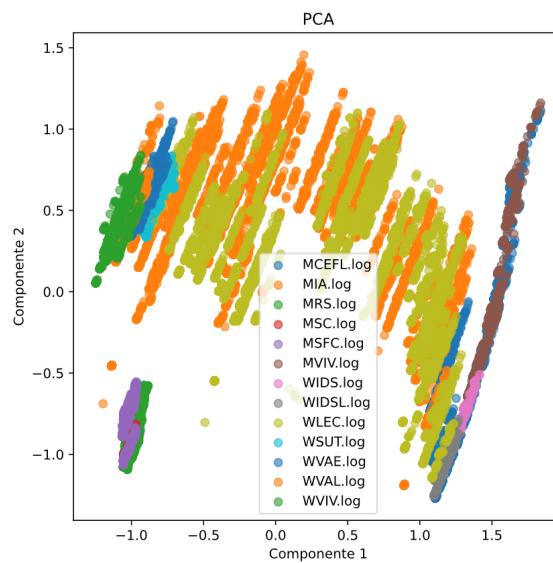


Figura 6.2: Reducción de dimensionalidad con el algoritmo PCA.

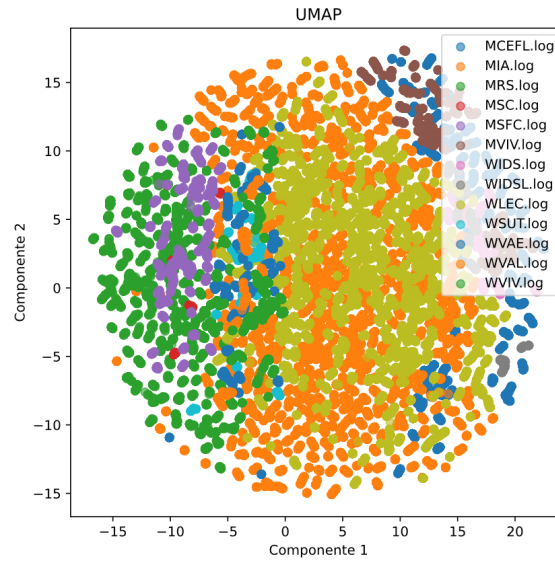


Figura 6.3: Reducción de dimensionalidad con el algoritmo UMAP.

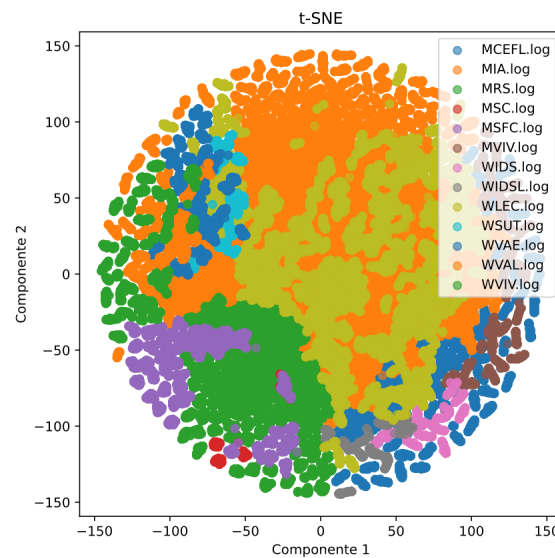


Figura 6.4: Reducción de dimensionalidad con el algoritmo t-SNE.

Los parámetros utilizados en cada uno de los métodos fueron los siguientes:

- En el caso de PCA, se especificó el parámetro $n_components=2$, como se muestra en las líneas 48 a 49 del Anexo L. Este parámetro permite proyectar los datos originales en las dos componentes principales que generan la mayor proporción de la varianza en el conjunto de datos. La principal ventaja de PCA es que preserva la estructura global de la varianza, lo cual facilita la identificación de direcciones dominantes en los datos. Sin embargo, su capacidad para capturar relaciones no lineales entre las variables es limitada, lo que dificulta la separación de clases cuando los datos presentan estructuras complejas.
- Para el caso de t-SNE, se configuraron los parámetros $n_components=2$, $random_state=42$ y $perplexity=30$, como se muestra en las líneas 53 a 54 del Anexo L. Aunque t-SNE es especialmente

eficaz para preservar estructuras locales en los datos, en este caso los resultados muestran una fuerte concentración de puntos, lo que dificulta distinguir agrupamientos asociados a los diferentes tipos de ataque.

- En el caso de UMAP, se utilizó el parámetro $n_components=2$ con el objetivo de reducir la dimensionalidad a dos dimensiones, así como $random_state=42$ para asegurar la consistencia y reproducibilidad de los resultados. Estos parámetros se muestran en las líneas 58 a 59 del Anexo L. UMAP es un método que busca preservar simultáneamente la estructura local y parte de la estructura global de los datos. No obstante, en el conjunto de datos, la proyección resultante tampoco muestra una separación clara entre los diferentes tipos de ataques debido a su nivel de traslape.

En términos generales, los resultados obtenidos indican que, si bien estas técnicas reducen efectivamente la dimensionalidad del conjunto de datos y preservan las relaciones de proximidad local para facilitar su visualización, ninguna de ellas logra discriminar de manera contundente los tipos de ataques SFI. El traslape y la dispersión de las firmas observadas en las proyecciones bidimensionales evidencian que los métodos usados presentan un alto grado de ambigüedad cuando se proyectan a bajas dimensiones. Estos hallazgos sugieren que la estructura subyacente de los datos involucra correlaciones no lineales de mayor complejidad. Por esta razón, en la siguiente sección se introduce un enfoque alternativo basado en el Análisis Topológico de Datos (TDA). Esta metodología permite estudiar la forma y las propiedades homológicas del conjunto de datos directamente en espacios de alta dimensión, con el objetivo de identificar patrones y estructuras globales que no pueden ser detectados por sí solos mediante los métodos de proyección tradicionales.

6.3. Representación en R^2 de las bitácoras usando Mapper

Los métodos tradicionales para el análisis de datos, no siempre permite obtener una clasificación inmediata en datos de alta dimensionalidad. En este sentido, el Análisis Topológico de Datos con un enfoque distinto resulta adecuado para este tipo de problemáticas. Mediante la construcción y el estudio de complejos simpliciales, el TDA posibilita la extracción de características topológicas, proporcionando información estructural adicional en comparación con otros métodos. La figura 6.5 ilustra la fase inicial de la metodología propuesta, en la cual se emplea el algoritmo Mapper para generar un grafo o complejo simplicial cuyas componentes conexas o agrupaciones representan la firma de un ataque específico o un conjunto de ellos. Para su construcción, se utilizó la biblioteca GUDHI de Python, específicamente la clase CoverComplex. Este método requiere la definición formal de una cubierta abierta $\mathcal{C}$ sobre el conjunto de puntos de entrada P , la cual consiste en una colección de subconjuntos indexados cuya unión interseca y reconstruye el espacio métrico original.

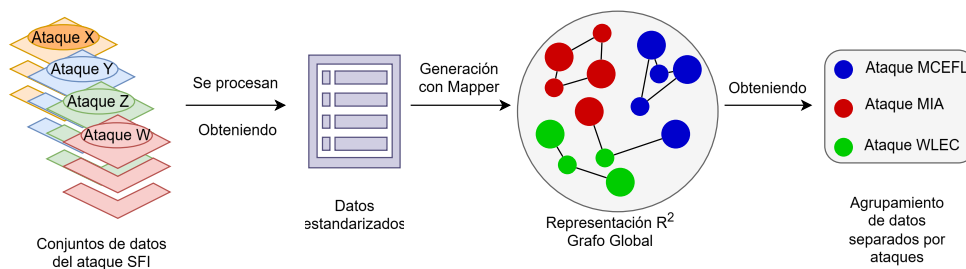


Figura 6.5: Generación de grafo global con Mapper.

Esta cubierta se construye a partir de la preimagen de una familia de **hipercubos** que recubren el espacio de proyección, obtenidos al transformar el conjunto de puntos de entrada mediante una función de filtro $f : P \rightarrow \mathbb{R}^n$. Aquí, f actúa como un operador de proyección que mapea cada elemento del espacio original a un espacio euclidiano de menor dimensión. Formalmente, la cubierta estructurada sobre el espacio indexado cumple con la condición geométrica:

$$\bigcup_{U \in \mathcal{C}} U = P. \quad (6.4)$$

La geometría de dicha familia de **hipercubos** se encuentra parametrizada por dos variables fundamentales: la resolución, que determina el número de particiones u operaciones de mallado en el espacio de proyección, y el solapamiento, que cuantifica el porcentaje de intersección simétrica entre **hipercubos adyacentes** [37]. La calibración de estos parámetros determina críticamente la resolución y la granularidad de la representación topológica resultante, permitiendo codificar la estructura subyacente de los datos con diferentes niveles de abstracción.

En este trabajo se utilizó como función de proyección el algoritmo PCA, el cual permite transformar el espacio métrico de entrada a un espacio euclidiano de dimensión dos. La elección de PCA se fundamenta en su capacidad para preservar la mayor parte de la varianza global de los datos mediante una transformación lineal, lo que favorece la obtención de una representación estable para la construcción del complejo simplicial. Si bien existen alternativas no lineales para la función de proyección, tales como t-SNE y UMAP, las cuales son capaces de capturar variedades complejas en los datos, aquí se optó por PCA debido a sus ventajas estructurales en términos de eficiencia computacional, estabilidad y reproducibilidad de los resultados. En particular, esta técnica reduce significativamente el tiempo de procesamiento, lo cual resulta especialmente crítico al interactuar con conjuntos de datos masivos; de este modo, se evita el elevado costo computacional asociado a la preservación de relaciones de vecindad local característico de los enfoques no lineales. Asimismo, al proyectar sobre los componentes principales de mayor varianza, este método contribuye intrínsecamente a la mitigación y reducción de ruido aleatorio en las métricas del sistema [38].

Para la construcción de la cubierta se empleó una resolución de 10 hipercubos con un solapamiento del 15 %. Estos parámetros se determinaron mediante un enfoque iterativo de prueba y error, considerando un agrupamiento inicial sin perder las relaciones entre los distintos subconjuntos de datos mediante el porcentaje de solapamiento. Hasta donde sabemos, no existe una técnica matemática que pueda separar los grupos de forma analítica, por ello, se va calibrando hasta que en cada constelación la mayoría de los datos pertenezcan a un vector de ataque.

Asimismo, para la etapa de agrupamiento dentro de cada subconjunto se utilizó el algoritmo DBSCAN, configurado con un valor de $\epsilon = 0.7$ y un mínimo de 200 puntos por vecindad. La elección de DBSCAN se fundamenta en su capacidad para identificar agrupamientos de forma robusta ante el ruido, características particularmente útiles en datos de alta dimensionalidad. La implementación de este proceso se describe en las líneas 130 a 139 del Anexo N. De manera similar, podrían emplearse otros algoritmos de agrupamiento como K-means o Hierarchical Clustering.

Es importante destacar que los parámetros seleccionados (resolución, solapamiento, ϵ y número mínimo de puntos) no se deducen mediante una metodología analítica cerrada o exacta. En su lugar, el diseño experimental se rige por un enfoque heurístico y exploratorio, mediante el cual se examinan distintas combinaciones en el espacio de parámetros para evaluar propiedades topológicas críticas como la conectividad del grafo, la emergencia de componentes conectadas y la estabilidad de las estructuras complejas. De este modo, se demuestra la inexistencia de una única configuración óptima global; por el

contrario, se identifica un régimen o conjunto de valores robustos que generan representaciones estables y computacionalmente útiles para la caracterización de los datos.

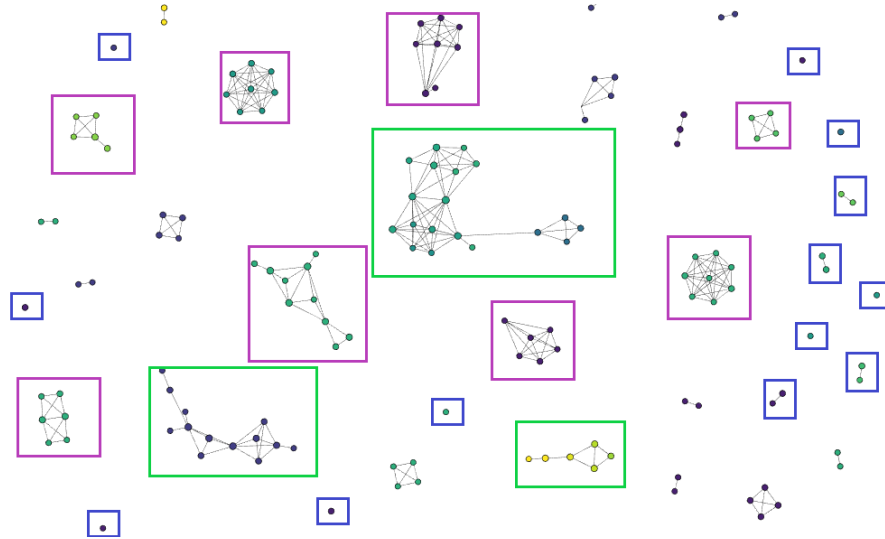


Figura 6.6: Extracción de la estructura topológica de los datos recolectados de los ataques. En color azul se muestran grafos pequeños (uno o dos nodos), considerados como ruido debido a su baja conectividad. En color morado se observan componentes conexas formadas por registros de un único tipo de ataque SFI. En color verde se presentan grafos compuestos por múltiples tipos de ataque, evidenciando mayor heterogeneidad en su estructura y composición.

La figura 6.6 muestra el grafo resultante del algoritmo Mapper, se identifican diversas agrupaciones de nodos que pueden interpretarse según su estructura y composición. En primer lugar, las nubes de puntos en color azul corresponden a grafos de uno o dos nodos, los cuales, en el contexto de este trabajo, se consideran como ruido debido a la escasa interrelación entre los subconjuntos de datos que los conforman. Por otro lado, los grafos representados en color morado corresponden a componentes conexas asociadas a un único tipo de ataque SFI. Finalmente, los grafos resaltados en color verde representan estructuras más complejas, que están formados por registros correspondientes a múltiples tipos de ataque SFI. Esta característica se refleja en una disposición geométrica más heterogénea, lo que sugiere la existencia de relaciones más complejas entre los datos.

Para ejemplificar de mejor forma, adentrándose a una sola constelación de puntos como se observa en la figura 6.7, al seleccionar uno de sus puntos, se puede observar del lado izquierdo que los datos que conforman ese punto y color, pertenecen a un único tipo de ataque SFI, que para este ejemplo es WLEC. Este comportamiento sugiere que los subconjuntos de datos que conforman dichos nodos comparten características similares entre sí, lo que provoca su cercanía en la proyección y su enlazamiento con los demás nodos.

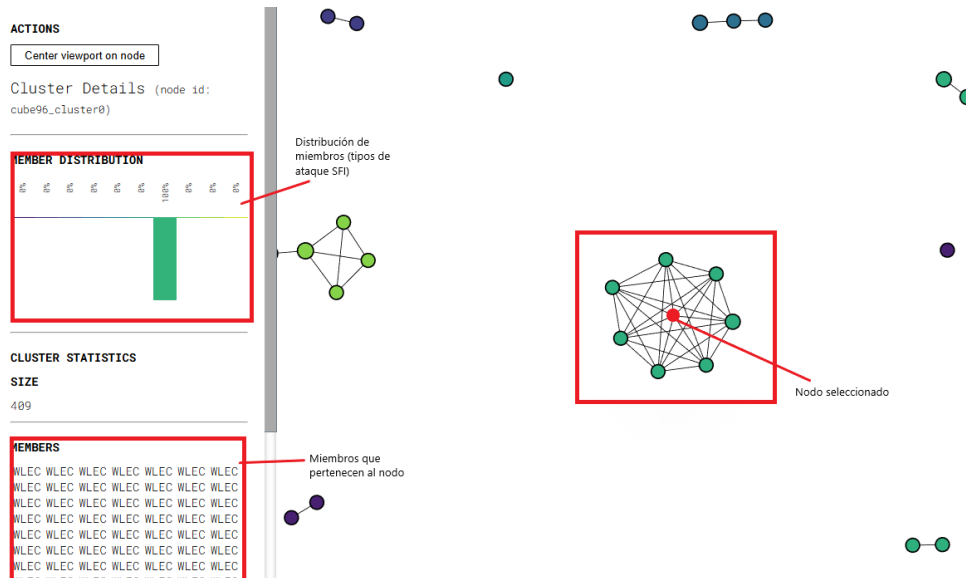


Figura 6.7: Nube de puntos de grafo global.

Asimismo, existen conjuntos de puntos que entrelazan uno o más tipos de ataque SFI, como se muestra en la figura 6.8 y 6.9. Se observa que su distribución no pertenece únicamente a un grupo (o color) y que el punto seleccionado ha sido construido a través de datos obtenidos de registros de diferentes tipos de ataques SFI. En particular, se observa que ciertos tipos de ataque tienden a agruparse con mayor frecuencia, lo que indica una similitud estructural en sus patrones, lo que sugiere que no existe una dominancia clara de un solo tipo de ataque. Este fenómeno puede estar relacionado con la cercanía entre distribuciones de los datos, lo cual, en etapas posteriores del análisis, se verá reflejado en distancias, al emplear la distancia de Wasserstein.

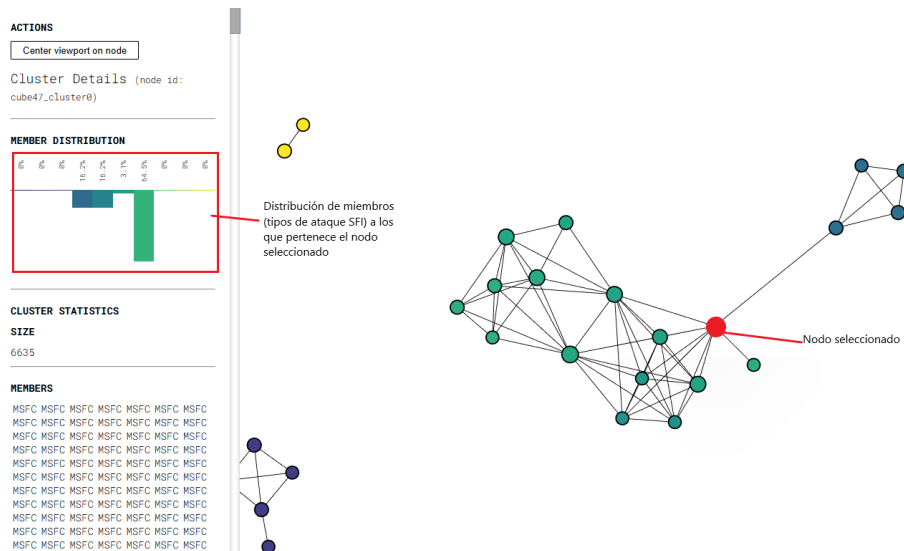


Figura 6.8: Nube de puntos de grafo global formado por más de un tipo de ataque.

De esta manera, estas zonas de intersección no solo evidencian la complejidad de los datos, sino que también permiten identificar regiones donde los distintos tipos de ataque presentan comportamientos similares, lo que resulta relevante para su caracterización.

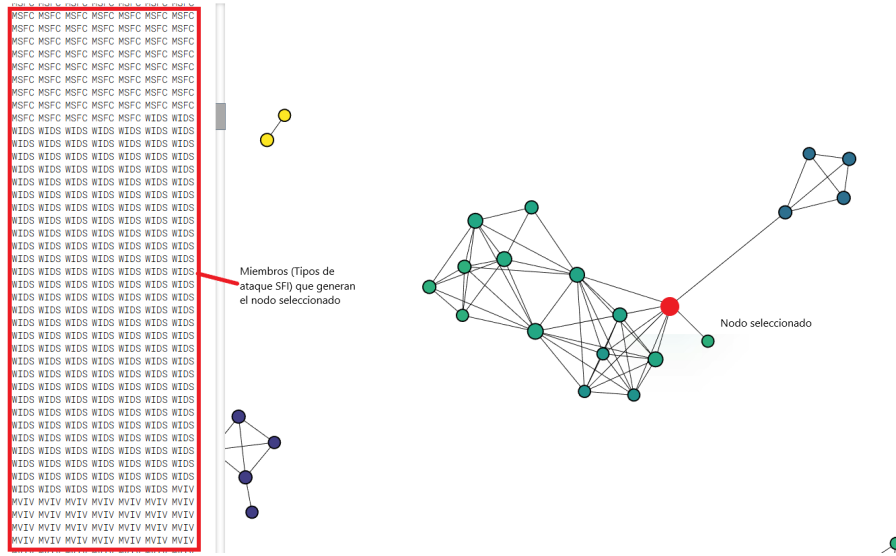


Figura 6.9: Nube de puntos de grafo global formado por más de un tipo de ataque.

Es importante destacar que el algoritmo Mapper genera como salida un grafo que captura la estructura relacional de los datos; sin embargo, esta representación por sí sola no constituye un espacio para el cómputo directo de invariantes topológicas. Por esta razón, el segundo bloque de la metodología propuesta consiste en transformar dicho grafo en un complejo simplicial que permita operar dentro de un marco topológico formal. Para tal fin, se construye un complejo de Vietoris–Rips sobre la estructura obtenida, el cual habilita el cálculo de los invariantes topológicos fundamentales, tales como los grupos de homología y los números de Betti, derivando finalmente en la obtención de los diagramas de persistencia. De este modo, se logra la transición desde una abstracción basada en grafos hacia un espacio topológico. Este procedimiento se despliega a través de las siguientes fases:

- Para cada tipo de ataque, aplicar nuevamente el algoritmo Mapper sobre los datos correspondientes.
- Identificar los nodos conexos más representativos y eliminar el ruido, es decir, los nodos sin conexión significativa.
- A partir de estos nodos conexos, recuperar los índices de los datos estandarizados utilizados para construir los grafos.

Es importante notar que, aunque previamente se había aplicado Mapper para obtener una representación global de todos los vectores de ataque (figura 6.6), en esta etapa solo se utiliza Mapper de manera local por grupo de ataque. Esto a consecuencia de limitaciones de memoria dentro del servidor utilizado para el desarrollo de esta tesis. Con esta decisión, se puede trabajar con subconjuntos más pequeños, y así construir complejos simpliciales más pequeños que permitan extraer de manera eficiente las invariantes topológicas.

Atendiendo esta limitante, a través de la segregación de las bitácoras en los diferentes tipos de ataques y mediante la generación de grafos para identificar componentes conexas, se identifican los puntos involucrados de estas componentes para extraer sus índices y conocer la ubicación específica de las bitácoras de los experimentos para permitir la obtención de conjuntos de componentes principales, como se describe en la figura 6.10. Este código es detallado en el Anexo Ñ, específicamente ver las líneas 149 a la 156. Para cada Mapper local, los parámetros utilizados para la construcción de los grafos fueron:

- Como entrada de datos, una matriz de distancia de los registros estandarizados asociados a un tipo de ataque SFI.
- No se utilizó un algoritmo de *cluster*.
- Se emplearon 10 hipercubos como cobertura, con una resolución de primer nivel de 20 (número de subdivisiones del espacio) y una resolución de segundo nivel de 2 (dos celdas para cubrir el espacio). Estos parámetros definen la granularidad de los hipercubos.

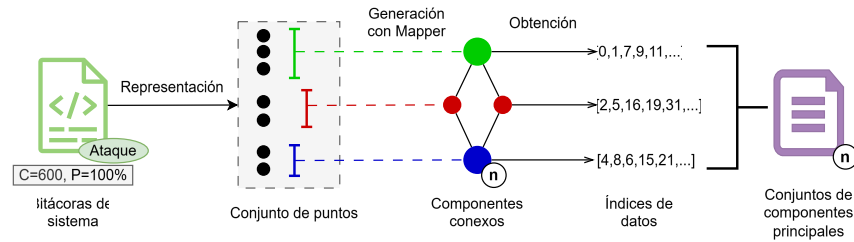


Figura 6.10: Generación de grafos para cada tipo de ataque usando Mapper.

6.4. Cómputo de características topológicas

A partir del grafo construido en la sección anterior, se pueden crear los complejos de **Vietoris-Rips**, los cuales resultan esenciales para determinar las invariantes topológicas. Como se mencionó en el Capítulo 3, el complejo de **Vietoris-Rips** es una estructura abstracta construida sobre un espacio métrico. Dado un conjunto de puntos P y un parámetro de proximidad $\epsilon > 0$, un k -simplex se incorpora al complejo si y solo si la distancia euclidiana entre cada par de sus vértices es menor o igual a ϵ [39]. La figura 6.11 muestra los diagramas de persistencia obtenidos de la construcción del complejo **Vietoris-Rips** de un vector de ataque SFI.

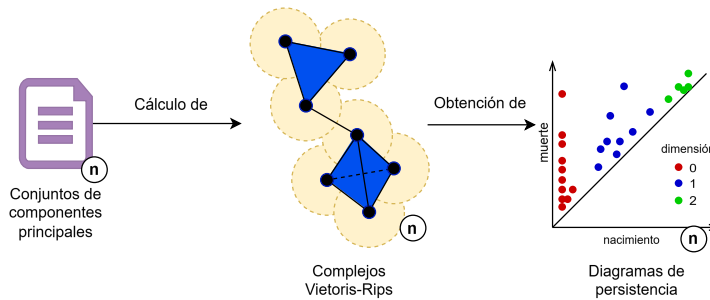


Figura 6.11: Generación de grafos para cada tipo de ataque usando Mapper.

En el Anexo Ñ, líneas 197 a la 201 se presenta el código para crear los complejos **Vietoris-Rips**. El proceso inicia con la clase *RipsComplex* a partir del conjunto de puntos *component_points*. Dentro de esta configuración, se establece el parámetro *sparse=0.6* para habilitar una aproximación dispersa de la estructura simplicial; se seleccionó este valor para optimizar el uso de memoria y evitar el desbordamiento de los recursos de *hardware*.

Posteriormente, se genera un árbol simplicial (*simplex_tree*) derivado del complejo de Rips con una restricción de dimensión máxima igual a tres (*max_dimension=3*). Esto implica que el sistema indexa simplices hasta de dimensión tres, límite computacional impuesto estratégicamente para evitar el crecimiento exponencial asociado al incremento de la dimensión del complejo.

Finalmente, se evalúa la persistencia topológica del árbol simplicial mediante el método *simplex_tree.persistence()*. Este proceso permite extraer las características homológicas más robustas de los datos. Los diagramas de persistencia resultantes constituyen las firmas topológicas entre los diferentes vectores de ataque SFI. La figura 6.12 muestra la firma topológica para la componente 5 el vector de ataque MCEFL, donde se puede observar la conexión progresiva de los complejos simpliciales, hasta terminar con una sola componente conexa (grupo de homología 0), así mismo, no presenta un ciclo dominante ni una cavidad dominante, ya que todos los grupos de homología 1 y 2 se encuentran muy cerca de la diagonal.

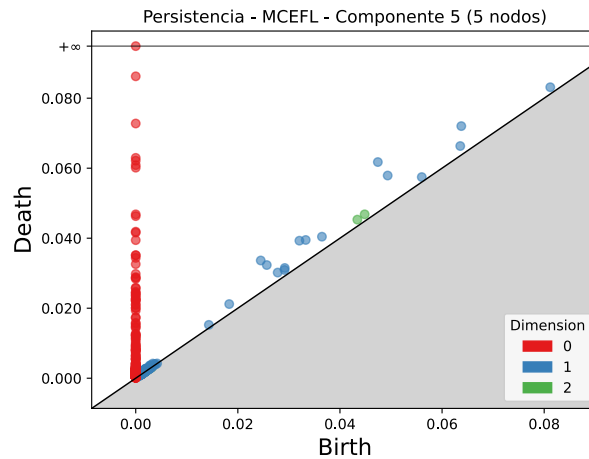


Figura 6.12: Ejemplo de Diagrama de persistencia para el vector MCEFL.

Este procedimiento se repite para cada uno de los vectores de ataque SFI. Como se mencionó previamente, los datos fueron particionados en subconjuntos con cardinalidad porcentual en incrementos del 10 %. Esta estrategia tuvo como objetivo analizar el comportamiento de cada tipo de ataque en función de la cantidad de datos disponibles, permitiendo identificar puntos de estabilidad a partir de los cuales el análisis resulta comparable con la totalidad de los datos. En este sentido, se busca detectar umbrales en los que el incremento en la cantidad de datos no produzca cambios significativos en las características topológicas extraídas, lo que sugiere una representación suficientemente robusta del comportamiento del sistema. Este proceso se ilustra en la figura 6.13 y corresponde al ciclo **for** descrito en la línea 112 del Anexo Ñ.

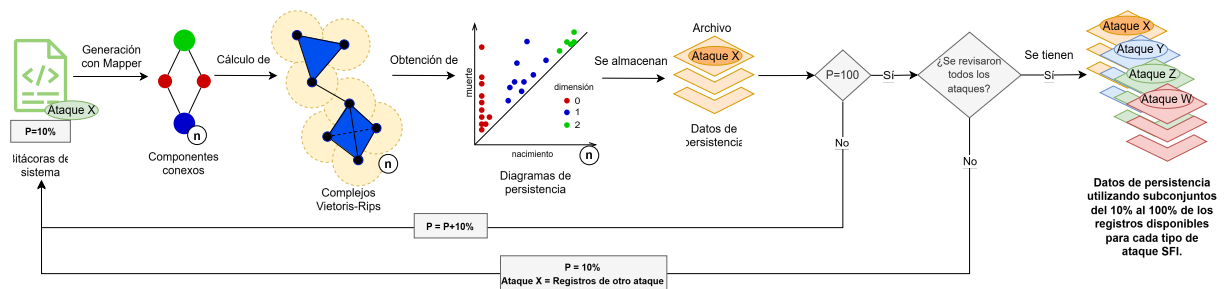


Figura 6.13: Generación de Diagramas de persistencia para cada vector de ataque usando Mapper y los subconjuntos de porcentajes desde el 10 % hasta el 100 %.

Como resultado de este proceso, se obtuvieron los diagramas de persistencia organizados según el vector de ataque y la cardinalidad porcentual. Con estos, se puede calcular la distancia de Wasserstein como medida de similitud entre los diagramas, y así poder distinguir la variación de las características

topológicas conforme se incrementa la cantidad de datos respecto al 100 % de la muestra. De este modo, se podrá evaluar la estabilidad y consistencia de las estructuras topológicas identificadas.

6.4.1. Comparación de características topológicas

A partir de los diagramas de persistencia, se obtienen las coordenadas de nacimiento y muerte de las características topológicas, así como la dimensión a la que pertenecen para cada una de las componentes significativas identificadas. Esta información se almacenó en archivos CSV, generados en las líneas 210 a 217 del Anexo Ñ. Posteriormente, se comparó las estructuras topológicas obtenidas mediante el cálculo de la distancia de Wasserstein. El código se encuentra descrito en el Anexo O. Específicamente, el código realiza el cálculo de la distancia de Wasserstein entre diagramas de persistencia almacenados en distintas carpetas, las cuales representan diferentes porcentajes de datos (del 10 % al 100 %) con la finalidad de observar en qué porcentaje de densidad de datos se observa un comportamiento similar al conjunto total analizado. Para cada porcentaje, el programa recorre subcarpetas identificadas por vector de ataque SFI y carga los archivos CSV que contienen las coordenadas. Es importante recordar que estos diagramas de persistencia corresponden a los grupos de homología considerados en el análisis, en este caso para dimensiones 0, 1 y 2, lo que equivale a los números de Betti 0, 1 y 2. A partir de esta información, se realiza la comparación entre los diagramas. Este proceso se implementa mediante la función `calcular_y_guardar`, descrita en las líneas 36 a 50. La figura 6.14 ilustra este proceso.

Cada resultado incluye el tipo de ataque SFI, los porcentajes de datos comparados, los archivos involucrados, la dimensión topológica y el valor correspondiente de la distancia de Wasserstein. Finalmente, todos los resultados se consolidan en un único archivo CSV, el cual permite analizar de manera integral la similitud entre diagramas de persistencia bajo diferentes condiciones de cardinalidad y tipo de ataque.

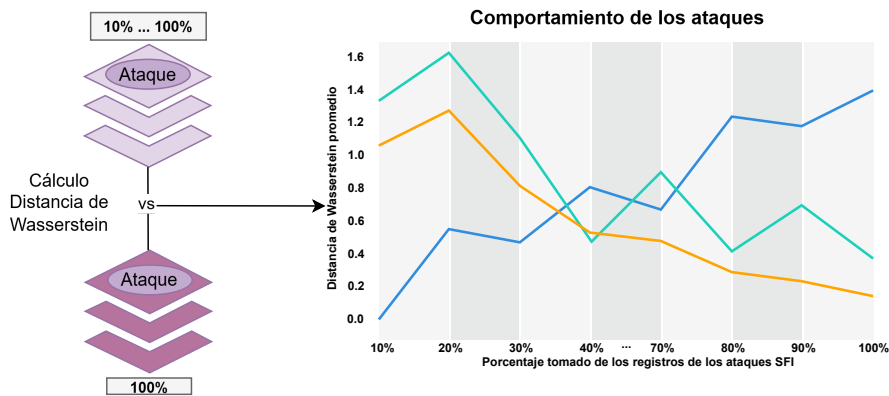


Figura 6.14: Cálculo de la distancia de Wasserstein.

De forma general, se cuantifica la disimilitud topológica entre subconjuntos porcentuales, permitiendo analizar su estabilidad y comportamiento. Así, para cada tipo de ataque se tiene una evolución de su distancia de Wasserstein en función al porcentaje de datos, como se puede observar en la figura 6.15, 6.16 y 6.17, para Betti 0, 1 y 2, respectivamente. En la figuras se muestran las distancias promedio con respecto al porcentaje de datos. El código usado se presenta en el Anexo P.

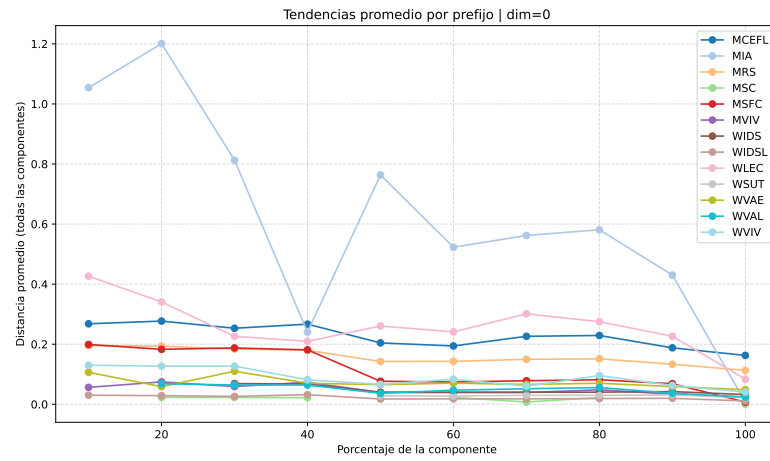


Figura 6.15: Comportamiento por tipo de ataque - Dimensión 0.

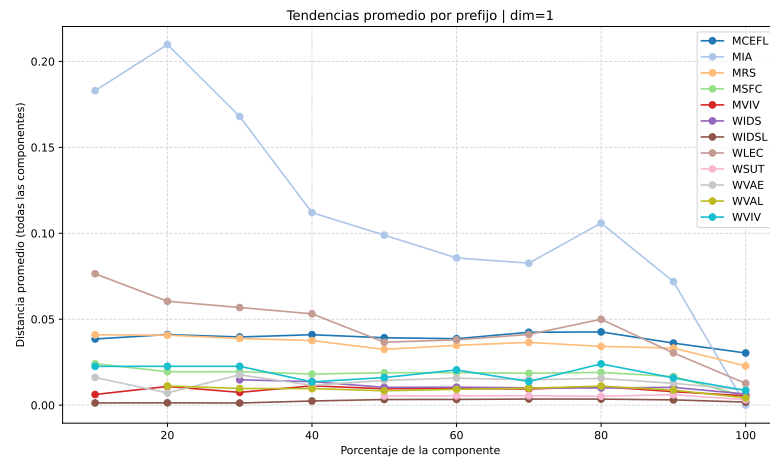


Figura 6.16: Comportamiento por tipo de ataque - Dimensión 1.

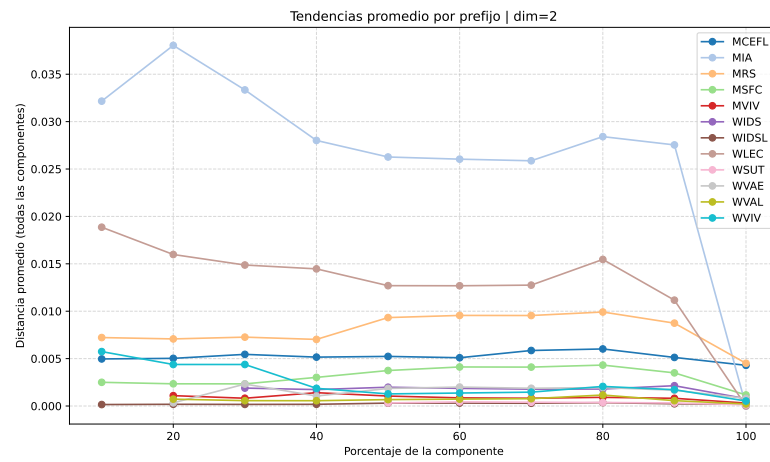


Figura 6.17: Comportamiento por tipo de ataque - Dimensión 2.

La figura 6.15 (dimensión 0), muestra que algunos ataques, como MIA, presentan valores elevados

en la distancia de Wasserstein en porcentajes de datos bajos (entre 10 % y 30 %), lo que indica una alta disimilitud respecto al total de la muestra y del resto de ataques. Sin embargo, conforme aumenta la cantidad de datos, su comportamiento tiende a estabilizarse, mostrando una disminución progresiva, lo que sugiere que requiere una mayor cantidad de información para caracterizar adecuadamente su estructura topológica. Por otro lado, ataques como MCEFL muestran un comportamiento más estable a lo largo de los diferentes porcentajes, con variaciones relativamente pequeñas desde etapas tempranas (alrededor del 20 % al 30 %), lo que indica que su estructura topológica es capturada de manera más consistente incluso con subconjuntos reducidos de datos. Asimismo, se identifican ataques como MSFC que presentan una transición intermedia, donde inicialmente muestran cierta variabilidad, pero alcanzan estabilidad a partir de porcentajes cercanos al 60 %, sugiriendo que existe un umbral mínimo de datos necesario para obtener una representación confiable.

La figura 6.16 (dimensión 1), muestra una tendencia general hacia valores más bajos de la distancia de Wasserstein, lo que indica que las características topológicas son menos sensibles a la variación. Sin embargo, nuevamente se distingue el comportamiento de MIA, que presenta mayor variabilidad en comparación con otros ataques, reforzando la idea de que su estructura topológica es más compleja o menos consistente en subconjuntos pequeños.

Por último, en la figura 6.17 (dimensión 2), se observa una disminución general en los valores de la distancia de Wasserstein en comparación con dimensiones inferiores. No obstante, en esta dimensión destacan los ataques MIA, WIDS y MRS, los cuales presentan una mayor separación respecto al resto, evidenciando un comportamiento distintivo. A diferencia de lo observado en dimensiones previas, estos ataques no muestran picos abruptos en porcentajes bajos, sino que mantienen una tendencia más estable a lo largo de toda la evolución porcentual. Este comportamiento sugiere que, las estructuras topológicas en dimensión 2 presentan una mayor consistencia, lo que permite capturar características más robustas y diferenciadoras entre los tipos de ataque.

En conjunto, estos resultados evidencian que no todos los tipos de ataque convergen de la misma manera hacia la estructura topológica del conjunto completo. Algunos presentan estabilidad temprana, mientras que otros requieren una mayor cantidad de datos para alcanzar una representación consistente. Este comportamiento refleja diferencias en la complejidad y variabilidad de las características topológicas asociadas a cada ataque, tanto en dimensión 0 como en dimensiones superiores (números de Betti 1 y 2), como se discutió en el Capítulo 2.

Ante este análisis, para buscar una agrupación que permitiera poder clasificar un nuevo experimento hacia algunos de los ataques SFI conocidos, se obtuvo un promedio del total de las distancias de Wasserstein de cada subconjunto percentil y por tipo de ataque, considerando todas las dimensiones analizadas con la finalidad de evaluar el comportamiento de cada ataque de forma integral. A partir de los archivos CSV que contienen las distancias de Wasserstein se genera una matriz cuadrada donde cada entrada (i, j) representa el promedio de las distancias asociadas a los tipos de ataque. Este proceso se realiza mediante la función `calcular_matriz_por_archivo`, detallada en el programa del Anexo Q, que recorre todas las combinaciones posibles de prefijos (tipos de ataque) y calcula el promedio correspondiente, generando una matriz simétrica. Posteriormente, todas las matrices individuales se promedian entre sí para obtener una matriz global.

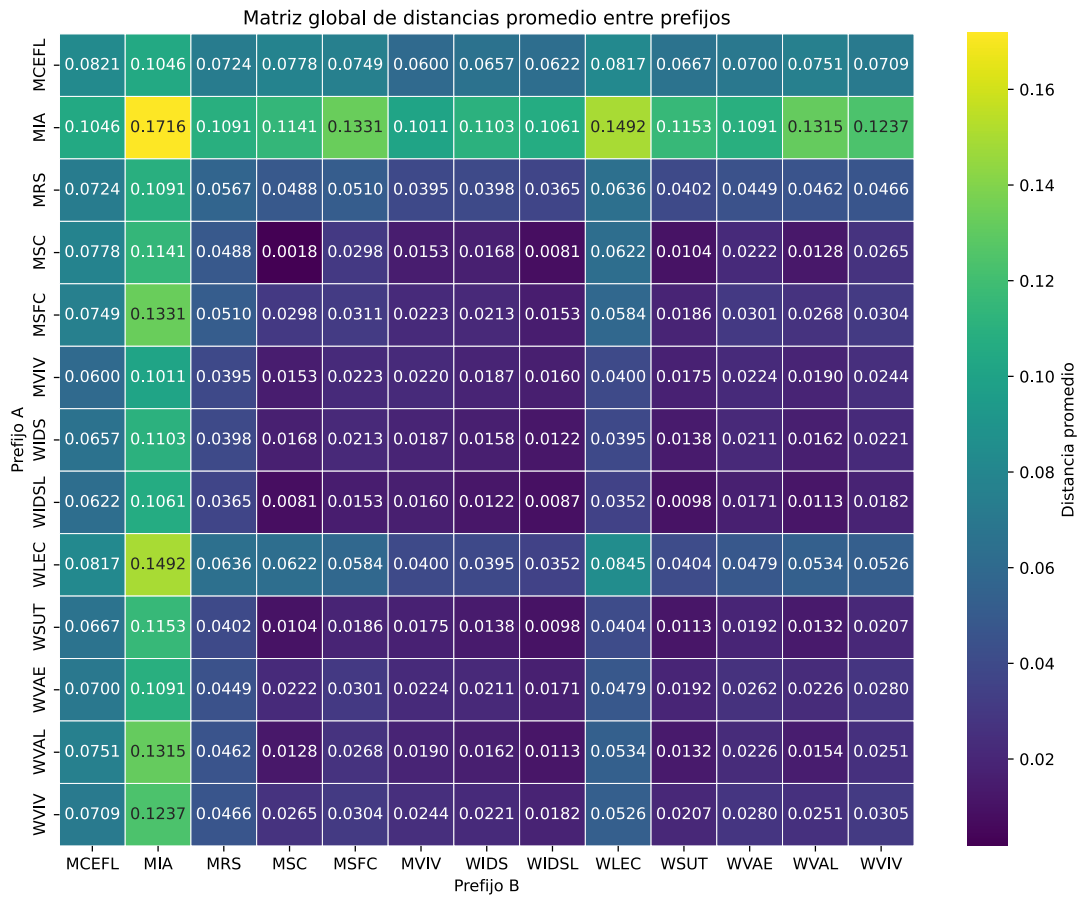


Figura 6.18: Matriz global de distancias de Wasserstein por cada tipo de ataque SFI.

El resultado final se visualiza como un mapa de calor (figura 6.18) representando una matriz global de distancias promedio de Wasserstein entre los diagramas de persistencia asociados a cada tipo de ataque SFI. Cada entrada de la matriz representa la distancia promedio entre dos tipos de ataque, lo cual permite cuantificar qué tan similares son sus estructuras topológicas. En primer lugar, puede observarse que la matriz es simétrica y que los valores más pequeños se concentran en ciertos subconjuntos de ataques. Esto indica que algunos ataques generan características topológicas similares.

Un caso particularmente notable corresponde a los ataques *MSC*, *MVIV*, y *WIDSL*, cuyos valores de distancia son considerablemente pequeños entre sí. Por ejemplo, la distancia entre *MSC* y *WIDSL* es aproximadamente 0.0081, mientras que entre *MSC* y *MVIV* es cercana a 0.0153. Estas magnitudes sugieren que los diagramas de persistencia asociados a estos ataques comparten una estructura topológica muy parecida.

Otro grupo interesante aparece entre los ataques *MCEFL*, *MRS* y *WLEC*, donde por ejemplo, la distancia entre *MRS* y *WLEC* es cercana a 0.0636, mientras que entre *MCEFL* y *MRS* es aproximadamente 0.0724. Aunque estas distancias son mayores que las anteriores, también sugiere que estos tres tipos de ataque comparten características estructurales que los diferencian del resto.

Cuando varios ataques presentan distancias de Wasserstein cercanas entre sí, es común observar que los nodos en los grafos de Mapper tienden a aparecer en regiones cercanas o incluso a mezclarse dentro de las mismas componentes conexas. Por lo tanto, esta matriz ayuda a explicar la estructura observada en la representación global topológica creada inicialmente (ver figura 6.6).

En contraste, el ataque *MIA* presenta distancias significativamente mayores respecto a la mayoría de los otros ataques. Por ejemplo, la distancia entre *MIA* y *MRS* es aproximadamente 0.1091, mientras que entre *MIA* y *WLEC* alcanza alrededor de 0.1492, siendo uno de los valores más altos en la matriz. Esto indica que las características topológicas asociadas a este ataque difieren considerablemente del resto. Esto refleja que este tipo de ataque conlleva un comportamiento distinto dentro del sistema.

Es importante señalar que, debido a las limitaciones de poder de cómputo del entorno de ejecución utilizado, fue necesario adoptar estrategias de optimización como el uso de representaciones dispersas (descritas en la Sección 6.4) y la reducción de precisión numérica mediante el tipo de dato `float32`. También es importante resaltar que sobre la diagonal, no en todos los casos se tiene un valor cercano a cero, esto se debe a dos situaciones: uno, a la falta de precisión al recortar a 32 bits, y la más importante a que existen escenarios en el que el número de tomas de datos no fue suficiente para estabilizar las características topológicas. Un ejemplo claro es el vector *MIA*, que presentaba saltos en las dimensiones 0 y 1. Asimismo, a través del mapa de calor no se observa una frontera clara que ayude a clasificar cada vector de ataque. Para ello, se construyó un dendrograma jerárquico a partir de la matriz de distancia global mediante el método de **Ward**, que agrupa los elementos minimizando la varianza entre grupos. De esta forma, se obtiene un dendrograma que representa la estructura jerárquica de similitudes globales entre los vectores de ataque. En el anexo R se presenta el código propuesto. La figura 6.19 muestra el resultado del dendrograma.

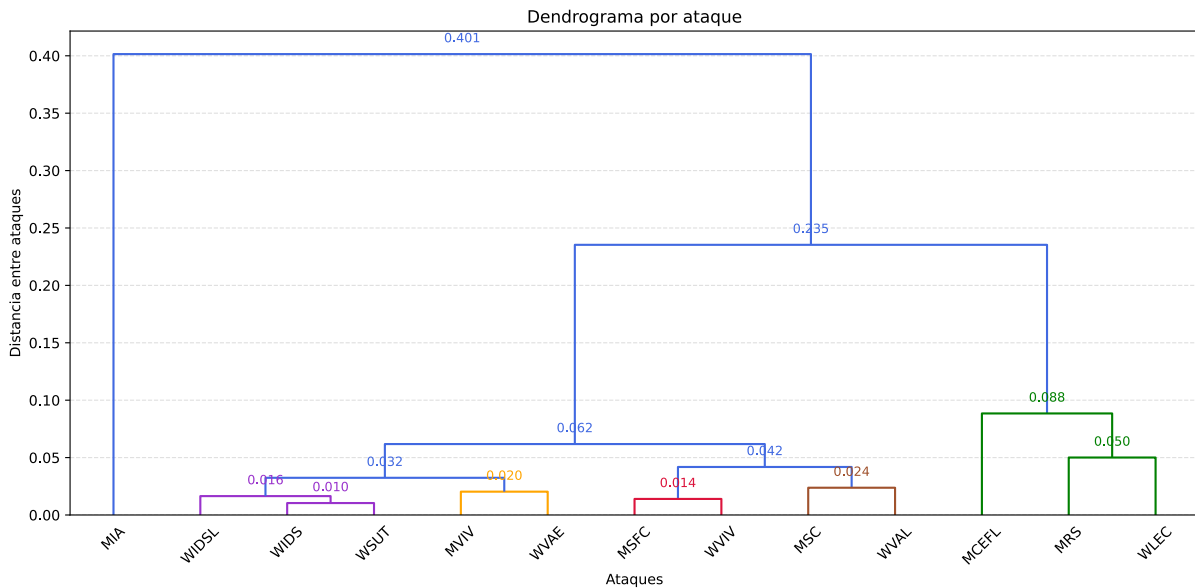


Figura 6.19: Jerarquización de los ataques conforme a sus distancia de Wasserstein.

El dendrograma permite interpretar la estructura de agrupamiento de la siguiente manera:

- Cada hoja corresponde a un tipo de ataque.
- Las uniones entre ramas indican procesos de agrupamiento progresivo según el método jerárquico de **Ward**.
- La altura a la que se fusionan dos ramas representa el nivel de similitud promedio entre los ataques.
- Fusiones a menor altura indican ataques con comportamientos topológicos más similares, mientras que fusiones a mayor altura reflejan una mayor diferencia estructural.

Se formaron seis grupos de ataques:

- Grupo1: MIA.
- Grupo2: WIDSL, WIDS, WSUT.
- Grupo3: MVIV, WVAE.
- Grupo4: MSFC, WVIV.
- Grupo5: MSC, WVAL.
- Grupo6: MCEFL, MRS, WLEC.

Estos grupos evidencian un alto nivel de similitud entre los tipos de ataque que los conforman, por lo que podría generar confusión al diferenciar uno del otro. El primer grupo corresponde al conjunto de vectores formado por *WIDSL*, *WIDS* y *WSUT*, que se agrupan a distancias pequeñas, cercanas a 0.01 (conforme al el método de Ward). Esto confirma lo observado en la matriz de distancias, donde estos ataques presentan valores muy bajos entre sí. La proximidad en el dendrograma indica que los diagramas de persistencia de estos ataques comparten una estructura muy similar, lo cual sugiere que las características topológicas generadas por estos son prácticamente equivalentes. Posteriormente, este grupo se une con vectores como *MVIV* y *WVAE*. Estos forman un grupo ligeramente diferente, pero aún con distancias similares. Otro grupo en el dendrograma incluye ataques como *MSFC* y *WVIV*, los cuales se agrupan a una distancia cercana de 0.014, indicando nuevamente una fuerte similitud estructural. De manera similar, los ataques *MSC* y *WVAL* forman otro subgrupo con distancias relativamente pequeñas, pero conservándose dentro del mismo grupo que se engloba por una distancia aproximada de 0.042.

Por otro lado, el conjunto formado por *MCEFL*, *MRS* y *WLEC* aparece como un grupo más independiente dentro de la jerarquía, lo cual coincide con lo observado en la matriz de distancias. Estos ataques comparten similitudes entre sí, pero presentan mayores diferencias respecto al resto de grupos. Finalmente, el ataque *MIA* aparece separado del resto con un nivel de distancia mayor, alcanzando aproximadamente 0.401 antes de integrarse al resto de la jerarquía. Esta separación indica que su estructura topológica es significativamente distinta a la de los otros ataques, lo cual refuerza la hipótesis de que este tipo de ataque genera patrones de comportamiento muy complejos topológicamente.

Finalmente, el dendrograma permiten identificar grupos de ataques que comparten patrones topológicos similares. Estos resultados son coherentes con las estructuras observadas en los grafos generados mediante el algoritmo Mapper. Desde la perspectiva del análisis topológico de datos, este comportamiento indica que los diagramas de persistencia capturan estructuras topológicas relevantes del sistema bajo distintos escenarios de ataque. La distancia de Wasserstein permite comparar estas estructuras de manera cuantitativa, mientras que el dendrograma facilita la identificación de relaciones jerárquicas entre los distintos tipos de ataque. En consecuencia, esto proporciona una herramienta útil para la clasificación de los vectores de ataque SFI.

6.5. Comentarios finales

Lo abordado en este capítulo permite demostrar que los métodos tradicionales de reducción de dimensionalidad, presentan limitaciones importantes al momento de identificar y separar adecuadamente los patrones asociados a los distintos tipos de ataques SFI. Aunque estas técnicas permiten visualizar

parcialmente la estructura de los datos, la complejidad y la alta variabilidad del comportamiento registrado en los eventos de seguridad dificultan una caracterización clara y consistente de cada categoría de ataque. Esto evidencia que las aproximaciones convencionales no son suficientes para capturar relaciones entre los subconjuntos de datos. Por otro lado, el uso de TDA, particularmente mediante el uso de diagramas de persistencia, permite identificar estructuras y patrones relevantes que permanecen ocultos para los métodos tradicionales. A través del análisis topológico fue posible observar características distintivas en el comportamiento de cada tipo de ataque, reflejadas en la persistencia de componentes y relaciones presentes en las bitácoras recolectadas. Estos resultados confirman que TDA es una herramienta robusta para el análisis de datos complejos, ya que no solo mejora la capacidad de caracterización y diferenciación entre ataques, sino que también aporta una perspectiva más profunda sobre el comportamiento inherente de los experimentos.

Capítulo 7

Análisis y resultados

En este capítulo, se presenta el análisis de las pruebas de manera más granular, esto con el fin de evaluar la estabilidad de las características topológicas identificadas e incrementar la capacidad de discriminación entre vectores de ataques **SFI**. Se presentan las métricas utilizadas para evaluar la eficacia del modelo propuesto bajo diferentes escenarios.

7.1. Métricas de evaluación

Con el propósito de evaluar cuantitativamente el desempeño y la capacidad de discriminación del clasificador propuesto, se utilizaron métricas ampliamente utilizadas [40, 41]: precisión (*precision*), exhaustividad (*recall*) y el valor F1 (*F1-score*). Estos estimadores permiten analizar el comportamiento del modelo desde perspectivas complementarias, ponderando rigurosamente tanto la exactitud en los aciertos como el impacto de los errores de clasificación.

La **precisión** (*precision*) cuantifica la proporción de instancias correctamente asignadas a una clase específica con respecto al total de predicciones que el modelo determinó para dicha categoría. De este modo, esta métrica refleja la confiabilidad del clasificador al mitigar la tasa de falsos positivos. Formalmente, se define mediante la expresión:

$$\text{Precisión} = \frac{VP}{VP + FP}, \quad (7.1)$$

donde VP representa los verdaderos positivos y FP los falsos positivos. Una alta precisión indica que el modelo comete pocos errores al asignar etiquetas, es decir, cuando predice una clase, generalmente es correcto.

Por otro lado, la **exhaustividad** o **sensibilidad** (*recall*) mide la capacidad intrínseca del modelo para identificar correctamente la totalidad de las instancias que pertenecen genuinamente a una clase determinada. Por lo tanto, este estimador refleja la robustez del clasificador al minimizar la omisión de casos críticos o falsos negativos. Matemáticamente, se define como:

$$\text{Recall} = \frac{VP}{VP + FN}, \quad (7.2)$$

donde FN representa el número de falsos negativos. Un valor elevado en esta métrica indica que el modelo logra detectar la gran mayoría de las instancias reales de falla, minimizando la omisión de eventos críticos.

Finalmente, el **valor F1** (*F1-score*) condensa ambos criterios en un único indicador mediante la media armónica entre la precisión y la exhaustividad. Esta formulación matemática proporciona una

evaluación robusta y equilibrada del desempeño global del clasificador, siendo especialmente útil debido a que penaliza los desbalances extremos entre las dos métricas anteriores. Su expresión formal es:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (7.3)$$

7.2. Pruebas de evaluación

Para evaluar el desempeño, se utilizaron los datos correspondientes al conjunto de prueba y las siguientes herramientas de análisis.

- **F1-score:** se emplea para medir el equilibrio entre precisión y exhaustividad en la clasificación de los ataques SFI.
- **Diagramas de caja:** permiten visualizar la dispersión y variabilidad presente en los distintos tipos de ataques.
- **Matriz de confusión:** permite identificar los aciertos y errores de clasificación entre los distintos ataques SFI.
- **Mapa de calor:** facilita la visualización de patrones de similitud y diferencias entre ataques a partir de las métricas utilizadas.
- **Ranking de F1-score:** permite ordenar y comparar los ataques de acuerdo a la métrica F1-score.

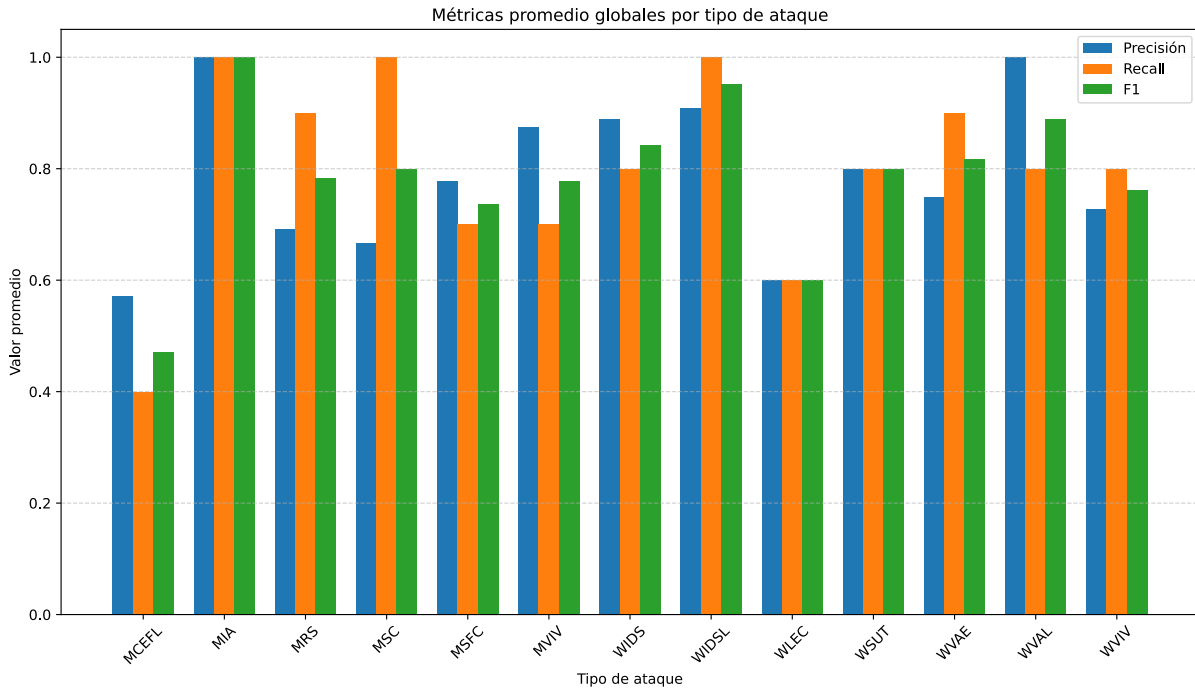


Figura 7.1: Métricas promedio de *precision*, *recall* y *F1-score* por tipo de ataque SFI para el modelo de clasificación basado en características topológicas.

La figura 7.1 presenta el desempeño del modelo propuesto. Se puede observar que en la mayoría de los vectores de ataque el sistema presenta un desempeño alto, destacando ataques como MIA y WIDSL, que

alcanzan valores cercanos o iguales a 1.0 en *precision*, *recall* y *F1-score*. Esto indica que el modelo no solo es capaz de identificar correctamente estos ataques, sino que lo hace con alta confiabilidad, derivado de que para estos casos no se generan falsos positivos o falsos negativos significativos. Asimismo, ataques como WVAL, WVAE y WSUT presentan valores elevados en *F1-score*, mostrando un buen equilibrio entre *precisión* y *recall*, evidenciando una adecuada capacidad de discriminación entre las estructuras topológicas para estos tipos de ataque.

Por otro lado, se identifican vectores de ataque donde el desempeño del clasificador es sutilmente menor, tales como MCEFL y MSC, en los cuales se observa una discrepancia notable entre la *precisión* y la *exhaustividad* (*recall*). En particular, para el ataque MSC se obtiene un *recall* ideal en contraste con una baja *precisión*; este comportamiento revela una alta tasa de falsos positivos, indicando que el modelo presenta un sesgo de sobreestimación hacia esta clase. De manera similar, en el caso de MCEFL, tanto la *precisión* como la *exhaustividad* experimentan un decremento en comparación con las demás categorías, lo que se traduce en un *F1-score* reducido y evidencia la dificultad del algoritmo para discernir las firmas topológica. En conjunto, estos hallazgos demuestran que, si bien el enfoque propuesto es efectivo en la mayoría de los escenarios, existen ciertos vectores de ataque cuya geometría subyacente sugiere la necesidad de un ajuste más profundo en el espacio de parámetros de los complejos simpliciales, o bien, la incorporación de estrategias complementarias de aprendizaje.

Con el objetivo de evaluar la distribución de los errores de clasificación y la variabilidad estadística de los resultados, se emplearon diagramas de caja (*box plots*). Esta técnica de visualización permite identificar con precisión la mediana, el rango intercuartil y la presencia de valores atípicos (*outliers*), con el fin de tener una interpretación de la dispersión, la simetría y la estabilidad del modelo a través de los diferentes vectores de ataque evaluados.

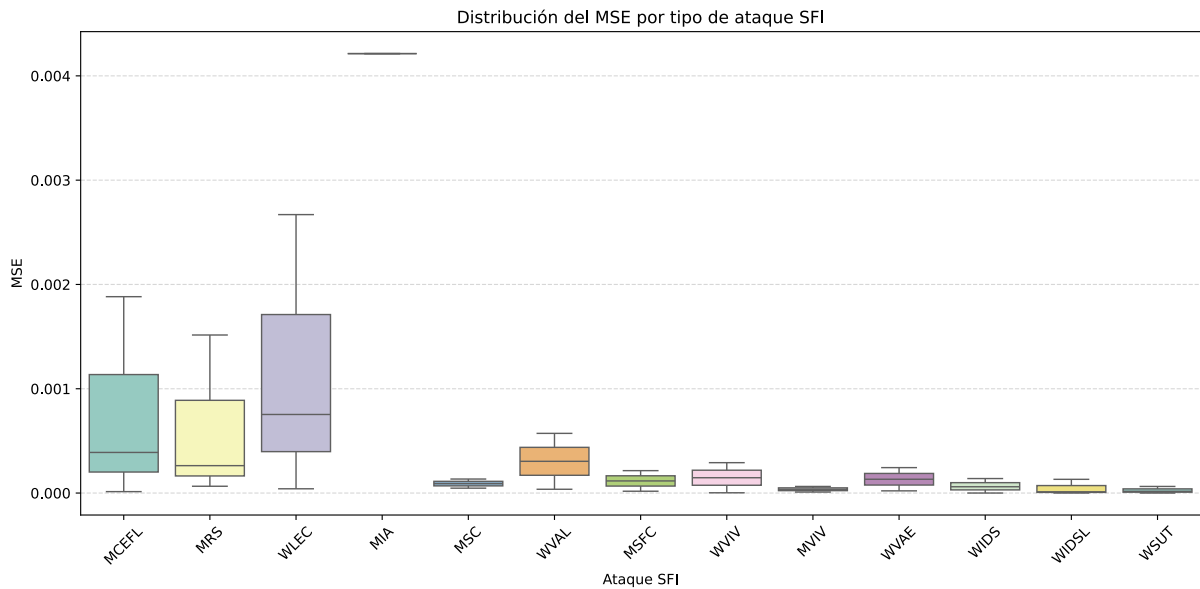


Figura 7.2: Distribución del error cuadrático medio por tipo de ataque SFI utilizando diagramas de caja.

La figura 7.2 muestra la variación en la clasificación del modelo según el tipo de ataque SFI, basado en el error cuadrático medio (MSE) como métrica. Se observa que algunos vectores como MCEFL, MRS y WLEC presentan una mayor dispersión y valores más altos de MSE, lo que indica que para estos casos la clasificación del modelo es menos precisa, lo que confirma lo observado en la figura 7.1. En particular, WLEC muestra una dispersión mayor con algunos valores máximos altos, esto señala que el modelo

presenta dificultades al comparar las características topológicas dentro de este tipo de ataque. Por otro lado, vectores como MSC, WVAE, WIDS, WIDSL y WSUT presentan una varianza muy baja, reflejando que la clasificación es más consistente y confiable en estos grupos. Además, la posición de la mediana y la longitud de los bigotes en la mayoría de las distribuciones indican que el 50 % de los datos se concentra en rangos acotados, lo cual constituye un indicador robusto de la ausencia de sesgos o fallas sistemáticas en el clasificador. Los valores atípicos (*outliers*) se manifiestan únicamente en vectores de ataque específicos; tal es el caso de MIA, el cual exhibe un comportamiento singular caracterizado por registrar el mayor margen de error absoluto pero con una varianza mínima. Esto sugiere que la firma topológica de MIA posee una estructura topológica considerablemente disímil respecto al resto de los vectores SFI (véase el Anexo U).

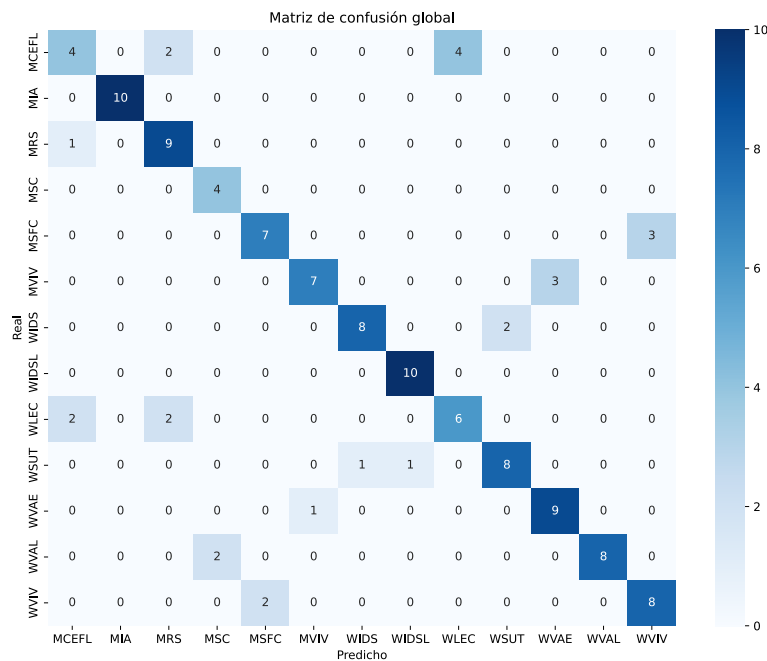


Figura 7.3: Matriz de confusión global del modelo.

Adicionalmente, se construyó la matriz de confusión correspondiente, en la cual cada fila indexa las instancias reales de un determinado vector de ataque, mientras que las columnas cuantifican las predicciones emitidas por el modelo. Los componentes dispuestos sobre la diagonal principal representan las clasificaciones correctas, en tanto que los elementos fuera de ella exponen las tasas de error y los patrones de ambigüedad entre las diferentes clases simuladas. Como se ilustra en la Figura 7.3, se observa un claro predominio en la densidad de aciertos sobre la diagonal principal, lo cual convalida el alto desempeño general del clasificador. Sin embargo, se identifican patrones de error sistemáticos y relevantes; tal es el caso de los vectores MCEFL, WLEC y MSC, los cuales exhiben una mayor dispersión o falsas clasificaciones hacia otras categorías de ataques SFI. Estas divergencias sugieren un traslape significativo dentro de sus invariantes topológicas. Con base en estos hallazgos, se concluye que para este subconjunto de vectores el modelo demanda un análisis más robusto o descriptores simpliciales de mayor resolución que permitan discriminar con mayor eficiencia sus firmas topológicas.

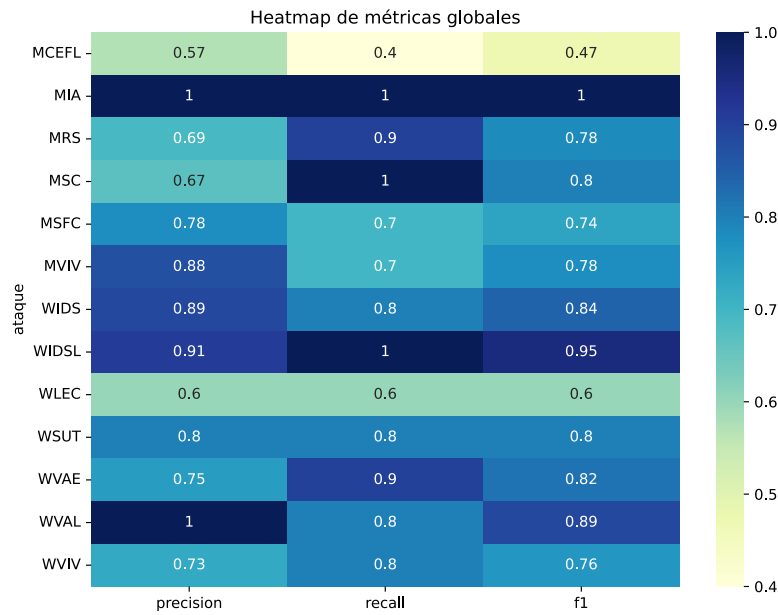


Figura 7.4: Mapa de calor con las métricas globales.

En la Figura 7.4 se presenta un mapa de calor para las métricas de *precision*, *recall* y *F1-score*, donde se observa que, ataques como MIA y WIDSL, alcanzan valores cercanos a 1 en todas las métricas, lo que indica una clasificación prácticamente perfecta. En contraste, clases como MCEFL y WLEC presentan valores más bajos, evidenciando limitaciones en la capacidad del modelo para distinguir correctamente estos ataques.

Finalmente, en la figura 7.5 se presenta el ranking de los ataques basado en la métrica *F1-score*. En esta figura se puede observar que ataques como MIA y WIDSL ocupan las primeras posiciones, lo que confirma su alta detectabilidad por parte del modelo. Por otro lado, ataques como MCEFL y WLEC se sitúan en los niveles más bajos, reflejando la dificultad del modelo para identificar con claridad estos ataques. Este ordenamiento permite evidenciar qué tipos de ataques SFI presentan mayor similitud entre sí en cuanto a sus estructuras topológicas. En términos generales, el modelo es bastante efectivo para algunos grupos de ataques, mientras que en otros, se debe ajustar para mejorar su robustez y precisión. Este resultado refleja que el enfoque basado en Análisis Topológico de Datos, apoyado en la comparación de diagramas de persistencia mediante la distancia de Wasserstein, permite capturar características relevantes para la clasificación entre vectores de ataques. Sin embargo, los resultados también evidencian ciertas limitaciones, particularmente al momento de discernir entre ataques con estructuras topológicas con alta similitud. Estas limitaciones están asociadas, en parte, a la dimensión máxima considerada en la construcción de los complejos de Vietoris–Rips (hasta Betti 2), lo que restringe la cantidad de información topológica capturada, esto sugiere que al incrementar la dimensión de los complejos simpliciales se podrían identificar características topológicas adicionales, lo que mejoraría la distinción entre tipos de ataque SFI y, por ende, la precisión del modelo. No obstante, esta mejora implicaría un incremento considerable en el costo computacional.

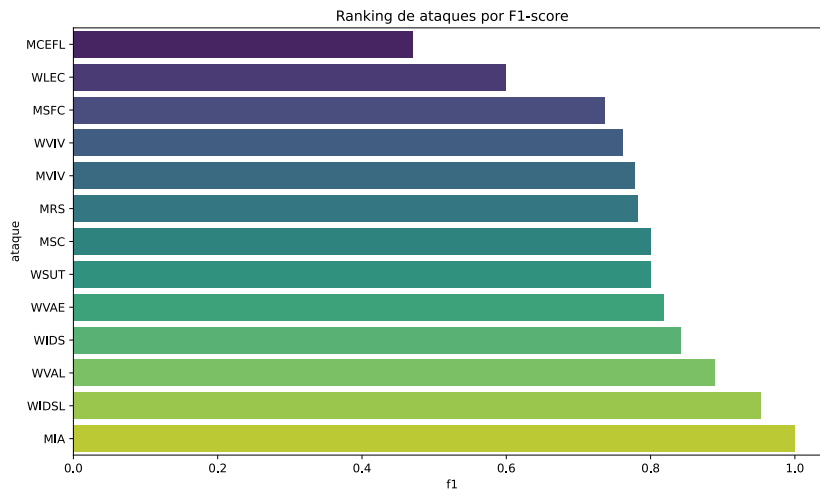


Figura 7.5: Ranking de la métrica F1-score de los ataques SFI.

A partir de los resultados obtenidos, se retoma la hipótesis planteada en este trabajo, la cual establece:

El uso de TDA para la detección de anomalías dentro de una IaaS permite identificar un ataque SFI en el servicio de aprovisionamiento de instancias, y con ello mandar estas alertas a un controlador inteligente que tome acciones.

En este sentido, la evidencia experimental permite afirmar que la pregunta de investigación se cumple, ya que el modelo propuesto logra identificar patrones topológicos distintivos y alcanzar una clasificación correcta hasta del 100 % para algunos vectores y de aproximadamente el 80 % para el promedio de los 13 vectores de ataque estudiados. Esto evidencia que, si bien el enfoque propuesto es capaz de capturar información relevante del sistema para clasificar de manera correcta varios vectores de ataque, existen límites en su capacidad de discernir bajo las condiciones y parámetros considerados en el ambiente de este trabajo.

7.3. Comentarios finales

Las distintas métricas de evaluación permiten realizar un análisis integral del desempeño del modelo en la clasificación de los ataques SFI. El uso del F1-score resulta fundamental para medir el equilibrio entre la precisión y la exhaustividad. Además, proporciona una evaluación más robusta del comportamiento del modelo de clasificación ante datos complejos o con estructuras topológicas similares entre los distintos tipos de ataque. Asimismo, la incorporación de diagramas de caja permite identificar la dispersión y variabilidad presente en cada tipo de ataque.

De igual manera, la matriz de confusión facilita la identificación detallada de los aciertos y errores de clasificación, permitiendo reconocer qué tipos de ataques presentan mayores similitudes y cuáles generan confusiones recurrentes dentro del modelo. Complementariamente, el uso de mapas de calor permite visualizar de forma más intuitiva las relaciones de similitud y diferencia entre ataques a partir de las métricas evaluadas, mientras que el *ranking* basado en F1-score ayuda a comparar de forma clara el desempeño alcanzado para cada categoría de ataque. En conjunto, estas herramientas de análisis validan la efectividad del enfoque propuesto y ofrecen una comprensión más profunda del comportamiento del sistema frente a escenarios reales de ciberseguridad.

Capítulo 8

Conclusiones

La presente tesis propone un modelo basado en TDA para detectar distintos vectores de ataque SFI dentro de una IaaS de OpenStack. La metodología propuesta fue desarrollada a lo largo de los capítulos 5 y 6, desde la recolección y procesamiento de bitácoras del sistema, hasta la generación de características topológicas y su evaluación mediante el cálculo de la distancia de Wasserstein como métrica de similitud. Los resultados muestran que el enfoque propuesto permite identificar estructuras topológicas relevantes dentro de conjuntos de datos, alcanzando una clasificación correcta del 80 % como promedio de los 13 vectores de ataque analizados. Sin embargo, en varios de estos vectores el modelo tiene una eficacia del 100 %. De manera general, los resultados muestran un comportamiento estable y consistente, sustentado en las métricas de *precision*, *recall* y *F1-score*, así como en las gráficas de cajas, las cuales evidencian una baja variabilidad en la mayoría de los experimentos realizados.

Entre los principales hallazgos obtenidos destacan los siguientes:

- Los ataques WVAL, WVAE y WSUT presentan valores elevados de *F1-score*, reflejando un equilibrio adecuado entre precisión y *recall*, así como una correcta discriminación entre las estructuras topológicas asociadas a estos vectores de ataque.
- Los ataques MSC, WVAE, WIDS, WIDSL y WSUT exhiben una varianza reducida, indicando que la clasificación obtenida es consistente y confiable a lo largo de las diferentes ejecuciones del modelo.
- Ataques como MIA y WIDSL ocuparon las primeras posiciones en desempeño, confirmando una alta detectabilidad por parte del modelo propuesto.
- En contraste, vectores como MCEFL y WLEC presentaron los niveles más bajos de clasificación, evidenciando una mayor dificultad para distinguir sus patrones topológicos característicos.
- Los vectores MCEFL, WLEC y MSC muestran una mayor dispersión y una tendencia más pronunciada hacia falsas clasificaciones en otras categorías de ataques SFI, lo que sugiere un traslape considerable entre sus invariantes topológicas.

El modelo propuesto demostró que la información topológica extraída a partir de los datos de una infraestructura como servicio, específicamente de OpenStack, permite diferenciar comportamientos asociados a distintos tipos de ataques *Software Fault Injection*. En particular, el uso de diagramas de persistencia y de la distancia de Wasserstein, en conjunto con un esquema de clasificación basado en el Error Cuadrático Medio, permitió establecer un mecanismo funcional y efectivo para la comparación entre experimentos y la clasificación de los ataques.

8.1. Desafíos del modelo

A pesar de los resultados obtenidos, durante el proceso de evaluación se identificaron diversas limitaciones que influyen en la capacidad de discriminación del modelo. Estas limitaciones se relacionan tanto con la naturaleza de los datos analizados como con las restricciones computacionales presentes durante el desarrollo de los experimentos.

Entre los principales desafíos identificados destacan los siguientes:

- Algunos tipos de ataques presentan comportamientos muy similares entre sí, generando un traslape en sus invariantes topológicos y dificultando una separación clara entre categorías.
- Ataques como MCEFL y WLEC evidenciaron mayores niveles de dispersión y falsas clasificaciones, lo que sugiere que ciertas estructuras topológicas no son suficientemente distintivas para el modelo propuesto.
- Las restricciones en los recursos computacionales disponibles limitaron el alcance, particularmente en la generación de grafos mediante el algoritmo Mapper y en la construcción de complejos simpliciales.
- Debido a las restricciones del punto anterior, fue necesario acotar parámetros relevantes del análisis, como el número de hipercubos utilizados y la cantidad máxima de grupos de homología considerados durante el procesamiento.
- La reducción de dichos parámetros puede impactar en la capacidad del modelo para capturar estructuras topológicas de mayor complejidad, afectando potencialmente la precisión en ciertos vectores de ataque.

No obstante, a pesar de estas limitaciones, los resultados obtenidos permiten concluir que el uso de Análisis Topológico de Datos constituye una herramienta útil y viable para la caracterización de ataques SFI. El enfoque propuesto demostró ser capaz de identificar patrones y estructuras relevantes dentro de los datos.

8.2. Puntos destacados en el desarrollo

Los puntos destacados de este trabajo pueden resumirse en:

- **Aplicación de TDA para la caracterización de ataques SFI:** Se implementó un enfoque basado en TDA para analizar el comportamiento de distintos tipos de ataques de tipo SFI a partir de bitácoras del módulo Nova de la IaaS, OpenStack, permitiendo identificar estructuras y relaciones complejas presentes en los datos y en cada uno de los experimentos realizados.
- **Clasificación de ataques SFI mediante Homología persistente:** Se evidenció que el uso de TDA a través de diagramas de persistencia permite identificar patrones topológicos distintivos asociados al comportamiento de los ataques SFI, proporcionando una representación más enriquecedora de la información contenida en las bitácoras recolectadas y observando la similitud entre estas con el cálculo de la distancia de Wasserstein.
- **Análisis visual y estadístico del comportamiento de los ataques:** Se incorporaron herramientas de visualización y análisis estadístico que facilitan la interpretación de la dispersión, similitud y variabilidad entre ataques, contribuyendo a una mejor comprensión de los patrones presentes

en los datos analizados. Identificando así que tipos de ataques SFI presentan una similitud mayor entre sí y por ende una tendencia de clasificación más compleja y difícil de caracterizar

- **Definición de un esquema integral de evaluación mediante métricas de desempeño:** Se estableció una metodología de análisis basada en métricas como F1-score, matrices de confusión, diagramas de caja y mapas de calor, permitiendo evaluar de manera detallada la capacidad de clasificación y diferenciación entre ataques SFI.

8.3. Trabajo futuro

A partir de los resultados obtenidos y de las limitaciones identificadas durante el desarrollo de esta investigación, se plantean diversas líneas de trabajo futuro orientadas a mejorar la capacidad de discriminación y robustez del modelo propuesto.

Entre las principales propuestas destacan las siguientes:

- Incorporar grupos de homología de mayor dimensión con el fin de representar relaciones topológicas más profundas y así mejorar la separación entre distintos tipos de ataques.
- Explorar métricas alternativas para la comparación entre diagramas de persistencia, evaluando su impacto en la capacidad de clasificación y discriminación del modelo.
- Ampliar el conjunto de datos mediante la incorporación de un mayor número de experimentos y escenarios de ataque, con el fin de construir modelos más robustos, generalizables y menos sensibles a variaciones específicas de los datos.
- Evaluar diferentes configuraciones del algoritmo Mapper, incluyendo variaciones en el número de hipercubos, porcentaje de traslape y funciones lente, con el objetivo de optimizar la representación topológica obtenida.
- Integrar técnicas de Inteligencia Artificial y Aprendizaje Automático que permitan complementar las características topológicas extraídas, por ejemplo mediante clasificadores híbridos o redes neuronales.
- Analizar la viabilidad de implementar mecanismos de detección en tiempo real, integrando el enfoque topológico con sistemas de monitoreo y análisis continuo en infraestructuras de nube.

Lo anterior sugiere que, al considerar dimensiones superiores y representaciones topológicas más complejas, el modelo podría capturar patrones topológicos más representativos de cada vector de ataque, incrementando así la capacidad de separación y diferenciación entre categorías SFI. De esta manera, el enfoque basado en Análisis Topológico de Datos podría evolucionar hacia soluciones más precisas, escalables y aplicables a entornos reales de ciberseguridad en infraestructuras como servicio.

Anexo A

Creación de Máquinas Virtuales de la IaaS

En este anexo se presentan los comandos utilizados para la creación de las máquinas virtuales Controller y Compute.

Consola A.1: Creación de las máquinas virtuales en KVM

```
1 sudo virt-install --connect qemu:///system --name ANDRES-CONTROLLER --os-variant=ubuntu18.04 --  
    memory 4096 --vcpus 2 --location http://archive.ubuntu.com/ubuntu/dists/bionic/main/  
    installer-amd64 --network bridge=br0 --network bridge=br0 --disk path=/dev/vg1/ANDRES-  
    Controller --disk path=/dev/vg1/ANDRES-Controller-sw --graphics none --console pty,  
    target_type=serial --extra-args='console=ttyS0,115200n8 serial' --debug  
2  
3 sudo virt-install --connect qemu:///system --name ANDRES-COMPUTE --os-variant=ubuntu18.04 --  
    memory 4096 --vcpus 2 --location http://archive.ubuntu.com/ubuntu/dists/bionic/main/  
    installer-amd64 --network bridge=br0 --network bridge=br0 --disk path=/dev/vg1/ANDRES-  
    Compute --disk path=/dev/vg1/ANDRES-Compute-sw --graphics none --console pty,target_type=  
    serial --extra-args='console=ttyS0,115200n8 serial' --debug
```

Anexo B

Instalación y Configuración de OpenStack

En este anexo se describen los pasos realizados para la instalación y configuración manual de OpenStack sobre dos nodos: Controller y Compute. Se incluye una breve descripción técnica de cada modificación realizada en los archivos del sistema.

B.1. Configuración de Red

B.1.1. Nodo Controller

Se configuraron dos interfaces de red estáticas para separar tráfico de gestión y servicios internos del clúster.

Consola B.1: Configuración Netplan en Controller

```
1 sudo nano /etc/netplan/01-netcfg.yaml
2     ## PARA CONTROLLER
3     network:
4         version: 2
5         renderer: networkd
6         ethernet:
7             enp1s0:
8                 addresses: [ 192.168.10.97/24 ]
9                 gateway4: 192.168.10.2
10                nameservers:
11                    search: [ tda ]
12                    addresses:
13                        - "132.248.10.2"
14            enp2s0:
15                addresses: [ 192.168.10.98/24 ]
16                gateway4: 192.168.10.2
17                nameservers:
18                    search: [ tda ]
19                    addresses:
20                        - "132.248.10.2"
```

```
21 netplan apply
```

La interfaz `enp1s0` se utiliza para la red de gestión (192.168.10.0/24), mientras que `enp2s0` permite separar tráfico adicional de servicios.

B.1.2. Nodo Compute

Configuración de red estática para permitir comunicación con el nodo Controller.

Consola B.2: Configuración Netplan en Compute

```
1 sudo nano /etc/netplan/01-netcfg.yaml
2     network:
3         version: 2
4         renderer: networkd
5         ethernets:
6             enp1s0:
7                 addresses: [ 192.168.10.100/24 ]
8                 gateway4: 192.168.10.2
9                 nameservers:
10                     search: [ tda ]
11                     addresses:
12                         - "132.248.10.2"
13             enp2s0:
14                 addresses: [ 192.168.10.99/24 ]
15                 gateway4: 192.168.10.2
16                 nameservers:
17                     search: [ tda ]
18                     addresses:
19                         - "132.248.10.2"
20
21 netplan apply
```

El nodo Compute utiliza direccionamiento dentro de la misma red de gestión para permitir comunicación con los servicios.

B.2. Resolución de Nombres

Se agregaron entradas estáticas en el archivo `/etc/hosts` para permitir la comunicación por nombre entre los nodos sin depender de un servidor DNS.

Consola B.3: Configuración de `/etc/hosts` en ambos nodos

```
1 echo -e "#_controller\n192.168.10.97_controller\n#_compute1\n192.168.10.100_compute1" >> /etc/
   hosts
2
3 sudo systemctl restart systemd-networkd
```

La resolución local de nombres garantiza que los servicios internos (RabbitMQ, Nova, Neutron) puedan comunicarse usando nombres de host en lugar de direcciones IP directas, lo que facilita administración y mantenimiento.

B.2.1. Verificación de Conectividad

Se valida conectividad externa y conectividad interna entre nodos, asegurando que la red esté correctamente configurada antes de continuar con la instalación.

```
1 ping -c 4 docs.openstack.org
2 ping -c 4 compute1
3 ping -c 4 openstack.org
4 ping -c 4 controller
```

B.3. Sincronización de Tiempo

Se instala y configura Chrony para mantener sincronización horaria entre nodos, requisito fundamental para autenticación mediante tokens en Keystone.

B.3.1. Nodo Controller

```
1 apt install -y chrony
2 nano /etc/chrony/chrony.conf
3     server ntpdgctic.redunam.unam.mx iburst
4     allow 192.168.10.0/24
5
6 service chrony restart
7 chronyc sources
```

B.3.2. Nodo Compute

```
1 apt install -y chrony
2 nano /etc/chrony/chrony.conf
3     server controller iburst
4
5 service chrony restart
6 chronyc sources
```

El nodo Controller actúa como servidor NTP interno, mientras que el nodo Compute sincroniza su hora con el Controller. La sincronización precisa es crítica para evitar fallos de autenticación por desincronización de tokens.

B.3.3. Repositorio Oficial de OpenStack

Se habilitó el repositorio cloud-archive correspondiente a la versión Ussuri.

```
1 apt install -y software-properties-common
2 add-apt-repository cloud-archive:ussuri
3 apt install -y python3-openstackclient
```

El repositorio cloud-archive:ussuri permite instalar versiones oficiales y estables de OpenStack. El paquete python3-openstackclient proporciona la herramienta CLI para administración del entorno.

B.4. Servicios Base - Nodo Controller

B.4.1. Configuración de MariaDB

Se configuró MariaDB como backend de base de datos para los servicios de OpenStack.

```
1 apt install -y mariadb-server python-pymysql
2 nano /etc/mysql/mariadb.conf.d/99-openstack.cnf ###CREAR
3     [mysqld]
4         bind-address = 192.168.10.97
5         default-storage-engine = innodb
6         innodb_file_per_table = on
7         max_connections = 4096
8         collation-server = utf8_general_ci
9         character-set-server = utf8
10
11 service mysql restart
12 mysql_secure_installation
```

B.4.2. Configuración de RabbitMQ, memcached y etcd

RabbitMQ se instaló en el nodo Controller como sistema de mensajería interna para OpenStack. Este servicio permite la comunicación asíncrona entre los distintos componentes del sistema, principalmente entre Nova (Controller) y los servicios ejecutados en el nodo Compute.

```
1 apt install -y rabbitmq-server
2 rabbitmqctl add_user openstack p0aqsww
3 rabbitmqctl set_permissions openstack ".*" ".*" ".*"
```

El servicio Memcached se instaló en el nodo Controller como sistema de almacenamiento en memoria para mejorar el rendimiento de autenticación en OpenStack.

```
1 apt install -y memcached python-memcache
2 nano /etc/memcached.conf
3     -l 192.168.10.97
4 service memcached restart
```

El servicio etcd se instaló en el nodo Controller para almacenar objetos de clave-valor utilizados principalmente por el servicio Placement dentro de OpenStack.

```
1 apt install -y etcd
2 nano /etc/default/etcd
3
4     ETCD_NAME="controller"
5     ETCD_DATA_DIR="/var/lib/etcd"
6     ETCD_INITIAL_CLUSTER_STATE="new"
7     ETCD_INITIAL_CLUSTER_TOKEN="etcd-cluster-01"
8     ETCD_INITIAL_CLUSTER="controller=http://192.168.10.97:2380"
9     ETCD_INITIAL_ADVERTISE_PEER_URLS="http://192.168.10.97:2380"
10    ETCD_ADVERTISE_CLIENT_URLS="http://192.168.10.97:2379"
11    ETCD_LISTEN_PEER_URLS="http://0.0.0.0:2380"
```

```
12 ETCD_LISTEN_CLIENT_URLS="http://192.168.10.97:2379"
13
14 systemctl enable etcd
15 systemctl restart etcd
```

B.5. Servicio de Identidad (Keystone)

Keystone proporciona autenticación y autorización centralizada.

B.5.1. Nodo Controller

Se configuró el backend de base de datos y el proveedor de tokens Fernet para manejo seguro de credenciales.

```
1 mysql -u root -p
2     CREATE DATABASE keystone;
3     GRANT ALL PRIVILEGES ON keystone.* TO 'keystone'@'localhost' IDENTIFIED BY '
4         KEYSTONE_DBPASS';
5     GRANT ALL PRIVILEGES ON keystone.* TO 'keystone'@'%' IDENTIFIED BY 'KEYSTONE_DBPASS';
```

```
1 apt install -y keystone
2 nano /etc/keystone/keystone.conf
3     [database]
4     # ...
5     connection = mysql+pymysql://keystone:KEYSTONE_DBPASS@controller/keystone
6
7     [token]
8     # ...
9     provider = fernet
10
11 su -s /bin/sh -c "keystone-manage_db_sync" keystone
12 keystone-manage fernet_setup --keystone-user keystone --keystone-group keystone
13 keystone-manage credential_setup --keystone-user keystone --keystone-group keystone
14 keystone-manage bootstrap --bootstrap-password ADMIN_PASS \
15     --bootstrap-admin-url http://controller:5000/v3/ \
16     --bootstrap-internal-url http://controller:5000/v3/ \
17     --bootstrap-public-url http://controller:5000/v3/ \
18     --bootstrap-region-id RegionOne
19
20 nano /etc/apache2/apache2.conf
21     ServerName controller
22
23 service apache2 restart
24 export OS_USERNAME=admin
25 export OS_PASSWORD=ADMIN_PASS
26 export OS_PROJECT_NAME=admin
27 export OS_USER_DOMAIN_NAME=Default
28 export OS_PROJECT_DOMAIN_NAME=Default
```

```

29 export OS_AUTH_URL=http://controller:5000/v3
30 export OS_IDENTITY_API_VERSION=3
31
32 openstack domain create --description "An_Example_Domain" example
33 openstack project create --domain default --description "Service_Project" service
34 openstack project create --domain default --description "Demo_Project" myproject
35 openstack user create --domain default --password-prompt myuser
36 openstack role create myrole
37 openstack role add --project myproject --user myuser myrole
38
39 unset OS_AUTH_URL OS_PASSWORD
40 openstack --os-auth-url http://controller:5000/v3 \
41 --os-project-domain-name Default --os-user-domain-name Default \
42 --os-project-name admin --os-username admin token issue
43
44
45 openstack --os-auth-url http://controller:5000/v3 \
46 --os-project-domain-name Default --os-user-domain-name Default \
47 --os-project-name myproject --os-username myuser token issue
48
49 nano admin-openrc
50     export OS_PROJECT_DOMAIN_NAME=Default
51     export OS_USER_DOMAIN_NAME=Default
52     export OS_PROJECT_NAME=admin
53     export OS_USERNAME=admin
54     export OS_PASSWORD=ADMIN_PASS
55     export OS_AUTH_URL=http://controller:5000/v3
56     export OS_IDENTITY_API_VERSION=3
57     export OS_IMAGE_API_VERSION=2

```

B.6. Servicio de Gestión de imágenes (Glance)

Glance gestiona las imágenes de máquinas virtuales.

B.6.1. Nodo Controller

```

1 mysql -u root -p
2     CREATE DATABASE glance;
3     GRANT ALL PRIVILEGES ON glance.* TO 'glance'@'localhost' IDENTIFIED BY 'GLANCE_DBPASS';
4     GRANT ALL PRIVILEGES ON glance.* TO 'glance'@'%' IDENTIFIED BY 'GLANCE_DBPASS';
5
6 . admin-openrc
7 openstack user create --domain default --password-prompt glance
8 openstack role add --project service --user glance admin
9 openstack service create --name glance --description "OpenStack_Image" image
10 openstack endpoint create --region RegionOne image public http://controller:9292
11 openstack endpoint create --region RegionOne image internal http://controller:9292

```

```

12 openstack endpoint create --region RegionOne image admin http://controller:9292
13
14 apt install -y glance
15 nano /etc/glance/glance-api.conf
16     [database]
17     # ...
18     connection = mysql+pymysql://glance:GLANCE_DBPASS@controller/glance
19
20     [keystone_authtoken]
21     # ...
22     www_authenticate_uri = http://controller:5000
23     auth_url = http://controller:5000
24     memcached_servers = controller:11211
25     auth_type = password
26     project_domain_name = Default
27     user_domain_name = Default
28     project_name = service
29     username = glance
30     password = gtloaqnhcde3
31
32     [paste_deploy]
33     # ...
34     flavor = keystone
35
36     [glance_store]
37     # ...
38     stores = file,http
39     default_store = file
40     filesystem_store_datadir = /var/lib/glance/images/
41
42 su -s /bin/sh -c "glance-manage_db_sync" glance
43 service glance-api restart
44
45 . admin-openrc
46 wget http://download.cirros-cloud.net/0.4.0/cirros-0.4.0-x86_64-disk.img
47 glance image-create --name "cirros" --file cirros-0.4.0-x86_64-disk.img --disk-format qcow2 --
    container-format bare --visibility=public
48 glance image-list

```

B.7. Servicio de Gestión de recursos disponibles (Placement)

El servicio Placement se instaló en el nodo Controller como componente encargado de gestionar y consultar la disponibilidad de recursos dentro del entorno OpenStack.

B.7.1. Nodo Controller

```

1 mysql -u root -p

```

```

2      CREATE DATABASE placement;
3      GRANT ALL PRIVILEGES ON placement.* TO 'placement'@'localhost' IDENTIFIED BY '
        PLACEMENT_DBPASS';
4      GRANT ALL PRIVILEGES ON placement.* TO 'placement'@'%' IDENTIFIED BY 'PLACEMENT_DBPASS';
5
6  . admin-openrc
7  openstack user create --domain default --password-prompt placement
8  openstack role add --project service --user placement admin
9  openstack service create --name placement --description "Placement_API" placement
10 openstack endpoint create --region RegionOne placement public http://controller:8778
11 openstack endpoint create --region RegionOne placement internal http://controller:8778
12 openstack endpoint create --region RegionOne placement admin http://controller:8778
13
14 apt install -y placement-api
15 nano /etc/placement/placement.conf
16     [placement_database]
17     connection = mysql+pymysql://placement:PLACEMENT_DBPASS@controller/placement
18     [api]
19     auth_strategy = keystone
20     [keystone_authtoken]
21     auth_url = http://controller:5000/v3
22     memcached_servers = controller:11211
23     auth_type = password
24     project_domain_name = Default
25     user_domain_name = Default
26     project_name = service
27     username = placement
28     password = p0loaqcde3mje3nht5
29
30 su -s /bin/sh -c "placement-manage_db_sync" placement
31 service apache2 restart
32
33 . admin-openrc
34 placement-status upgrade check

```

B.8. Servicio de Gestión de recursos de las instancias de máquinas virtuales dentro de OpenStack (Nova)

El servicio Nova se instaló para gestionar el ciclo de vida y los recursos asignados a las instancias de máquinas virtuales dentro del entorno OpenStack.

B.8.1. Nodo Controller

```

1 mysql -u root -p
2     CREATE DATABASE nova_api;
3     CREATE DATABASE nova;
4     CREATE DATABASE nova_cell0;

```

```
5
6     GRANT ALL PRIVILEGES ON nova_api.* TO 'nova'@'localhost' IDENTIFIED BY 'NOVA_DBPASS';
7     GRANT ALL PRIVILEGES ON nova_api.* TO 'nova'@'%' IDENTIFIED BY 'NOVA_DBPASS';
8     GRANT ALL PRIVILEGES ON nova.* TO 'nova'@'localhost' IDENTIFIED BY 'NOVA_DBPASS';
9     GRANT ALL PRIVILEGES ON nova.* TO 'nova'@'%' IDENTIFIED BY 'NOVA_DBPASS';
10    GRANT ALL PRIVILEGES ON nova_cell0.* TO 'nova'@'localhost' IDENTIFIED BY 'NOVA_DBPASS';
11    GRANT ALL PRIVILEGES ON nova_cell0.* TO 'nova'@'%' IDENTIFIED BY 'NOVA_DBPASS';
12
13    . admin-openrc
14    openstack user create --domain default --password-prompt nova
15    openstack role add --project service --user nova admin
16    openstack service create --name nova --description "OpenStack Compute" compute
17    openstack endpoint create --region RegionOne compute public http://controller:8774/v2.1
18    openstack endpoint create --region RegionOne compute internal http://controller:8774/v2.1
19    openstack endpoint create --region RegionOne compute admin http://controller:8774/v2.1
20
21    apt install -y nova-api nova-conductor nova-novncproxy nova-scheduler
22    nano /etc/nova/nova.conf
23        [DEFAULT]
24        # ...
25        transport_url = rabbit://openstack:p0aqsww@controller:5672/
26        my_ip = 192.168.10.97
27
28        [api_database]
29        # ...
30        connection = mysql+pymysql://nova:NOVA_DBPASS@controller/nova_api
31
32        [database]
33        # ...
34        connection = mysql+pymysql://nova:NOVA_DBPASS@controller/nova
35
36        [api]
37        # ...
38        auth_strategy = keystone
39
40        [keystone_authtoken]
41        # ...
42        www_authenticate_uri = http://controller:5000/
43        auth_url = http://controller:5000/
44        memcached_servers = controller:11211
45        auth_type = password
46        project_domain_name = Default
47        user_domain_name = Default
48        project_name = service
49        username = nova
50        password = nho9vfaq
51
52        [vnc]
53        enabled = true
```

```

54     # ...
55     server_listen = $my_ip
56     server_proxyclient_address = $my_ip
57
58     [glance]
59     # ...
60     api_servers = http://controller:9292
61
62     [oslo_concurrency]
63     # ...
64     lock_path = /var/lib/nova/tmp
65
66     [placement]
67     # ...
68     region_name = RegionOne
69     project_domain_name = Default
70     project_name = service
71     auth_type = password
72     user_domain_name = Default
73     auth_url = http://controller:5000/v3
74     username = placement
75     password = p0loaqcde3mje3nht5
76
77 su -s /bin/sh -c "nova-manage _api_db_sync" nova
78 su -s /bin/sh -c "nova-manage _cell_v2_map_cell0" nova
79 su -s /bin/sh -c "nova-manage _cell_v2_create_cell _--name=cell1 _--verbose" nova
80 su -s /bin/sh -c "nova-manage _db_sync" nova
81 su -s /bin/sh -c "nova-manage _cell_v2_list_cells" nova
82
83
84 service nova-api restart
85 service nova-scheduler restart
86 service nova-conductor restart
87 service nova-novncproxy restart

```

B.8.2. Nodo Compute

```

1 apt install -y nova-compute
2 egrep -c '(vmx|svm)' /proc/cpuinfo
3 nano /etc/nova/nova.conf
4     [DEFAULT]
5     # ...
6     transport_url = rabbit://openstack:p0aqswsw@controller
7     my_ip = 192.168.10.100
8
9     [api]
10     ...
11     auth_strategy = keystone

```

```
12     [keystone_authtoken]
13     # ...
14     www_authenticate_uri = http://controller:5000/
15     auth_url = http://controller:5000/
16     memcached_servers = controller:11211
17     auth_type = password
18     project_domain_name = Default
19     user_domain_name = Default
20     project_name = service
21     username = nova
22     password = nho9vfaq
23
24     [vnc]
25     # ...
26     enabled = true
27     server_listen = 0.0.0.0
28     server_proxyclient_address = $my_ip
29     novncproxy_base_url = http://controller:6080/vnc_auto.html
30
31     [glance]
32     # ...
33     api_servers = http://controller:9292
34
35     [oslo_concurrency]
36     # ...
37     lock_path = /var/lib/nova/tmp
38
39     [placement]
40     # ...
41     region_name = RegionOne
42     project_domain_name = Default
43     project_name = service
44     auth_type = password
45     user_domain_name = Default
46     auth_url = http://controller:5000/v3
47     username = placement
48     password = p0loaqcde3mje3nht5
49
50
51     [libvirt]
52     # ...
53     virt_type = qemu
54
55
56 service nova-compute restart
```

Para establecer la correcta comunicación con el nodo Controller, dentro de este, se ejecuta el siguiente comando:

```
1 . admin-openrc
```



```

2 openstack compute service list --service nova-compute
3 su -s /bin/sh -c "nova-manage cell_v2 discover_hosts --verbose" nova

```

B.9. Servicio de administración de redes virtuales (Neutron)

B.9.1. Nodo Controller

Se configuró el plugin ML2 con el mecanismo LinuxBridge.

```

1 mysql -u root -p
2     CREATE DATABASE neutron;
3     GRANT ALL PRIVILEGES ON neutron.* TO 'neutron'@'localhost' IDENTIFIED BY 'NEUTRON_DBPASS'
4     ;
5     GRANT ALL PRIVILEGES ON neutron.* TO 'neutron'@'%' IDENTIFIED BY 'NEUTRON_DBPASS';
6
7 . admin-openrc
8 openstack user create --domain default --password-prompt neutron
9 openstack role add --project service --user neutron admin
10 openstack service create --name neutron --description "OpenStack Networking" network
11 openstack endpoint create --region RegionOne network public http://controller:9696
12 openstack endpoint create --region RegionOne network internal http://controller:9696
13
14 apt install -y neutron-server neutron-plugin-ml2 neutron-linuxbridge-agent neutron-dhcp-agent
15     neutron-metadata-agent
16 sysctl net.bridge.bridge-nf-call-iptables ## CHECAR DEBERIA SER 1
17 sysctl net.bridge.bridge-nf-call-ip6tables ## CHECAR DEBERIA SER 1
18
19 nano /etc/neutron/neutron.conf
20     [database]
21     # ...
22     connection = mysql+pymysql://neutron:NEUTRON_DBPASS@controller/neutron
23
24     [DEFAULT]
25     # ...
26     core_plugin = ml2
27     service_plugins =
28     transport_url = rabbit://openstack:p0aqsww@controller
29     auth_strategy = keystone
30     notify_nova_on_port_status_changes = true
31     notify_nova_on_port_data_changes = true
32
33     [keystone_authtoken]
34     # ...
35     www_authenticate_uri = http://controller:5000
36     auth_url = http://controller:5000
37     memcached_servers = controller:11211
38     auth_type = password
39     project_domain_name = Default

```

```

39     user_domain_name = Default
40     project_name = service
41     username = neutron
42     password = nhe3u7t5r4o9nh
43
44     [nova]
45     # ...
46     auth_url = http://controller:5000
47     auth_type = password
48     project_domain_name = Default
49     user_domain_name = Default
50     region_name = RegionOne
51     project_name = service
52     username = nova
53     password = nho9vfaq
54
55     [oslo_concurrency]
56     # ...
57     lock_path = /var/lib/neutron/tmp
58
59 nano /etc/neutron/plugins/ml2/ml2_conf.ini
60
61     [ml2]
62     # ...
63     type_drivers = flat,vlan
64     tenant_network_types =
65     mechanism_drivers = linuxbridge
66     extension_drivers = port_security
67
68     [ml2_type_flat]
69     flat_networks = provider
70
71     [securitygroup]
72     # ...
73     enable_ipset = true
74
75 nano /etc/neutron/plugins/ml2/linuxbridge_agent.ini
76     [linux_bridge]
77     physical_interface_mappings = provider:enp1s0
78
79     [vxlan]
80     enable_vxlan = false
81
82     [securitygroup]
83     # ...
84     enable_security_group = true
85     firewall_driver = neutron.agent.linux.iptables_firewall.IptablesFirewallDriver
86
87 nano /etc/neutron/dhcp_agent.ini

```

```

88
89     [DEFAULT]
90     # ...
91     interface_driver = linuxbridge
92     dhcp_driver = neutron.agent.linux.dhcp.Dnsmasq
93     enable_isolated_metadata = true
94
95 nano /etc/neutron/metadata_agent.ini
96     [DEFAULT]
97     # ...
98     nova_metadata_host = controller
99     metadata_proxy_shared_secret = METADATA_SECRET
100
101 nano /etc/nova/nova.conf
102     [neutron]
103     # ...
104     auth_url = http://controller:5000
105     auth_type = password
106     project_domain_name = default
107     user_domain_name = default
108     region_name = RegionOne
109     project_name = service
110     username = neutron
111     password = nhe3u7t5r4o9nh
112     service_metadata_proxy = true
113     metadata_proxy_shared_secret = METADATA_SECRET
114
115 nano /etc/mysql/mariadb.conf.d/99-openstack.cnf
116     innodb_large_prefix = on #AGREGAR
117 systemctl restart mariadb.service
118
119 mysql -u root -p
120     show variables like '%innodb_large_prefix%';
121     use neutron;
122     CREATE TABLE ovn_hash_ring (
123     node_uuid VARCHAR(36) NOT NULL,
124         group_name VARCHAR(256) NOT NULL,
125         hostname VARCHAR(256) NOT NULL,
126         created_at DATETIME NOT NULL,
127         updated_at DATETIME NOT NULL,
128         PRIMARY KEY (node_uuid, group_name)
129     )ENGINE=InnoDB DEFAULT CHARSET=utf8 ROW_FORMAT=DYNAMIC;
130
131 su -s /bin/sh -c "neutron-db-manage --config-file /etc/neutron/neutron.conf --config-file /etc/
    neutron/plugins/ml2/ml2_conf.ini upgrade head" neutron
132
133 mysql -u root -p
134     use neutron;
135     ALTER TABLE subnets DROP FOREIGN KEY subnets_ibfk_1;

```

```

136
137 su -s /bin/sh -c "neutron-db-manage --config-file /etc/neutron/neutron.conf --config-file /etc/
    neutron/plugins/ml2/ml2_conf.ini upgrade head" neutron
138
139 service nova-api restart
140 service neutron-server restart
141 service neutron-linuxbridge-agent restart
142 service neutron-dhcp-agent restart
143 service neutron-metadata-agent restart
144
145 . admin-openrc
146 openstack network create --share --external --provider-physical-network provider --provider-
    network-type flat provider
147 openstack subnet create --network provider --allocation-pool start=192.168.74.100,end
    =192.168.74.250 --dns-nameserver 132.248.10.2 --gateway 192.168.10.2 --subnet-range
    192.168.74.0/24 provider

```

B.9.2. Nodo Controller

El agente LinuxBridge permite que las instancias se conecten a la red física provider.

```

1 apt install -y neutron-linuxbridge-agent
2 sysctl net.bridge.bridge-nf-call-iptables
3 sysctl net.bridge.bridge-nf-call-ip6tables
4
5 nano /etc/neutron/neutron.conf
6
7     [DEFAULT]
8     # ...
9     transport_url = rabbit://openstack:p0aqsww@controller
10    auth_strategy = keystone
11
12    [keystone_authtoken]
13    #
14    www_authenticate_uri = http://controller:5000
15    auth_url = http://controller:5000
16    memcached_servers = controller:11211
17    auth_type = password
18    project_domain_name = Default
19    user_domain_name = Default
20    project_name = service
21    username = neutron
22    password = nhe3u7t5r4o9nh
23
24    [oslo_concurrency]
25    # ...
26    lock_path = /var/lib/neutron/tmp
27
28 nano /etc/neutron/plugins/ml2/linuxbridge_agent.ini

```

```

29     [linux_bridge]
30     physical_interface_mappings = provider:enp1s0
31
32     [vxlan]
33     enable_vxlan = false
34
35     [securitygroup]
36     # ...
37     enable_security_group = true
38     firewall_driver = neutron.agent.linux.iptables_firewall.IptablesFirewallDriver
39
40 nano /etc/nova/nova.conf
41     [neutron]
42     # ...
43     auth_url = http://controller:5000
44     auth_type = password
45     project_domain_name = Default
46     user_domain_name = Default
47     region_name = RegionOne
48     project_name = service
49     username = neutron
50     password = nhe3u7t5r4o9nh
51
52 service nova-compute restart
53 service neutron-linuxbridge-agent restart

```

B.10. Servicio de administración vía interfaz Web (Horizon)

Horizon proporciona la interfaz web para administración de OpenStack.

B.10.1. Nodo Controller

Se configuró Apache con soporte SSL.

```

1 apt install -y openstack-dashboard
2 nano /etc/openstack-dashboard/local_settings.py
3
4     OPENSTACK_HOST = "controller"
5     ALLOWED_HOSTS = ['*']
6     SESSION_ENGINE = 'django.contrib.sessions.backends.cache'
7     CACHES = {
8         'default': {
9             'BACKEND': 'django.core.cache.backends.memcached.MemcachedCache',
10            'LOCATION': 'controller:11211',
11        }
12    }
13
14     OPENSTACK_KEYSTONE_URL = "http://%s:5000/identity/v3" % OPENSTACK_HOST

```

```

15 OPENSTACK_KEYSTONE_MULTIDOMAIN_SUPPORT = True
16 OPENSTACK_API_VERSIONS = {
17     "identity": 3,
18     "image": 2,
19     "volume": 3,
20 }
21
22 OPENSTACK_KEYSTONE_DEFAULT_DOMAIN = "Default"
23 OPENSTACK_KEYSTONE_DEFAULT_ROLE = "user"
24 OPENSTACK_NEUTRON_NETWORK = {
25     'enable_router': False,
26     'enable_quotas': False,
27     'enable_ipv6': False,
28     'enable_distributed_router': False,
29     'enable_ha_router': False,
30     'enable_lb': False,
31     'enable_firewall': False,
32     'enable_vpn': False,
33     'enable_fip_topology_check': False,
34 }
35 TIME_ZONE = "America/Mexico_City"
36
37 nano /etc/apache2/conf-available/openstack-dashboard.conf
38     WSGIApplicationGroup %{GLOBAL}
39
40 nano /etc/apache2/ports.conf
41     ##### CAMBIAR PUERTO QUE SE REQUIERA
42
43 openssl req -new -x509 -nodes -out horizon.crt -keyout horizon.key
44 mv horizon.crt horizon.key /etc/openstack-dashboard/
45 chmod 0600 /etc/openstack-dashboard/horizon.*
46 nano /etc/apache2/conf-available/openstack-dashboard.conf
47     <VirtualHost *:80>
48         ServerName controller
49     <IfModule mod_rewrite.c>
50         RewriteEngine On
51         RewriteCond %{HTTPS} off
52         RewriteRule (.*?) https://%{HTTP_HOST}%{REQUEST_URI}
53     </IfModule>
54     <IfModule !mod_rewrite.c>
55         RedirectPermanent / https://192.168.10.98
56     </IfModule>
57 </VirtualHost>
58
59 <VirtualHost *:441>
60     ServerName controller
61
62     SSLEngine On
63     # Remember to replace certificates and keys with valid paths in your environment

```

```

64 SSLCertificateFile /etc/openstack-dashboard/horizon.crt
65 #SSLCACertificateFile /etc/apache2/SSL/openstack.example.com.crt
66 SSLCertificateKeyFile /etc/openstack-dashboard/horizon.key
67 #SetEnvIf User-Agent ".*MSIE.*" nokeepalive ssl-unclean-shutdown
68
69 WSGIScriptAlias /horizon /usr/share/openstack-dashboard/openstack_dashboard/wsgi.py
    process-group=horizon
70 WSGIDaemonProcess horizon user=horizon group=horizon processes=3 threads=10 display-name
    =%{GROUP}
71 WSGIProcessGroup horizon
72 WSGIApplicationGroup %{GLOBAL}
73
74 Alias /static /var/lib/openstack-dashboard/static/
75 Alias /horizon/static /var/lib/openstack-dashboard/static/
76
77 #<Directory /usr/share/openstack-dashboard/openstack_dashboard>
78 # Require all granted
79 #</Directory>
80
81 <Directory /var/lib/openstack-dashboard/static>
82     Require all granted
83 </Directory>
84
85 <Directory /usr/share/openstack-dashboard/openstack_dashboard>
86 <ifVersion < 2.4>
87     #Order allow,deny
88     Allow from all
89 </ifVersion>
90 <ifVersion >= 2.4>
91 Options All
92     AllowOverride All
93     Require all granted
94 </ifVersion>
95 </Directory>
96
97 <Directory /usr/share/openstack-dashboard/static>
98 <ifVersion >= 2.4>
99 Options All
100     AllowOverride All
101     Require all granted
102 </ifVersion>
103 <IfVersion < 2.4>
104     #Order allow,deny
105     Allow from all
106 </IfVersion>
107 </Directory>
108
109 </VirtualHost>
110

```

```
111 chmod 755 /var/lib/openstack-dashboard/static/dashboard/  
112 chmod 755 -R /var/lib/openstack-dashboard/static/dashboard/  
113 service memcached restart  
114 systemctl reload apache2.service
```


Anexo C

Modificación de parámetros dentro de FIT4Python

A continuación se muestran los parámetros de configuración modificados del programa FIT4Python [6], en específico, el archivo `FIT4Python.git/pyFaultInjection/resources/settings.yml`

El archivo `settings.yml` define la configuración general para la ejecución de pruebas de inyección de fallas sobre el entorno OpenStack. Este archivo especifica los nodos remotos, credenciales de acceso, ubicación de repositorios, scripts de ejecución y tipos de fallas a evaluar.

Consola C.1: Configuración de `FIT4Python.git/pyFaultInjection/resources/settings.yml`

```
1 # Array of remote worker details (no fixed limit)
2 remote-workers:
3   - remote-worker:
4     # Remote machine name (just for identification)
5     hostname: controller.tda
6     # SSH user to connect to
7     user: ubuntu
8     # SSH password
9     password: ubuntu
10    # Remote machine address
11    host: 192.168.10.97
12    # SSH port
13    port: 22
14    # SSH Docker port - unit and functional tests are dockerized
15    docker-port: 55022
16    # ssh public key if the remote machine requires key authentication (blank if not)
17    local-ssh-public-key:
18    # path to clone the repo for integration tests
19    remote-nova-clone-location: /opt/stack
20    # path to copy the remote setup and test execution scripts
21    remote-script-location: /home/ubuntu/pyFaultInjectionScripts
22    # path to clone the nova repo inside the docker container
23    remote-nova-clone-docker-location: /home/ubuntu
24    # path to copy the remote setup and test execution scripts inside the docker container
25    remote-script-docker-location: /home/ubuntu/pyFaultInjectionScripts
```

```

26     #setup-script-arguments:
27
28
29 # File to be injected (must be in the resources of the tool)
30 remote-injection-files:
31     - resources/api.py
32
33 use-docker: true
34 dockerfile: resources/Dockerfile
35
36 # Configuration for setup and test execution scripts
37 scripts:
38     setup: resources/remote_setup.sh
39     setup-devstack: resources/launch_devstack.sh
40     relaunch-devstack: resources/relaunch_devstack.sh
41     run-unit-tests: resources/run_unit_tests.sh
42     run-functional-tests: resources/run_functional_tests.sh
43     run-integration-tests: resources/run_integration_tests.sh
44
45 run-unit-tests: true
46 run-functional-tests: true
47 # Integration tests are disabled as the contribution to test effectiveness is minimal
48 run-integration-tests: false
49 launch-openstack: false # not necessary for unit and functional tests
50 # Limit the number of injections per fault type
51 max-number-injections-per-fault-type: 0 # 0 -> not limited
52
53 test-results-folder: test_results
54
55 report-update-frequency-seconds: 60
56
57 # Array of fault type IDS to execute
58 fault-types:
59     - MPFC
60     - MIA
61     - WLEC
62     - MVIE
63     - MRS
64     - MCEFL
65     - MVIV
66     - MSFC
67     - MVAV
68     - WVIV
69     - WVAL
70     - WIDS
71     - WVAE
72     - WIDSL
73     - WSUT
74     - MSC

```

Anexo D

Archivo main.py de FIT4Python

El archivo main.py constituye el punto de entrada del sistema de Fault Injection. Su función principal función es inicializar la configuración del entorno y ejecutar el gestor de inyecciones de fallas.

Consola D.1: Configuración de FIT4Python.git/pyFaultInjection/___main___py

```
1 import logging
2 import argparse
3 from pyFaultInjection.fault_injection_settings import FaultInjectionSettings
4 from pyFaultInjection.injection_manager import InjectionManager
5
6
7 def main():
8
9     logging_format = "[% (asctime)s]_[% (filename)s: %(lineno)d]_[% (levelname)s]_[% (message)s]"
10    logging.basicConfig(format=logging_format, level=logging.INFO,
11                        datefmt="%H: %M: %S")
12
13    settings = FaultInjectionSettings()
14
15    parser = argparse.ArgumentParser(description='Python_Fault_Injection')
16    parser.add_argument('-r', '--resume', type=str, help="Resume_testing_giving_report")
17    args = parser.parse_args()
18    if args.resume:
19        InjectionManager(settings).run(args.resume)
20    else:
21        InjectionManager(settings).run()
22
23 if __name__ == '__main__':
24    main()
```

Anexo E

Archivo injection__manager.py de FIT4Python

El archivo presentado en este anexo contiene la implementación del núcleo del sistema de inyección de fallas. Su función principal es coordinar la transformación del código fuente (API de Nova), y ejecutar las pruebas en entornos remotos.

Consola E.1: Configuración de FIT4Python.git/pyFaultInjection/injection__manager.py

```
1 import concurrent.futures
2 import logging
3 import os
4 import threading
5 from datetime import datetime
6
7 import pandas
8 import paramiko
9 from tqdm import tqdm
10
11 from faults.fault_factory import FaultFactory
12 from pyFaultInjection import fault_injection_settings
13 from pyFaultInjection import utils
14 from pyFaultInjection.remote_machine_pool import RemoteMachinePool
15 from pyFaultInjection.report_generator import ReportGenerator, TestExecution
16 from pyFaultInjection.utils import local_folder
17
18
19 def line_buffered(f):
20     line_buf = ""
21     while not f.channel.exit_status_ready():
22         line_buf += f.read(1).decode('UTF-8')
23         if line_buf.endswith('\n'):
24             yield line_buf
25             line_buf = ''
26
27
```

```

28 def _sftp_upload_files(ssh_connection: paramiko.SSHClient, files_dict: dict, make_executable=
    False):
29     sftp_client = ssh_connection.open_sftp()
30     for key in files_dict:
31         sftp_client.put(os.path.join(local_folder, key), files_dict[key])
32         if make_executable:
33             ssh_connection.exec_command("chmod+x" + files_dict[key])
34     sftp_client.close()
35
36
37 def _check_if_file_exists(ssh_connection: paramiko.SSHClient, path):
38     sftp_client = ssh_connection.open_sftp()
39     exists = False
40     try:
41         sftp_client.stat(path)
42         exists = True
43     except IOError:
44         exists = False
45     sftp_client.close()
46     return exists
47
48
49 def _sftp_get_files(ssh_connection: paramiko.SSHClient, files_dict: dict):
50     sftp_client = ssh_connection.open_sftp()
51     for key in files_dict:
52         sftp_client.get(key, os.path.join(local_folder, files_dict[key]))
53     sftp_client.close()
54
55
56 def _delete_all_files_in_remote_folder(ssh_connection, path):
57     sftp_client = ssh_connection.open_sftp()
58     remote_script_location_files = sftp_client.listdir(path)
59     for file in remote_script_location_files:
60         file_path = os.path.join(path, file)
61         # logging.info("{} - Removing {}".format(hostname, file_path))
62         sftp_client.remove(file_path)
63     sftp_client.close()
64
65
66 def _setup_remote(worker: fault_injection_settings.RemoteWorker,
67                 ssh: paramiko.SSHClient,
68                 ssh_docker: paramiko.SSHClient,
69                 settings: fault_injection_settings,
70                 remote_machine_pool: RemoteMachinePool):
71
72     # Create remote script folder
73     sftp_client = ssh.open_sftp()
74     try:
75         sftp_client.mkdir(worker.script_location)

```

```

76 except Exception:
77     logging.warning("{}_Folder{}_already_exists_{}_overwriting_files".format(worker.
80         hostname, worker.script_location))
78     _delete_all_files_in_remote_folder(ssh, worker.script_location)
79 finally:
80     sftp_client.close()
81     # Upload the scripts to the remote folder
82     upload_files = {}
83     for script in settings.get_local_scripts():
84         upload_files[script] = os.path.join(worker.script_location, os.path.basename(script))
85     _sftp_upload_files(ssh, upload_files, True)
86
87     execution = ""
88     if settings.run_unit_tests is True or settings.run_functional_tests is True:
89         execution = "docker"
90     if settings.run_integration_tests is True:
91         execution = "all_>_test.txt_&&" + os.path.join(worker.script_location,
92             os.path.basename(settings.
93                 remote_setup_devstack_script))
94
95     # Execute setup script
96     logging.info("{}_Executing_remote_setup".format(worker.hostname))
97     ssh_stdin, ssh_stdout, ssh_stderr = ssh.exec_command(os.path.join(worker.script_location,
98         os.path.basename(settings.
99             remote_setup_script)) +
100         "_" + execution,
101         get_pty=False)
102
103     for l in line_buffered(ssh_stdout):
104         logging.info(worker.hostname + "-" + l.rstrip())
105
106     logging.info("{}_Setup_done".format(worker.hostname))
107     remote_machine_pool.release(worker, ssh, ssh_docker)
108
109 def _execute_test(vm, ssh, ssh_docker, original_code, transformed_code, fault_type, line,
110     file, report_generator: ReportGenerator, settings, remote_vm_pool:
111     RemoteMachinePool):
112
113     logging.info("Executing_test_on{}".format(vm.host))
114
115     new_file_name = os.path.join(local_folder, utils.random_string_generator() + ".tmp")
116
117     # Create the new file py file
118     new_file = open(new_file_name, "w")
119     new_file.write(transformed_code)
120     new_file.close()
121
122     logging.info("Connected_to{}".format(vm.hostname))

```

```

121
122     # Upload the modified code
123     try:
124         _sftp_upload_files(ssh_docker, {"resources/tox_unit.ini": os.path.join(vm.
125             clone_docker_location, "nova/tox.ini"),
126                                     new_file_name: os.path.join(vm.clone_docker_location, "nova/nova/
127                                         compute/api.py")})
128
129     except Exception as ex:
130         logging.error("{}_Error_uploading_the_transformed_code".format(ex))
131
132     # Execute unit tests
133     if settings.run_unit_tests:
134         logging.info("{}_Executing_unit_tests_original_code->\n{}".format(vm.hostname,
135             original_code))
136         ssh_stdin, ssh_stdout, ssh_stderr = ssh_docker.exec_command("bash-c" + os.path.join(
137             vm.script_docker_location,
138             settings.remote_run_unit_tests_script.split("/")[-1]), get_pty=True)
139
140         exit_status = ssh_stdout.channel.recv_exit_status()
141         # if exit_status:
142         # logging.error("{} - Error running unit tests".format(vm.hostname))
143
144     # Execute functional tests
145     if settings.run_functional_tests:
146         logging.info("{}_Executing_functional_tests".format(vm.hostname))
147
148         _sftp_upload_files(ssh_docker, {"./resources/tox_functional.ini": os.path.join(vm.
149             clone_docker_location, "nova/tox.ini")})
150
151         ssh_stdin, ssh_stdout, ssh_stderr = ssh_docker.exec_command("bash-c" + os.path.join(
152             vm.script_docker_location,
153             settings.remote_run_functional_tests_script.split("/")[-1]), get_pty=True)
154
155         exit_status = ssh_stdout.channel.recv_exit_status()
156         # if exit_status:
157         # logging.warning("{} - Error running functional tests".format(vm.hostname))
158
159     if settings.run_integration_tests:
160         logging.info("{}_Executing_integration_tests".format(vm.hostname))
161
162         _sftp_upload_files(ssh, {new_file_name: os.path.join(vm.clone_location, "nova/nova/
163             compute/api.py")})
164
165         ssh_stdin, ssh_stdout, ssh_stderr = ssh.exec_command("bash-c" + os.path.join(
166             vm.script_location,
167             settings.remote_relaunch_devstack_script.split("/")[-1]), get_pty=True)
168
169         exit_status = ssh_stdout.channel.recv_exit_status()

```



```

165     ssh_stdin, ssh_stdout, ssh_stderr = ssh.exec_command("bash_c_" + os.path.join(
166         vm.script_location,
167         settings.remote_run_integration_tests_script.split("/")[-1]), get_pty=True)
168
169     exit_status = ssh_stdout.channel.recv_exit_status()
170
171     now = datetime.now()
172     unit_test_report_name = now.strftime("%d%m%Y%H%M%S") + "_unit_result_" + str(line)
173     functional_test_report_name = now.strftime("%d%m%Y%H%M%S") + "_functional_result_" + str(
174         line)
175     integration_test_report_name = now.strftime("%d%m%Y%H%M%S") + "_integration_result_" + str
176         (line)
177
178     try:
179         if not os.path.isdir("./test_results"):
180             os.mkdir("./test_results")
181
182         get_files_dict = {}
183         if settings.run_unit_tests:
184             get_files_dict[os.path.join(vm.clone_docker_location, "nova/unit_tests.txt")] = os.
185                 path.join(
186                     "./test_results", unit_test_report_name)
187         if settings.run_functional_tests:
188             get_files_dict[os.path.join(vm.clone_docker_location, "nova/functional_tests.txt")] =
189                 os.path.join(
190                     "./test_results", functional_test_report_name)
191
192         if len(get_files_dict) != 0:
193             _sftp_get_files(ssh_docker, get_files_dict)
194
195         if settings.run_integration_tests:
196             _sftp_get_files(ssh, {os.path.join(vm.clone_location, "tempest/integration_tests.txt"
197                 ): os.path.join(
198                     "./test_results", integration_test_report_name)})
199
200     except Exception:
201         logging.error("{}_Error_getting_the_results_from_the_remote_location".format(vm.
202             hostname))
203
204     logging.info("{}_Closing_sftp_connection".format(vm.hostname))
205
206     # Delete file after upload
207     os.remove(new_file_name)
208
209     # logging.debug(original_code + " -> " + transformed_code + "->line " + str(line))
210
211     unit_test_report = os.path.join(local_folder, "./test_results",
212         unit_test_report_name) if settings.run_unit_tests else ''
213

```



```
253     number_of_injections = self._settings.number_injections_per_fault_type
254
255     for file in self._settings.fault_injection_files:
256         for fault_type in self._settings.fault_types:
257             fault_transformer = FaultFactory.create_fault_injector(os.path.join(local_folder,
258                                     file), fault_type)
259             total_number_of_injections = fault_transformer.count_all()
260             n_injected = 0
261             with concurrent.futures.ThreadPoolExecutor(max_workers=n_vms) as executor:
262                 for original_code, transformation, line in tqdm(fault_transformer.transform(),
263                                     desc=fault_type,
264                                     total=total_number_of_injections):
265                     if report:
266                         if self._check_fault_ran(fault_type, line):
267                             continue
268                     if number_of_injections == 0 or n_injected < number_of_injections:
269                         vm, ssh, ssh_docker = remote_machines_pool.acquire()
270                         executor.submit(_execute_test, vm, ssh, ssh_docker, original_code,
271                                     transformation,
272                                     fault_transformer.get_fault_id(), line,
273                                     file, report_generator, self._settings,
274                                     remote_machines_pool)
275                         n_injected += 1
276                     else:
277                         break
278
279     report_generator.set_terminate()
280     rg_thread.join()
281
282 def _check_fault_ran(self, fault_type, line):
283     ran = False
284     for ind in self._report.index:
285         ran = self._report["FAULT_TYPE"][ind] == fault_type and \
286             self._report['CODE_LINE'][ind] == line
287     if ran:
288         break
289     return ran
```

Anexo F

Archivo wlec_fault.py de FIT4Python

El código presentado implementa la clase WLECFault, la cual hereda de la clase base Fault y define una estrategia específica de transformación del código fuente mediante manipulación del Árbol de Sintaxis Abstracta (AST).

Consola F.1: Configuración de FIT4Python.git/faults/wlec_fault.py

```
1 import random
2
3 from redbaron import (RedBaron, ComparisonOperatorNode, BooleanOperatorNode)
4
5 from faults.fault import Fault
6
7
8 class WLECFault(Fault):
9
10     FAULT_TYPE = "WLEC"
11
12     def __init__(self, file):
13         super(WLECFault, self).__init__(file)
14
15         self._index = 1
16
17     def transform(self):
18         checking_list = self.AST_FILE.find_all("IfNode")
19         for index in range(0, len(checking_list)):
20
21             original_node = checking_list[index].copy()
22             f = open(self.FILE, "r")
23             if f.mode == 'r':
24                 ast_tmp_file = RedBaron(f.read())
25             else:
26                 raise Exception("Cannot open {} in read mode".format(self.FILE))
27             _ast_copy = ast_tmp_file.find_all("IfNode")
```

```

28         line = checking_list[index].value.absolute_bounding_box.top_left.line
29
30         count_failed = 0
31         try:
32             if isinstance(_ast_copy[index].test.value, str):
33                 _ast_copy[index].test.value = 'not_' + _ast_copy[index].test.value
34         except:
35             count_failed += 1
36         try:
37             if isinstance(checking_list[index].test.value, ComparisonOperatorNode):
38                 if _ast_copy[index].test.value.first == 'in':
39                     _ast_copy[index].test.value.first = 'not_in'
40                 elif _ast_copy[index].test.value.first == 'not_in':
41                     _ast_copy[index].test.value.first = 'in'
42                 elif _ast_copy[index].test.value.first in ['==', '!=', '>', '<', '<=', '>=']:
43                     _ast_copy[index].test.value.first = WLECFault._get_random_comparator(
44                         _ast_copy[index].test.value.first)
45                 else:
46                     count_failed += 1
47         except:
48             count_failed += 1
49         try:
50             if isinstance(_ast_copy[index].test, BooleanOperatorNode):
51                 _ast_copy[index].test.value = 'or' if _ast_copy[index].test.value == 'and'
52                 else 'and'
53         except:
54             count_failed += 1
55
56         if count_failed == 3:
57             continue
58
59         if original_node.dumps() == _ast_copy[index].dumps():
60             continue
61         yield original_node.dumps(), _ast_copy[index].root.dumps(), line
62
63     def count_all(self):
64         count = 0
65         for node in self.AST_FILE.find_all("IfNode"):
66             count += 1
67         return count
68
69     @staticmethod
70     def get_fault_id():
71         return WLECFault.FAULT_TYPE
72
73     @staticmethod
74     def _get_random_comparator(original_signal: str):
75
76         signals = ['==', '!=', '>', '<', '<=', '>=']

```

```
75     new_signal = original_signal
76     while new_signal == original_signal:
77         new_signal = signals[random.randint(0, len(signals) - 1)]
78
79     return new_signal
```

Anexo G

Archivo para la segmentación de registros por tipo de ataque SFI

Este script que por nombre lleva: organizador.sh, fue desarrollado para automatizar la segregación de los archivos generados durante el proceso de inyección de fallas. Su función principal es clasificar los archivos modificados generados en el directorio test_results, agrupándolos según su tipo de ataque SFI a través del prefijo identificador.

Consola G.1: script desarrollado organizador.sh

```
1  #!/bin/bash
2
3  # Directorio que contiene los archivos (puedes modificarlo o usar el actual)
4  DIRECTORIO_ORIGEN="./test_results"
5
6  cd "$DIRECTORIO_ORIGEN" || exit 1
7
8  # Iterar sobre los archivos que coincidan con el patrón
9  for archivo in *_transformed_code_*.py; do
10     if [[ -f "$archivo" ]]; then
11         # Extraer el prefijo antes del primer guion bajo
12         prefijo="${archivo%%_*}"
13
14         # Crear el directorio si no existe
15         mkdir -p "$prefijo"
16
17         # Mover el archivo al directorio correspondiente
18         mv "$archivo" "$prefijo/"
19     fi
20 done
21
22 echo "Archivos_organizados_por_prefijo."
```

Anexo H

Archivo para la automatización de pruebas con los archivos API modificados

El script, que por nombre lleva: ataque.sh, implementa el proceso automatizado de evaluación de los archivos modificados generados por el programa FIT4Python.

Su propósito es probar cada versión modificada del archivo api.py dentro del servicio Nova API, medir el comportamiento del sistema y clasificar los resultados según su éxito o fallo.

Consola H.1: script desarrollado ataque.sh

```
1  #!/bin/bash
2  #####
3  # Script para probar archivos api.py organizados por prefijos
4  # - Procesa cada archivo desde /modificados/<prefijo>/
5  # - Registra métricas en /resultados/<prefijo>/
6  # - Registra errores en /errores/<prefijo>/
7  # - Archivos exitosos en /yavistos/<prefijo>/
8  # - Archivos fallidos en /extranos/<prefijo>/
9  #####
10
11  BASE_DIR="/home/andres/modificados"
12  MODIFICADOS="${BASE_DIR}/test_results"
13  RESULTADOS="${BASE_DIR}/resultados"
14  ERRORES="${BASE_DIR}/errores"
15  YAVISTOS="${BASE_DIR}/yavistos"
16  EXTRANOS="${BASE_DIR}/extranos"
17  SERVICE="nova-api"
18
19  # Detener el servicio una sola vez
20  echo "Deteniendo el servicio $SERVICE"
21  systemctl stop "$SERVICE" || {
22      echo "$(date +%Y-%m-%d %H:%M:%S)_ERROR_al_detener_$SERVICE" >> "${ERRORES}/log_global.
      log"
```



```

23     exit 1
24 }
25
26 # Iterar por cada subcarpeta
27 for PREFIJO_DIR in "$MODIFICADOS"/*; do
28     PREFIJO=$(basename "$PREFIJO_DIR")
29     echo -e "\n====_Procesando_prefijo:_$PREFIJO_===="
30
31     # Rutas por prefijo
32     DIR_RESULTADOS="${RESULTADOS}/${PREFIJO}"
33     DIR_ERRORES="${ERRORES}/${PREFIJO}"
34     DIR_YAVISTOS="${YAVISTOS}/${PREFIJO}"
35     DIR_EXTRANOS="${EXTRANOS}/${PREFIJO}"
36
37     mkdir -p "$DIR_RESULTADOS" "$DIR_ERRORES" "$DIR_YAVISTOS" "$DIR_EXTRANOS"
38
39     # Iterar sobre archivos .py
40     for archivo in "$PREFIJO_DIR"/*.py; do
41         [[ -f "$archivo" ]] || continue
42         . admin-openrc
43         TIMESTAMP=$(date +%Y-%m-%d_%H:%M:%S)
44         NOMBRE_REGISTRO=$(basename "$archivo".py)_${TIMESTAMP}"
45
46         echo -e "\n---_Probando_archivo:_$archivo"
47
48         cp "$archivo" /usr/lib/python3/dist-packages/nova/compute/api.py || {
49             echo "$(date +%Y-%m-%d_%H:%M:%S)_ERROR_al_copiar_$archivo" >> "${DIR_ERRORES}/
50                 log_errores.log"
51             mv "$archivo" "$DIR_EXTRANOS/"
52             continue
53         }
54
55         systemctl restart "$SERVICE" || {
56             echo "$(date +%Y-%m-%d_%H:%M:%S)_ERROR_al_reiniciar_$SERVICE" >> "${DIR_ERRORES}/
57                 log_errores.log"
58             mv "$archivo" "$DIR_EXTRANOS/"
59             continue
60         }
61
62         sleep 15
63
64         echo "Creando_máquina_virtual_de_prueba_(timeout_5_minutos)"
65         if ! timeout 300 openstack server create \
66             --flavor 8162b9a9-cc4c-4f77-b6c0-90d8c291bdc9 \
67             --image 1bee5173-8416-4b7e-884d-e7b52c809ed3 \
68             CirrosPrueba; then
69             echo "$(date +%Y-%m-%d_%H:%M:%S)_ERROR:_creación_de_VM_falló_o_excedió_5_minutos_
70                 para_$archivo" >> "${DIR_ERRORES}/log_errores.log"
71             mv "$archivo" "$DIR_EXTRANOS/"
72             continue

```

```

69     fi
70
71     vmstat 1 240 > "${DIR_RESULTADOS}/${NOMBRE_REGISTRO}_vmstat.log" || {
72         echo "$(date +%Y-%m-%d %H:%M:%S)_ERROR_vmstat_$archivo" >> "${DIR_ERRORES}/
           logErrores.log"
73         mv "$archivo" "$DIR_EXTRANOS/"
74         continue
75     }
76
77     INSTANCIAS=$(openstack server list -f value -c ID)
78     if [[ -n "$INSTANCIAS" ]]; then
79         echo "$INSTANCIAS" | xargs -r openstack server delete || {
80             echo "$(date +%Y-%m-%d %H:%M:%S)_ERROR_al_eliminar_instancias" >> "${
           DIR_ERRORES}/logErrores.log"
81             mv "$archivo" "$DIR_EXTRANOS/"
82             continue
83         }
84     fi
85     sleep 5
86
87     mv "$archivo" "$DIR_YAVISTOS" || {
88         echo "$(date +%Y-%m-%d %H:%M:%S)_ERROR_al_mover_$archivo_a_yavistos" >> "${
           DIR_ERRORES}/logErrores.log"
89         mv "$archivo" "$DIR_EXTRANOS/"
90         continue
91     }
92
93     echo "_$archivo_procesado_exitosamente."
94 done
95 done
96
97 echo -e "\n_Todos_los_archivos_fueron_procesados."

```

Anexo I

Archivo para la limpieza de registros

El script implementado en Python tiene como objetivo limpiar los archivos de métricas generados mediante vmstat durante la fase experimental.

Su función principal es eliminar encabezados repetidos dentro de los archivos de registro, conservando únicamente el primer encabezado válido y manteniendo la estructura de los datos para su análisis posterior.

Consola I.1: script en python limpiar_vmstat.py

```
1 import os
2
3 def clean_vmstat_logs(folder_path):
4     # Carpeta para guardar los archivos limpiados
5     cleaned_folder = os.path.join("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/", "
        vmstat_cleaned_4m")
6     os.makedirs(cleaned_folder, exist_ok=True)
7
8     header_1 = "procs_-----memory-----_---swap--_-----io----_system--_-----cpu-----
        "
9
10    for filename in os.listdir(folder_path):
11        if filename.endswith("_vmstat.log"):
12            full_path = os.path.join(folder_path, filename)
13
14            with open(full_path, "r") as f:
15                lines = f.readlines()
16
17                cleaned_lines = []
18                i = 0
19                header_kept = False
20
21                while i < len(lines):
22                    line = lines[i].strip()
23
24                    if line == header_1:
25                        if not header_kept and i + 1 < len(lines):
```

```

26         # Mantener el encabezado y la línea siguiente solo la primera vez
27         cleaned_lines.append(lines[i].rstrip())
28         cleaned_lines.append(lines[i+1].rstrip())
29         header_kept = True
30         # Saltar este encabezado y la siguiente línea (lo mantuvimos o es repetido)
31         i += 2
32     else:
33         cleaned_lines.append(lines[i].rstrip())
34         i += 1
35
36     # Guardar archivo limpio
37     base_name = filename.replace("_vmstat.log", "")
38     cleaned_output = os.path.join(cleaned_folder, f"{base_name}_vmstat_cleaned.log")
39
40     with open(cleaned_output, "w") as f:
41         f.write("\n".join(cleaned_lines))
42
43     print(f"Procesado: {filename}")
44
45 # Ejemplo de uso:
46 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/MCEFL")
47 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/MIA")
48 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/MRS")
49 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/MS")
50 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/MSFC")
51 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/MVIV")
52 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/WIDS")
53 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/WIDSL")
54 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/WLEC")
55 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/WSUT")
56 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/WVAE")
57 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/WVAL")
58 clean_vmstat_logs("d:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/resultados4m/WVIV")

```

Anexo J

Archivo para segmentación por cardinalidad porcentual

El siguiente programa segmenta los archivos de métricas previamente limpiados, generando conjuntos con diferente cardinalidad porcentual.

Forma parte de la preparación de los datos previo al análisis topológico.

Consola J.1: script en python split_vmstat.py

```
1 import os
2 import glob
3
4 # Carpeta donde están todos los archivos .log
5 # Carpeta base donde están las carpetas de sufijos
6 INPUT_DIR = "D:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/vmstat_cleaned_4m/"
7 # Carpeta base para guardar resultados
8 OUTPUT_BASE = "D:/Users/amartinez/Documents/01-Maestria/TESIS/02-PRUEBA/separados_4m/"
9
10 # Lista de sufijos para filtrar archivos
11 SUFIJOS = ['MCEFL', 'MIA', 'MRS', 'MSC', 'MSFC', 'MVIV',
12            'WIDS', 'WIDSL', 'WLEC', 'WSUT', 'WVAE', 'WVAL', 'WVIV']
13
14 # Divisiones train/test (porcentaje)
15 SPLITS = [(90, 10), (80, 20), (70, 30), (60, 40), (50, 50), (100, 0)]
16
17 os.makedirs(OUTPUT_BASE, exist_ok=True)
18
19 # Buscar todos los archivos .log en la carpeta
20 todos_archivos = glob.glob(os.path.join(INPUT_DIR, "*.log"))
21
22 # Filtrar archivos que empiezan con alguno de los sufijos
23 archivos_filtrados = [f for f in todos_archivos if any(os.path.basename(f).startswith(suf) for
24                                                         suf in SUFIJOS)]
25
26 if not archivos_filtrados:
27     print("No se encontraron archivos con los sufijos indicados.")
```

```

27 else:
28     for archivo in archivos_filtrados:
29         try:
30             with open(archivo, "r", encoding="utf-8") as f:
31                 lineas = f.readlines()
32             except Exception as e:
33                 print(f"Error leyendo {archivo}: {e}")
34                 continue
35
36             if len(lineas) < 3:
37                 print(f"Archivo muy corto, se omite: {archivo}")
38                 continue
39
40             encabezados = lineas[:2]
41             datos = lineas[2:]
42             total_datos = len(datos)
43
44             nombre_base = os.path.splitext(os.path.basename(archivo))[0]
45
46             # Detectar sufijo usado para este archivo (primer sufijo coincidente)
47             sufijo_arch = next((suf for suf in SUFIJOS if nombre_base.startswith(suf)), None)
48             if sufijo_arch is None:
49                 print(f"No se pudo detectar sufijo para {archivo}, se omite")
50                 continue
51
52             for train_pct, test_pct in SPLITS:
53                 corte = int(total_datos * train_pct / 100)
54                 train_data = encabezados + datos[:corte]
55                 test_data = encabezados + datos[corte:]
56
57                 carpeta_out = os.path.join(OUTPUT_BASE, f"split_{train_pct}_{test_pct}", sufijo_arch)
58                 os.makedirs(carpeta_out, exist_ok=True)
59
60                 train_path = os.path.join(carpeta_out, f"{nombre_base}_train_{train_pct}.log")
61                 test_path = os.path.join(carpeta_out, f"{nombre_base}_test_{test_pct}.log")
62
63                 try:
64                     with open(train_path, "w", encoding="utf-8") as f:
65                         f.writelines(train_data)
66                     with open(test_path, "w", encoding="utf-8") as f:
67                         f.writelines(test_data)
68                 except Exception as e:
69                     print(f"Error guardando archivos para {archivo}: {e}")
70
71             print("Proceso completado.")

```

Anexo K

Archivo para la estandarización de los datos

Este Notebook de Python implementa la estandarización y almacenamiento de métricas vmstat previamente segmentadas.

Consola K.1: Celda 01 del Notebook de Python

```
1 import os
2 import glob
3 import pandas as pd # Asegúrate de tener pandas importado
4
5 def leer_todos_los_datos_vmstat(directorio, patron, prefijos=None):
6     """
7     Lee todos los archivos que coincidan con un patrón dentro de un directorio
8     y devuelve un diccionario con un DataFrame por archivo, filtrando por uno o varios prefijos.
9
10    Parámetros:
11    directorio(str): Ruta al directorio que contiene los archivos.
12    patron(str): Patrón de archivos a leer (ej. '*.txt', '*_vmstat_cleaned').
13    prefijos(str o list, opcional): Uno o varios prefijos que deben tener los archivos.
14
15    Retorna:
16    dict: Diccionario con nombre de archivo como clave y DataFrame como valor.
17    """
18    if isinstance(prefijos, str):
19        prefijos = [prefijos]
20
21    dataframes = {}
22    ruta_busqueda = os.path.join(directorio, patron)
23    archivos = glob.glob(ruta_busqueda)
24
25    for archivo in archivos:
26        nombre_archivo = os.path.basename(archivo)
27
28        # Si se especificaron prefijos, filtrar por ellos
```

```

29     if prefijos and not any(nombre_archivo.startswith(p) for p in prefijos):
30         continue
31
32     try:
33         with open(archivo, 'r') as f:
34             lines = f.readlines()
35
36             # Limpiar encabezado
37             header = lines[1].strip().split()
38             data = [line.strip().split() for line in lines[2:] if line.strip()]
39
40             # Crear y limpiar DataFrame
41             df = pd.DataFrame(data, columns=header).apply(pd.to_numeric)
42
43             dataframes[nombre_archivo] = df
44
45     except Exception as e:
46         print(f"Error al procesar {archivo}: {e}")
47
48     return dataframes

```

Consola K.2: Celda 02 del Notebook de Python

```

1 import pandas as pd
2 import numpy as np
3
4 def estandarizar_vmstat_dataframe_holistico(df_limpio):
5     """
6     Estandariza un DataFrame numérico utilizando un enfoque holístico.
7     En lugar de aplicar Z-score por columna, se calcula la media y desviación
8     estándar global del conjunto de datos numéricos y se aplica la fórmula:
9      $(X - \text{media\_global}) / \text{desviación\_global}$ .
10
11     Parámetros:
12     df_limpio (pd.DataFrame): DataFrame con los datos de vmstat ya limpios.
13
14     Retorna:
15     pd.DataFrame: DataFrame con los datos estandarizados de forma global.
16     """
17     df_estandarizado = df_limpio.copy()
18
19     # Seleccionar solo los valores numéricos
20     valores_numericos = df_estandarizado.select_dtypes(include=[np.number]).values.flatten()
21
22     # Calcular media y desviación estándar globales, ignorando NaN
23     media_global = np.nanmean(valores_numericos)
24     std_global = np.nanstd(valores_numericos)
25
26     # Si la desviación estándar es 0, retornar ceros (evitar división por cero)

```



```

27     if std_global == 0:
28         print("Advertencia: La desviación estándar global es 0. Se retorna DataFrame con ceros.")
29         return df_estandarizado.select_dtypes(include=[np.number]).applymap(lambda x: 0)
30
31     # Aplicar estandarización global solo a las columnas numéricas
32     for column in df_estandarizado.columns:
33         if pd.api.types.is_numeric_dtype(df_estandarizado[column]):
34             df_estandarizado[column] = (df_estandarizado[column] - media_global) / std_global
35         else:
36             print(f"Advertencia: La columna '{column}' no es numérica y no será estandarizada.")
37
38     return df_estandarizado

```

Consola K.3: Celda 03 del Notebook de Python

```

1  import pandas as pd
2
3  def limpiar_y_tipificar_vmstat_dataframe(df_raw):
4      """
5      Limpia y tipifica un DataFrame de vmstat, asignando nombres de columna correctos
6      y convirtiendo los datos a tipo numérico.
7
8      Parámetros:
9      df_raw (pd.DataFrame): DataFrame original con los datos de vmstat
10      (posiblemente con encabezados genéricos o incorrectos).
11
12      Retorna:
13      pd.DataFrame: DataFrame limpio con nombres de columna y tipos de datos correctos.
14      Listo para análisis o estandarización estadística.
15      """
16      column_names = ['r', 'b', 'swpd', 'free', 'buff', 'cache', 'si', 'so', 'bi', 'bo', 'in', 'cs',
17                      'us', 'sy', 'id', 'wa', 'st']
18
19      if len(df_raw.columns) == len(column_names):
20          df_raw.columns = column_names
21      else:
22          print(f"Advertencia: El número de columnas del DataFrame ({len(df_raw.columns)}) no coincide con el esperado ({len(column_names)}). Ajustando el DataFrame.")
23          df_raw = df_raw.iloc[:, :len(column_names)]
24          df_raw.columns = column_names
25
26      # Convertir todas las columnas a tipo numérico, manejando errores (convierte a NaN si falla)
27      df_limpio = df_raw.apply(pd.to_numeric, errors='coerce')
28
29      return df_limpio

```

Consola K.4: Celda 04 del Notebook de Python

```

1  prefijo0 = 'MCEFL'
2  prefijo1 = 'MIA'

```

```

3 prefijo2 = 'MPFC' #
4 prefijo3 = 'MRS'
5 prefijo4 = 'MSC'
6 prefijo5 = 'MSFC'
7 prefijo6 = 'MVIE' #
8 prefijo7 = 'MVIV'
9 prefijo8 = 'WIDS'
10 prefijo9 = 'WIDSL'
11 prefijo10 = 'WLEC'
12 prefijo11 = 'WSUT'
13 prefijo12 = 'WVAE'
14 prefijo13 = 'WVAL'
15 prefijo14 = 'WVIV'
16 prefijos=[prefijo0,prefijo1,prefijo3,prefijo4,prefijo5,prefijo7,prefijo8,prefijo9,prefijo10,
    prefijo11,prefijo12,prefijo13,prefijo14]

```

Consola K.5: Celda 05 del Notebook de Python

```

1 resultados_vmstat = leer_todos_los_datos_vmstat(directorio='d:/Users/amartinez/Documents/01-
    Maestria/TESIS/02-PRUEBA/separados_4m/split_100_0', patron='*.log', prefijos=prefijos)

```

Consola K.6: Celda 06 del Notebook de Python

```

1 import os
2 import pandas as pd
3
4 # Definir la carpeta de salida
5 output_directory = 'datosEstandarizados4m/split_100_0'
6
7 # Crear la carpeta de salida si no existe
8 os.makedirs(output_directory, exist_ok=True)
9 print(f"Carpeta_{output_directory}_asegurada.")
10
11 # 1. Leer los datos inicialmente (DataFrames 'crudos')
12 raw_dataframes = resultados_vmstat
13
14 # 2. Limpiar y tipificar los DataFrames
15 cleaned_dataframes = {}
16 for nombre_archivo, df_raw in raw_dataframes.items():
17     print(f"Limpiando_y_tipificando_{nombre_archivo}...")
18     df_cleaned = limpiar_y_tipificar_vmstat_dataframe(df_raw)
19     cleaned_dataframes[nombre_archivo] = df_cleaned
20
21 # 3. Estandarizar los DataFrames limpios
22 standardized_dataframes_holistico = {}
23 for nombre_archivo, df_cleaned in cleaned_dataframes.items():
24     print(f"Estandarizando_{holistico}_{nombre_archivo}_y_guardando...")
25     df_holistico = estandarizar_vmstat_dataframe_holistico(df_cleaned)
26     standardized_dataframes_holistico[nombre_archivo] = df_holistico
27

```

```
28 # Construir la ruta completa para el archivo de salida
29 # Puedes mantener el mismo nombre de archivo o añadir un sufijo
30 nombre_base, extension = os.path.splitext(nombre_archivo)
31 output_filename = f"{nombre_base}_estandarizado{extension}" # Ejemplo:
    server1_vmstat_estandarizado.log
32 output_filepath = os.path.join(output_directory, output_filename)
33
34 # Guardar el DataFrame estandarizado en un archivo CSV
35 # index=False evita que pandas escriba el índice del DataFrame como una columna
36 df_holistico.to_csv(output_filepath, index=False)
37 print(f"Guardado: {output_filepath}")
38
39 print("\n---Proceso completado---")
```

Anexo L

Archivo de uso de algoritmos de reducción de dimensionalidad PCA, UMAP, t-SNE

```
1 import os
2 import glob
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from sklearn.decomposition import PCA
7 from sklearn.manifold import TSNE
8 from sklearn.preprocessing import StandardScaler
9 import umap
10
11 # Ruta de la carpeta
12 # =====
13 ruta_carpeta = "datosEstandarizados3m/split_100/unidos_por_prefijo/" # <-- CAMBIAR ESTO
14
15 # Buscar todos los archivos .log
16 archivos = glob.glob(os.path.join(ruta_carpeta, "*.log"))
17
18 if len(archivos) == 0:
19     raise ValueError("No se encontraron archivos .log en la carpeta")
20
21 print(f"Se encontraron {len(archivos)} archivos")
22
23 # Cargar y combinar datos
24 # =====
25 lista_dfs = []
26
27 for archivo in archivos:
28     df = pd.read_csv(archivo)
29     df["archivo"] = os.path.basename(archivo) # etiqueta por archivo
```

```
30     lista_dfs.append(df)
31
32 data = pd.concat(lista_dfs, ignore_index=True)
33
34 print("Shape total del dataset:", data.shape)
35
36 # Separar etiquetas
37 labels = data["archivo"]
38 data_features = data.drop(columns=["archivo"])
39
40 # Escalado
41 # =====
42 scaler = StandardScaler()
43 #data_scaled = scaler.fit_transform(data_features)
44 data_scaled = data_features
45
46 # PCA
47 # =====
48 pca = PCA(n_components=2)
49 pca_result = pca.fit_transform(data_scaled)
50
51 # t-SNE
52 # =====
53 tsne = TSNE(n_components=2, perplexity=30, random_state=42)
54 tsne_result = tsne.fit_transform(data_scaled)
55
56 # UMAP
57 # =====
58 umap_model = umap.UMAP(n_components=2, random_state=42)
59 umap_result = umap_model.fit_transform(data_scaled)
```

Anexo M

Archivo para unión de archivos estandarizados

Este programa agrupa múltiples archivos pertenecientes al mismo tipo de ataque SFI en un único archivo.

Consola M.1: Programa en python unir.py

```
1 import os
2 import glob
3 from collections import defaultdict
4
5 # Carpeta donde están los archivos
6 CARPETA_ENTRADA = "D:/Users/amartinez/Documents/01-Maestria/TESIS/00-IMPLEMENTACIONES/02-CODIGO
   /01-Estandarizar/datosEstandarizados4m/split_90_10"
7 CARPETA_SALIDA = os.path.join(CARPETA_ENTRADA, "unidos_por_prefijo")
8 os.makedirs(CARPETA_SALIDA, exist_ok=True)
9
10 # Buscar todos los archivos .log
11 archivos = sorted(glob.glob(os.path.join(CARPETA_ENTRADA, "*10_estandarizado.log")))
12
13 # Agrupar por prefijo (antes del primer "_")
14 grupos = defaultdict(list)
15 for archivo in archivos:
16     nombre = os.path.basename(archivo)
17     prefijo = nombre.split("_")[0]
18     grupos[prefijo].append(archivo)
19
20 # Unir archivos por prefijo
21 for prefijo, lista_archivos in grupos.items():
22     encabezado = []
23     contenido = []
24
25     for i, archivo in enumerate(lista_archivos):
26         with open(archivo, "r", encoding="utf-8") as f:
27             lineas = f.readlines()
```

```
28         if i == 0:
29             encabezado = lineas[:2] # Primeras dos líneas del primer archivo
30             contenido = lineas[2:]
31         else:
32             contenido += lineas[2:] # Saltar encabezados
33
34     archivo_salida = os.path.join(CARPETA_SALIDA, f"{prefijo}_10.log")
35     with open(archivo_salida, "w", encoding="utf-8") as f_out:
36         f_out.writelines(encabezado + contenido)
37
38     print(f"Prefijo_{prefijo}_unido_en:{archivo_salida}")
```

Anexo N

Archivo para la construcción de grafo global mediante TDA

Este programa implementa la construcción de un grafo topológico mediante el algoritmo de Mapper utilizando KeplerMapper, aplicando un procedimiento previo de balanceo de datos entre experimentos de distinta duración (3 minutos y 4 minutos) para evitar sesgos.

Primero, se cuentan las muestras disponibles por prefijo en el conjunto 3m y se usan como límite para seleccionar aleatoriamente el mismo número de muestras por prefijo en el conjunto 4m, garantizando representatividad equivalente.

Posteriormente, se extraen las variables numéricas y se proyectan a un espacio de menor dimensión usando PCA con scikit-learn. Sobre esta proyección se aplica una cobertura utilizando 60 hipercubos y con 15 % de traslape, seguido de un algoritmo de clustering con DBSCAN, generando así, un grafo global que permite visualizar la segmentación de los datos por tipo de ataque SFI.

Finalmente, el grafo se visualiza en formato HTML coloreado por tipo de ataque SFI, permitiendo analizar la separación estructural entre estos, la formación de componentes conexas y la posible existencia de regiones entrelazadas desde la perspectiva topológica.

Consola N.1: Programa en python DiagramaTopologico.py

```
1
2 import os
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6
7 from gudhi.cover_complex import MapperComplex
8 import gudhi as gd
9 import networkx as nx
10
11 from kmapper import KeplerMapper, Cover
12 from sklearn.preprocessing import StandardScaler
13 from sklearn.decomposition import PCA
14 from sklearn.cluster import DBSCAN
15
16
```



```

17 # -----
18 # Funciones auxiliares
19 # -----
20
21 def detectar_prefijo(nombre_archivo, lista_prefijos):
22     for prefijo in lista_prefijos:
23         if nombre_archivo.startswith(prefijo):
24             return prefijo.rstrip("_")
25     return "Otro"
26
27 def contar_lineas_3m(directorio_3m, lista_prefijos):
28     """Cuenta el número de líneas en los archivos .log de 3m por prefijo"""
29     lineas_por_prefijo = {}
30     if not os.path.exists(directorio_3m):
31         print(f" Carpeta {directorio_3m} no encontrada.")
32         return lineas_por_prefijo
33
34     for prefijo in lista_prefijos:
35         archivos = [
36             f for f in os.listdir(directorio_3m)
37             if f.startswith(prefijo) and f.endswith(".log")
38         ]
39         total_lineas = 0
40         for archivo in archivos:
41             with open(os.path.join(directorio_3m, archivo), "r") as f:
42                 total_lineas += sum(1 for _ in f) - 1 # quitar encabezado
43             lineas_por_prefijo[prefijo] = total_lineas
44     return lineas_por_prefijo
45
46 def cargar_y_unir_archivos_por_prefijos_limitado_random(directorio, lista_prefijos, limites,
47     seed):
48     """Carga los archivos de 5m pero seleccionando aleatoriamente N líneas (según límite de 3m)"""
49
50     if not os.path.exists(directorio):
51         print(f" Carpeta {directorio} no encontrada.")
52         return pd.DataFrame()
53
54     archivos = os.listdir(directorio)
55     archivos_filtrados = [
56         f for f in archivos
57         if any(f.startswith(prefijo) for prefijo in lista_prefijos)
58         and f.endswith('.log')
59     ]
60     if not archivos_filtrados:
61         print(" No se encontraron archivos con los prefijos indicados.")
62         return pd.DataFrame()
63
64     dataframes = []
65     for archivo in archivos_filtrados:

```

```

64     path = os.path.join(directorio, archivo)
65     prefijo = detectar_prefijo(archivo, lista_prefijos)
66     n_lineas = limites.get(prefijo, None)
67
68     # cargar todo el archivo
69     df_completo = pd.read_csv(path)
70     if n_lineas and len(df_completo) > n_lineas:
71         df = df_completo.sample(n=n_lineas, random_state=seed) # semilla diferente por
            iteraci>
72     else:
73         df = df_completo
74
75     df["archivo_origen"] = archivo
76     df["prefijo_origen"] = prefijo
77     dataframes.append(df)
78     return pd.concat(dataframes, ignore_index=True)
79
80 # -----
81 # Script principal
82 # -----
83 if __name__ == "__main__":
84     # Define prefijos
85     prefijos_todos = [
86         'MCEFL', # 0
87         'MIA', # 1
88         'MPFC', # 2 (excluir)
89         'MRS', # 3
90         'MSC', # 4
91         'MSFC', # 5
92         'MVIE', # 6 (excluir)
93         'MVIV', # 7
94         'WIDS', # 8
95         'WIDSL', # 9
96         'WLEC', # 10
97         'WSUT', # 11
98         'WVAE', # 12
99         'WVAL', # 13
100        'WVIV' # 14
101    ]
102
103
104     # Excluir prefijos 2 y 6
105     prefijos_a_usar = [p for i, p in enumerate(prefijos_todos) if i not in (2, 6)]
106
107     # Rutas a carpetas con logs (ajusta las rutas a tu entorno)
108     ruta_3m = "3m/datosEstandarizados3m_50"
109     ruta_5m = "4m/datosEstandarizados4m_50"
110
111     # Paso 1: Obtener límites por prefijo desde 3m

```

```
112     limites_por_prefijo = contar_lineas_3m(ruta_3m, prefijos_a_usar)
113
114     # Paso 2: Cargar datos de 5m con los mismos límites por prefijo
115     df_5m = cargar_y_unir_archivos_por_prefijos_limitado_random(
116         directorio=ruta_5m,
117         lista_prefijos=prefijos_a_usar,
118         limites=limites_por_prefijo,
119         seed=42
120     )
121
122     if df_5m.empty:
123         print("No se pudo construir el grafo. El DataFrame está vacío.")
124     else:
125         # Paso 3: Preparar datos numéricos
126         X = df_5m.select_dtypes(include=[np.number]).dropna(axis=1)
127         X = StandardScaler().fit_transform(X)
128
129         # Paso 4: Crear Mapper y aplicar proyección (PCA)
130         mapper = KeplerMapper(verbose=1)
131         lens = mapper.fit_transform(X, projection=PCA(n_components=3))
132
133         # Paso 5: Crear el grafo topológico
134         graph = mapper.map(
135             lens,
136             X,
137             cover=Cover(n_cubes=60, perc_overlap=0.15),
138             clusterer=DBSCAN(eps=0.7, min_samples=200)
139         )
140
141         # Paso 6: Visualizar el grafo con detalles por prefijo
142         mapper.visualize(
143             graph,
144             path_html="grafo_mapper.html",
145             title="Grafo Topológico por Prefijo",
146             color_values=df_5m["prefijo_origen"].astype("category").cat.codes,
147             color_function_name="Prefijo",
148             custom_tooltips=df_5m["prefijo_origen"]
149         )
150
151         print("Grafo generado: grafo_mapper.html")
```

Anexo Ñ

Archivo para la generación de diagramas de persistencia mediante complejos de Vietori-Rips

Este programa realiza un análisis topológico completo por prefijo utilizando el algoritmo de Mapper basado en matriz de distancias, incorporando además cálculo de homología persistente por componente conexa.

Primero, equilibra la cantidad de muestras entre experimentos 3m y 5m contando las líneas disponibles en 3m y usando ese número como límite para muestrear aleatoriamente los datos del conjunto 5m.

Luego, para cada nivel porcentual (10 % a 100 %) y para cada prefijo seleccionado, construye una matriz de distancias con `pairwise_distances`, sobre la cual genera un cubierta.

A partir de esto, se extrae el grafo para que posteriormente, se identifiquen las componentes conexas del grafo, para aquellas con más de cuatro nodos, recuperando los índices de los puntos originales. Con estos puntos originales, construye complejos de Vietori-Rips para calcular su persistencia homológica hasta la dimensión 3.

Finalmente, guarda tanto el diagrama de persistencia, como los intervalos de nacimiento y muerte, permitiendo analizar la estructura topológica interna de cada componente conexa por tipo de ataque SFI.

Consola Ñ.1: Programa en python `CalculoTDA.py`

```
1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from gudhi.cover_complex import MapperComplex
5 import gudhi as gd
6 import pandas as pd
7 import networkx as nx
8 from sklearn.metrics import pairwise_distances
9
10
11 verbose = False
12
13 # -----
```

```

14 # Funciones auxiliares
15 # -----
16
17 def detectar_prefijo(nombre_archivo, lista_prefijos):
18     for prefijo in lista_prefijos:
19         if nombre_archivo.startswith(prefijo):
20             return prefijo.rstrip("_")
21     return "Otro"
22
23
24 def contar_lineas_3m(directorio_3m, lista_prefijos):
25     """Cuenta el número de líneas en los archivos .log de 3m por prefijo"""
26     lineas_por_prefijo = {}
27     if not os.path.exists(directorio_3m):
28         print(f"Carpeta {directorio_3m} no encontrada.")
29         return lineas_por_prefijo
30
31     for prefijo in lista_prefijos:
32         archivos = [
33             f for f in os.listdir(directorio_3m)
34             if f.startswith(prefijo) and f.endswith(".log")
35         ]
36         total_lineas = 0
37         for archivo in archivos:
38             with open(os.path.join(directorio_3m, archivo), "r") as f:
39                 total_lineas += sum(1 for _ in f) - 1 # quitar encabezado
40             lineas_por_prefijo[prefijo] = total_lineas
41     return lineas_por_prefijo
42
43
44 def cargar_y_unir_archivos_por_prefijos_limitado_random(directorio, lista_prefijos, limites,
45 seed):
46     """Carga los archivos de 5m pero seleccionando aleatoriamente N líneas (según límite de 3m)"""
47
48     if not os.path.exists(directorio):
49         print(f"Carpeta {directorio} no encontrada.")
50         return pd.DataFrame()
51
52     archivos = os.listdir(directorio)
53     archivos_filtrados = [
54         f for f in archivos
55         if any(f.startswith(prefijo) for prefijo in lista_prefijos)
56         and f.endswith('.log')
57     ]
58     if not archivos_filtrados:
59         print("No se encontraron archivos con los prefijos indicados.")
60         return pd.DataFrame()
61
62     dataframes = []

```

```

61     for archivo in archivos_filtrados:
62         path = os.path.join(directorio, archivo)
63         prefijo = detectar_prefijo(archivo, lista_prefijos)
64         n_lineas = limites.get(prefijo, None)
65
66         # cargar todo el archivo
67         df_completo = pd.read_csv(path)
68         if n_lineas and len(df_completo) > n_lineas:
69             df = df_completo.sample(n=n_lineas, random_state=seed)
70         else:
71             df = df_completo
72
73         df["archivo_origen"] = archivo
74         df["prefijo_origen"] = prefijo
75         dataframes.append(df)
76     return pd.concat(dataframes, ignore_index=True)
77
78
79 # -----
80 # Script principal
81 # -----
82 if __name__ == "__main__":
83     prefijos_todos = [
84         'MCEFL', # prefijo0
85         'MIA', # prefijo1
86         'MPFC', # prefijo2 (excluir)
87         'MRS', # prefijo3
88         'MSC', # prefijo4
89         'MSFC', # prefijo5
90         'MVIE', # prefijo6 (excluir)
91         'MVIV', # prefijo7
92         'WIDS', # prefijo8
93         'WIDSL', # prefijo9
94         'WLEC', # prefijo10
95         'WSUT', # prefijo11
96         'WVAE', # prefijo12
97         'WVAL', # prefijo13
98         'WVIV' # prefijo14
99     ]
100
101     # Prefijos a procesar (excluye varios)
102     prefijos_a_usar = [p for i, p in enumerate(prefijos_todos) if i not in
103                        (1,2,3,4,5,6,7,8,9,10,11,12,13,14)]
104     print(prefijos_a_usar)
105
106     # Repetir 4 veces
107     for iteracion in range(1, 2):
108         print(f"\n=====")
109         print(f"░░░░Iteración░{iteracion}")

```

```

109     print(f"=====")
110
111     # Procesar para 10, 20, ..., 100
112     for n in range(10, 100, 10):
113         print(f"\n_Nivel_{n}")
114
115         carpeta_3m = f"3m/datosEstandarizados3m_{n}/"
116         carpeta_5m = f"5m/datosEstandarizados5m_{n}/"
117         carpeta_resultados = f"resultados5m2{iteracion}_{n}/"
118         os.makedirs(carpeta_resultados, exist_ok=True)
119
120         # calcular límites de líneas por prefijo con base en 3m
121         limites_lineas = contar_lineas_3m(carpeta_3m, prefijos_a_usar)
122
123         for prefijo in prefijos_a_usar:
124             print(f"Procesando_{prefijo}_{prefijo}")
125
126             # Crear carpeta de salida para este prefijo
127             carpeta_prefijo = os.path.join(carpeta_resultados, prefijo)
128             os.makedirs(carpeta_prefijo, exist_ok=True)
129
130             # Cargar datos de 5m limitados al número de líneas de 3m (aleatoriamente, semilla
131                 depende de iteración)
132             df_union = cargar_y_unir_archivos_por_prefijos_limitado_random(
133                 carpeta_5m, [prefijo], limites_lineas, seed=iteracion
134             )
135             if df_union.empty:
136                 print(f"No hay datos para_{prefijo}_{prefijo}, se salta.")
137                 continue
138
139             # Datos numéricos
140             df_numerico = df_union.select_dtypes(include='number').dropna()
141
142             # Si el tamaño no es suficiente, saltar
143             N_param = 600
144             if len(df_numerico) < N_param:
145                 print(f"Prefijo_{prefijo}_{prefijo} tiene solo_{len(df_numerico)}_filas_(<_{N_param}), se salta.")
146                 continue
147
148             # Matriz de distancias
149             print("Calculo_matriz_de_distancia")
150             D = pairwise_distances(df_numerico.values.astype(np.float32))
151
152             # Cover complex
153             cover_complex = MapperComplex(
154                 input_type='distance_matrix', min_points_per_node=0,
155                 clustering=None, N=N_param, beta=0., C=10,

```

```

156         )
157
158     print("UUUUUUUCalculoUcoverUcomplex")
159     try:
160         _ = cover_complex.fit(D)
161     except ValueError as e:
162         print(f"UErrorUenUcover_complex.fitUparaU{prefijo}:U{e}.USeUsalta.")
163         continue
164
165     # Grafo PNG
166     G = cover_complex.get_networkx()
167     plt.figure()
168     nx.draw(
169         G,
170         pos=nx.kamada_kawai_layout(G),
171         node_color=[cover_complex.node_info_[v]["colors"][0] for v in G.nodes()]
172     )
173     plt.savefig(os.path.join(carpeta_prefijo, f"grafo_{prefijo}.png"))
174     plt.close()
175
176     # HTML
177     cover_complex.data = df_numerico.values
178     cover_complex.save_to_html(
179         file_name=os.path.join(carpeta_prefijo, f"mapper_{prefijo}"),
180         data_name=f"{prefijo}",
181         cover_name="uniform",
182         color_name="height"
183     )
184
185     # Persistencia por componente conexa (> 4 nodos)
186     componentes = list(nx.connected_components(G))
187     for i, comp in enumerate(componentes):
188         if len(comp) > 4:
189             all_point_indices = []
190             for node in comp:
191                 point_indices = cover_complex.node_info_[node]["indices"]
192                 all_point_indices.extend(point_indices)
193
194             all_point_indices = np.unique(all_point_indices)
195             component_points = df_numerico.values[all_point_indices]
196
197             if len(component_points) > 0:
198                 rips_complex = gd.RipsComplex(points=component_points, sparse=0.6)
199                 simplex_tree = rips_complex.create_simplex_tree(max_dimension=3)
200                 persistence = simplex_tree.persistence()
201
202                 # PNG persistencia
203                 fig, ax = plt.subplots()
204                 gd.plot_persistence_diagram(persistence, axes=ax)

```



```
205         ax.set_title(f"Persistencia_{prefijo}_{Componente_{i+1}}_{len(comp)}_{nodos}")
206     plt.savefig(os.path.join(carpeta_prefijo, f"persistencia_{prefijo}_{componente_{i+1}}.svg"),bbox_inches="tight")
207     plt.close(fig)
208
209     # CSV persistencia
210     df_persistence = pd.DataFrame(
211         [(dim, birth, death) for dim, (birth, death) in persistence],
212         columns=["dimension", "birth", "death"]
213     )
214     df_persistence.to_csv(
215         os.path.join(carpeta_prefijo, f"persistencia_{prefijo}_{componente_{i+1}}.csv"),
216         index=False
217     )
218
219     print(f"_{prefijo}_{comp_{i+1}}:{len(comp)}_{nodos}_{len(component_points)}_{puntos}_{Guardado}")
```

Anexo O

Archivo para el cálculo de distancias de Wasserstein entre diagramas de persistencia

Este programa calcula las distancias de Wasserstein entre diagramas de persistencia almacenados en archivos CSV. El programa recorre una estructura de carpetas correspondiente a diferentes porcentajes de muestreo y carga los diagramas de persistencia asociados a cada conjunto de datos.

Posteriormente, se compara sistemáticamente los diagramas de persistencia entre distintos porcentajes de muestreo utilizando la función `wasserstein_distance` de la biblioteca `Gudhi`.

Finalmente, los resultados se almacenan en un archivo CSV que registra el tipo de ataque SFI del conjunto de datos, los porcentajes comparados, los nombres de los archivos analizados, la dimensión homológica correspondiente y el valor de la distancia calculada.

Consola O.1: Programa en python Comparacion.py

```
1 import os
2 import pandas as pd
3 import numpy as np
4 import csv
5 from gudhi.wasserstein import wasserstein_distance
6
7 # -----
8 # Parámetros globales
9 # -----
10 MAX_PUNTOS = 72000
11 BASE_DIR = "../5mt2/"
12 PORCENTAJES = [f"resultados5mt2_{i}" for i in range(10, 101, 10)]
13 OUTPUT_FILE = "distancias_wasserstein_5mt2.csv"
14
15 # -----
16 # Función para cargar un CSV como diagrama de persistencia
17 # -----
18 def cargar_diagrama(csv_path):
19     df = pd.read_csv(csv_path, dtype={
```

```

20     "dimension": np.int32,
21     "birth": np.float32,
22     "death": np.float32
23 })
24 diagramas = {}
25 for dim in df["dimension"].unique():
26     puntos = df[df["dimension"] == dim][["birth", "death"]].to_numpy(dtype=np.float32)
27     if len(puntos) > MAX_PUNTOS:
28         print(f"_{os.path.basename(csv_path)}_{dim}={dim}:_se_recortan_{len(puntos)}_-_{
29             MAX_PUNTOS}")
30         puntos = puntos[:MAX_PUNTOS]
31     diagramas[dim] = puntos
32     return diagramas
33 # -----
34 # Función para calcular y escribir distancias por dimensión
35 # -----
36 def calcular_y_guardar(diag_a, diag_b, prefijo, carpeta_a, archivo_a, carpeta_b, archivo_b,
37     writer):
38     dims = sorted(set(diag_a.keys()) | set(diag_b.keys()))
39     for dim in dims:
40         if dim in diag_a and dim in diag_b and len(diag_a[dim]) > 0 and len(diag_b[dim]) > 0:
41             dist = wasserstein_distance(diag_a[dim], diag_b[dim], order=2., internal_p=2.)
42             writer.writerow({
43                 "prefijo": prefijo,
44                 "porcentaje_a": carpeta_a.split("_")[-1],
45                 "archivo_a": archivo_a,
46                 "porcentaje_b": carpeta_b.split("_")[-1],
47                 "archivo_b": archivo_b,
48                 "dimension": dim,
49                 "distancia": np.float32(dist)
50             })
51             print(f"Distancia_escrita:_{prefijo}_{carpeta_a}-{archivo_a}_vs_{carpeta_b}-{
52                 archivo_b}_{dim}={dim}_{dist}={dist:.6f}")
53 # -----
54 # Ejecución principal
55 # -----
56 with open(OUTPUT_FILE, mode="w", newline="") as f_out:
57     writer = csv.DictWriter(f_out, fieldnames=[
58         "prefijo", "porcentaje_a", "archivo_a",
59         "porcentaje_b", "archivo_b", "dimension", "distancia"
60     ])
61     writer.writeheader()
62     print("Iniciando_proceso_global_de_cálculo_Wasserstein...\n")
63
64     for i, carpeta_a in enumerate(PORCENTAJES):
65         ruta_a = os.path.join(BASE_DIR, carpeta_a)

```

```

66     if not os.path.isdir(ruta_a):
67         print(f"Carpeta no encontrada: {carpeta_a}, se omite.")
68         continue
69
70     print(f"\nProcesando carpeta A: {carpeta_a}")
71
72     for prefijo in os.listdir(ruta_a):
73         ruta_prefijo_a = os.path.join(ruta_a, prefijo)
74         if not os.path.isdir(ruta_prefijo_a):
75             continue
76
77         print(f"Prefijo: {prefijo}")
78
79         archivos_a = [f for f in os.listdir(ruta_prefijo_a) if f.endswith(".csv")]
80         for archivo_a in archivos_a:
81             print(f"Archivo A: {archivo_a}")
82             ruta_archivo_a = os.path.join(ruta_prefijo_a, archivo_a)
83             diag_a = cargar_diagrama(ruta_archivo_a)
84
85             for carpeta_b in PORCENTAJES[i:]:
86                 ruta_prefijo_b = os.path.join(BASE_DIR, carpeta_b, prefijo)
87                 if not os.path.isdir(ruta_prefijo_b):
88                     continue
89
90                 print(f"Comparando con carpeta B: {carpeta_b}")
91                 archivos_b = [f for f in os.listdir(ruta_prefijo_b) if f.endswith(".csv")]
92
93                 for archivo_b in archivos_b:
94                     print(f"Archivo B: {archivo_b}")
95                     ruta_archivo_b = os.path.join(ruta_prefijo_b, archivo_b)
96                     diag_b = cargar_diagrama(ruta_archivo_b)
97
98                     calcular_y_guardar(
99                         diag_a, diag_b,
100                         prefijo, carpeta_a, archivo_a, carpeta_b, archivo_b,
101                         writer
102                     )
103
104 print(f"\nProceso terminado. Resultados guardados en {OUTPUT_FILE}")

```

Anexo P

Programa para la generación de gráficas comparativas de distancias de Wasserstein

Este programa realiza el análisis y visualización de las distancias de Wasserstein previamente calculadas entre diagramas de persistencia. El programa carga múltiples archivos CSV de los distintos experimentos.

Se generan dos tipos de visualizaciones: gráficas de tendencia que muestran la variación de las distancias promedio respecto al porcentaje de muestreo, y diagramas de radar que permiten comparar de manera global el comportamiento de cada tipo de ataque SFI.

Consola P.1: Programa en Jupyter Notebook TendenciasPorAtaque.ipynb

```
1 import os
2 import re
3 import pandas as pd
4 import seaborn as sns
5 import matplotlib.pyplot as plt
6 from matplotlib.backends.backend_pdf import PdfPages
7
8 # Funciones auxiliares
9 def extraer_num_carpeta(val):
10     if pd.isna(val):
11         return np.nan
12     val_str = str(val)
13     m = re.search(r'_(\d+)$', val_str)
14     if m:
15         return int(m.group(1))
16     elif val_str.isdigit():
17         return int(val_str)
18     else:
19         return np.nan
20
21 def extraer_num_componente(val):
```

```

22     if pd.isna(val):
23         return None
24     m = re.search(r'componente_(\d+)', str(val))
25     return int(m.group(1)) if m else None
26
27 # Función principal
28 def plot_prefijos_promedio(df, carpeta_ref=100, out_pdf="comparacion_prefijos_final.pdf"):
29     import numpy as np
30     import matplotlib.cm as cm
31     from math import pi
32
33     df = df.copy()
34     df["num_carpeta_a"] = df["carpeta_a"].apply(extraer_num_carpeta)
35     df["num_carpeta_b"] = df["carpeta_b"].apply(extraer_num_carpeta)
36     df["comp_a"] = df["archivo_a"].apply(extraer_num_componente)
37     df["comp_b"] = df["archivo_b"].apply(extraer_num_componente)
38     df["dimension"] = pd.to_numeric(df["dimension"], errors="coerce")
39     df["distancia"] = pd.to_numeric(df["distancia"], errors="coerce")
40     df = df.dropna(subset=["num_carpeta_a", "num_carpeta_b", "comp_a", "comp_b", "dimension", "distancia", "prefijo"])
41
42     df_bref = df[df["num_carpeta_b"] == carpeta_ref]
43     if df_bref.empty:
44         print("No hay datos para la carpeta de referencia indicada.")
45         return
46
47     with PdfPages(out_pdf) as pdf:
48         for dim, df_dim in df_bref.groupby("dimension"):
49             df_mean = df_dim.groupby(["prefijo", "num_carpeta_a"])["distancia"].mean().reset_index()
50             df_wide = df_mean.pivot(index="num_carpeta_a", columns="prefijo", values="distancia").sort_index()
51
52             plt.figure(figsize=(10,6))
53             num_prefijos = len(df_wide.columns)
54             colors = cm.get_cmap('tab20', max(num_prefijos, 13))
55             for i, col in enumerate(df_wide.columns):
56                 plt.plot(df_wide.index, df_wide[col], marker='o', label=col, color=colors(i))
57             plt.xlabel("Porcentaje de la componente")
58             plt.ylabel("Distancia promedio (todas las componentes)")
59             plt.title(f"Tendencias promedio por prefijo | dim={dim} | B={carpeta_ref}")
60             plt.legend()
61             plt.grid(True, linestyle="--", alpha=0.5)
62             plt.tight_layout()
63             pdf.savefig()
64             plt.close()
65
66             df_avg = df_mean.groupby("prefijo")["distancia"].mean().reset_index()
67             labels = df_avg["prefijo"].tolist()

```

```

68     values = df_avg["distancia"].tolist()
69
70     values += values[:1]
71     angles = [n / float(len(labels)) * 2 * pi for n in range(len(labels))]
72     angles += angles[:1]
73
74     plt.figure(figsize=(7,7))
75     ax = plt.subplot(111, polar=True)
76     plt.xticks(angles[:-1], labels, color='gray', size=9)
77     ax.plot(angles, values, linewidth=2, linestyle='solid', color='teal')
78     ax.fill(angles, values, color='teal', alpha=0.25)
79     plt.title(f"Radarm_de_distancia_promedio_por_prefijo_|_dim={dim}")
80     pdf.savefig()
81     plt.close()
82
83     print(f"Comparación_por_prefijo_guardada_en_{out_pdf}")
84
85     # Ejecucion
86 if __name__ == "__main__":
87     archivos = [
88         ## AGREGAR LOS ARCHIVOS DE LOS EXPERIMENTOS
89         "distancias_wasserstein_4m1.csv",
90         "distancias_wasserstein_4m2.csv",
91         "distancias_wasserstein_5m.csv",
92         "distancias_wasserstein_10m.csv",
93         "distancias_wasserstein_10m1.csv",
94         "distancias_wasserstein_10m2.csv",
95         #"distancias_wasserstein_10m3.csv",
96         "distancias_wasserstein_10m4.csv",
97         "distancias_wasserstein_10m5.csv",
98         "distancias_wasserstein_10m6.csv",
99         "distancias_wasserstein_10m7.csv",
100        #"distancias_wasserstein_10mt1.csv",
101        #"distancias_wasserstein_10mt2.csv",
102        #"distancias_wasserstein_10mt3.csv"
103    ]
104
105    dfs = []
106    for f in archivos:
107        df_temp = pd.read_csv(f)
108        dfs.append(df_temp)
109
110    df_all = pd.concat(dfs, ignore_index=True)
111    plot_prefijos_promedio(df_all)

```

Anexo Q

Programa para la construcción de una matriz global de distancias entre tipos de ataque

Este programa construye una matriz comparativa entre los tipos de ataque SFI, integrando múltiples archivos CSV generados en los diferentes experimentos con las distancias de Wasserstein.

Para cada archivo se calcula una matriz de distancias promedio entre todos los pares de tipos de ataque SFI presentes en los datos. Posteriormente, las matrices obtenidas se combinan para generar una matriz global promedio que resume la relación entre prefijos a lo largo de todos los experimentos considerados.

Finalmente, se guarda la matriz resultante en un archivo CSV y genera una visualización mediante un mapa de calor, lo que permite identificar de forma clara las similitudes y diferencias topológicas entre los distintos conjuntos de datos.

Consola Q.1: Programa en Jupyter Notebook MatrizDistancias.ipynb

```
1 import os
2 import pandas as pd
3 import itertools
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8 # -----
9 # CONFIGURACIÓN
10 # -----
11 carpeta_csv = "."
12 archivos_csv = [
13     "distancias_wasserstein_4m1.csv",
14     "distancias_wasserstein_4m2.csv",
15     "distancias_wasserstein_5m.csv",
16     "distancias_wasserstein_10m.csv",
17     "distancias_wasserstein_10m1.csv",
18     "distancias_wasserstein_10m2.csv",
19     "distancias_wasserstein_10m4.csv",
```



```

20     "distancias_wasserstein_10m5.csv",
21     "distancias_wasserstein_10m6.csv",
22     "distancias_wasserstein_10m7.csv"
23 ]
24
25 salida_csv = "matriz_global_prefijos.csv"
26 salida_figura = "matriz_global_prefijos.png"
27
28 # -----
29 # FUNCIÓN AUXILIAR
30 # -----
31 def calcular_matriz_por_archivo(df):
32     """
33     Calcula la matriz de distancias promedio entre prefijos dentro de un DataFrame.
34     """
35     prefijos = sorted(df["prefijo"].unique())
36     matriz = pd.DataFrame(0, index=prefijos, columns=prefijos, dtype=float)
37     conteo = pd.DataFrame(0, index=prefijos, columns=prefijos, dtype=int)
38
39     for i, j in itertools.combinations_with_replacement(prefijos, 2):
40         subset = df[(df["prefijo"] == i) | (df["prefijo"] == j)]
41         if subset.empty:
42             continue
43         media = subset["distancia"].mean()
44         matriz.loc[i, j] += media
45         matriz.loc[j, i] += media
46         conteo.loc[i, j] += 1
47         conteo.loc[j, i] += 1
48
49     conteo = conteo.replace(0, np.nan)
50     matriz = matriz / conteo
51     return matriz.fillna(0)
52
53 # -----
54 # PROCESAMIENTO GLOBAL
55 # -----
56 matrices = []
57 for archivo in archivos_csv:
58     ruta = os.path.join(carpeta_csv, archivo)
59     if not os.path.exists(ruta):
60         print(f"No se encontró el archivo: {ruta}")
61         continue
62
63     print(f"Procesando: {ruta}")
64     df = pd.read_csv(ruta, dtype={"prefijo": str})
65     df["distancia"] = pd.to_numeric(df["distancia"], errors="coerce")
66     df = df.dropna(subset=["prefijo", "distancia"])
67
68     matriz_local = calcular_matriz_por_archivo(df)

```

```

69     matrices.append(matriz_local)
70
71 if not matrices:
72     raise ValueError("No se cargó ningún archivo o no hay datos válidos.")
73
74 prefijos_globales = sorted(set().union(*[m.index for m in matrices]))
75 matriz_global = pd.DataFrame(0, index=prefijos_globales, columns=prefijos_globales, dtype=float)
76 conteo_global = pd.DataFrame(0, index=prefijos_globales, columns=prefijos_globales, dtype=int)
77
78 for m in matrices:
79     matriz_global = matriz_global.add(m, fill_value=0)
80     conteo_global = conteo_global.add((m != 0).astype(int), fill_value=0)
81
82 conteo_global = conteo_global.replace(0, np.nan)
83 matriz_global = matriz_global / conteo_global
84 matriz_global = matriz_global.fillna(0)
85
86 # -----
87 # GUARDAR Y VISUALIZAR
88 # -----
89 matriz_global.to_csv(salida_csv)
90 print(f"Matriz_global guardada en: {salida_csv}")
91
92 plt.figure(figsize=(10, 8))
93 sns.heatmap(
94     matriz_global,
95     annot=True, fmt=".4f",
96     cmap="viridis", linewidths=0.5,
97     cbar_kws={'label': 'Distancia_promedio'}
98 )
99
100 plt.title("Matriz_global de distancias_promedio entre prefijos")
101 plt.xlabel("Prefijo_B")
102 plt.ylabel("Prefijo_A")
103 plt.tight_layout()
104 plt.savefig(salida_figura, dpi=300)
105 plt.show()
106
107 print(f"Figura guardada como: {salida_figura}")

```

Anexo R

Programa para la generación de un dendrograma jerárquico a partir de la matriz global de distancias

Este programa realiza un agrupamiento jerárquico utilizando la matriz global de distancias promedio obtenida previamente. A partir de esta matriz, el programa calcula un enlace jerárquico mediante el método de Ward, el cual permite identificar similitudes estructurales entre los distintos conjuntos de datos analizados.

Posteriormente, se genera un dendrograma que representa visualmente la estructura de agrupamiento entre los tipos de ataque SFI.

Consola R.1: Programa en Jupyter Notebook Dendrograma.ipynb

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.cluster.hierarchy import linkage, dendrogram
4
5 # Convertir matriz a NumPy
6 matriz_numerica = matriz_global.astype(float).fillna(matriz_global.max().max()).to_numpy()
7
8 # Calcular linkage jerárquico
9 link = linkage(matriz_numerica, method='ward')
10
11 # =====
12 # Definición de grupos
13 # =====
14
15 grupos_colores = {
16     "grupo_1": {"miembros": ["MCEFL", "MRS", "WLEC"], "color": "green"},
17     "grupo_2": {"miembros": ["MVIV", "WSUT", "WIDSL", "WIDS"], "color": "orange"},
18     "grupo_3": {"miembros": ["MSC", "WVAL", "MSFC", "WVIV"], "color": "crimson"},
19     "grupo_4": {"miembros": ["WSUT", "WIDSL", "WIDS"], "color": "darkorchid"},
20     "grupo_5": {"miembros": ["MSC", "WVAL"], "color": "sienna"},
21     "grupo_6": {"miembros": ["MIA"], "color": "blue"},
```

```

22 }
23
24 # =====
25 # Función para el color de las ramas
26 # =====
27
28 def asignar_color(linkage_matrix, etiquetas, grupos):
29     colores = {}
30     etiquetas = list(etiquetas)
31     n = len(etiquetas)
32     grupos_inv = {} # prefijo color
33     for g, datos in grupos.items():
34         for pref in datos["miembros"]:
35             grupos_inv[pref] = datos["color"]
36
37     # Determina color por fusión (rama)
38     for i, (a, b, _, _) in enumerate(linkage_matrix):
39         indices = set()
40         for x in [a, b]:
41             if x < n:
42                 indices.add(etiquetas[int(x)])
43             else:
44                 indices |= colores.get(x, {"miembros": set()})["miembros"]
45
46     # Qué colores aparecen en los miembros
47     colores_presentes = {grupos_inv.get(x) for x in indices if x in grupos_inv}
48
49     # Si todos los miembros comparten color asignarlo
50     if len(colores_presentes) == 1:
51         color_final = list(colores_presentes)[0]
52     else:
53         color_final = "royalblue"
54
55     colores[n + i] = {"color": color_final, "miembros": indices}
56
57     return {k: v["color"] for k, v in colores.items()}
58
59 # =====
60 # Ejecucion principal
61 # =====
62
63 etiquetas = matriz_global.index.tolist()
64 link_colors = asignar_color(link, etiquetas, grupos_colores)
65
66 plt.figure(figsize=(12, 6))
67 dendro = dendrogram(
68     link,
69     labels=etiquetas,
70     orientation='top',

```

P32ograma para la generación de un dendrograma jerárquico a partir de la matriz global de distancias

```
71     leaf_rotation=45,  
72     leaf_font_size=10,  
73     link_color_func=lambda k: link_colors.get(k, "gray")  
74 )  
75  
76 # Mostrar valores de distancia sobre ramas  
77 for i, d, c in zip(dendro['icoord'], dendro['dcoord'], dendro['color_list']):  
78     x = 0.5 * sum(i[1:3])  
79     y = d[1]  
80     plt.text(x, y + 0.02 * max(link[:, 2]), f"{y:.3f}", va='bottom', ha='center', fontsize=9,  
81             color=c)  
82  
81 plt.title("Dendrograma por ataque")  
82 plt.xlabel("Ataques")  
83 plt.ylabel("Distancia entre ataques")  
84 plt.grid(axis='y', linestyle='--', alpha=0.4)  
85 plt.tight_layout()  
86  
87  
88 # Guardar en formato vectorial (mejor calidad)  
89 ##plt.savefig("aciertos_vs_incorrectos.pdf", bbox_inches="tight")  
90 # Alternativa vectorial  
91 plt.savefig("Dendrograma.svg", bbox_inches="tight")  
92  
93 plt.show()
```

Anexo S

Programa para la clasificación de nuevos experimentos y generación de reportes comparativos

Este programa implementa un procedimiento completo para la clasificación de nuevos experimentos a partir de distancias de Wasserstein previamente calculadas entre diagramas de persistencia. El objetivo es determinar a qué tipo de ataque SFI se asemejan más los nuevos datos, utilizando perfiles derivados de experimentos anteriores que fueron utilizados como entrenamiento.

Primero se cargan múltiples archivos de las distancias de Wasserstein promedio para cada tipo de ataque de los experimentos utilizados como entrenamiento. Posteriormente, los nuevos experimentos se comparan con estos perfiles mediante el cálculo del error cuadrático medio (MSE), lo que permite cuantificar la similitud entre los datos analizados y los conjuntos de referencia o entrenamiento.

Asimismo realiza una clasificación global, donde cada nuevo experimento se compara con todos los tipos de ataque SFI de los experimentos de entrenamiento, y además una clasificación por grupos conforme al dendrograma.

Finalmente, el programa genera múltiples salidas para facilitar el análisis de resultados.

Consola S.1: Programa en Jupyter Notebook Dendrograma.ipynb

```
1 import os
2 import re
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 from matplotlib.backends.backend_pdf import PdfPages
8 from sklearn.metrics import mean_squared_error
9
10 # -----
11 # Grupos de prefijos
12 # -----
13 grupos_prefijos = {
14     "Grupo1": ["WIDSL", "WIDS", "WSUT"],
```

```

15     "Grupo2": ["MVIV", "WVAE"],
16     "Grupo3": ["MSFC", "WVIV"],
17     "Grupo4": ["MSC", "WVAL"],
18     "Grupo5": ["MCEFL", "MRS", "WLEC"],
19     "Grupo6": ["MIA"],
20 }
21
22 # -----
23 # Funciones auxiliares
24 # -----
25 def extraer_prefijo(nombre_archivo):
26     base = os.path.basename(nombre_archivo)
27     # ajustar regex según tus archivos (aquí: distancias_wasserstein_10m3_XXX.csv)
28     m = re.search(r"distancias_wasserstein_10m3_([A-Z0-9]+)\.csv", base)
29     return m.group(1) if m else None
30
31 def obtener_grupo(prefijo):
32     for grupo, lista in grupos_prefijos.items():
33         if prefijo in lista:
34             return grupo
35     return None
36
37 def extraer_num_carpeta(val):
38     if pd.isna(val):
39         return np.nan
40     val_str = str(val)
41     m = re.search(r'_(\d+)\$', val_str)
42     if m:
43         return int(m.group(1))
44     elif val_str.isdigit():
45         return int(val_str)
46     else:
47         return np.nan
48
49 def extraer_num_componente(val):
50     if pd.isna(val):
51         return np.nan
52     m = re.search(r'componente_(\d+)', str(val))
53     return int(m.group(1)) if m else np.nan
54
55 def preparar_dataframe_ref(df):
56     df = df.copy()
57     df["dimension"] = pd.to_numeric(df["dimension"], errors="coerce")
58     df["distancia"] = pd.to_numeric(df["distancia"], errors="coerce")
59     return df.dropna(subset=["dimension", "distancia"])
60
61 def preparar_dataframe_nuevo(df):
62     df = df.copy()
63     df["dimension"] = pd.to_numeric(df["dimension"], errors="coerce")

```

```

64 df["distancia"] = pd.to_numeric(df["distancia"], errors="coerce")
65 return df.dropna(subset=["dimension", "distancia"])
66
67 def calcular_promedios_ref(df):
68     # devuelve DataFrame con columnas: prefijo, dimension, distancia (promedio)
69     return df.groupby(["prefijo", "dimension"])["distancia"].mean().reset_index()
70
71 # -----
72 # Clasificadores
73 # -----
74 def clasificar_nuevo(df_nuevo, ref_profiles, nombre_archivo, grupo_prefijo):
75     """Clasificación por grupo (usa ref_profiles = DataFrame con columnas prefijo, dimension,
76         distancia)."""
77     resultados = []
78     nuevo_prom = df_nuevo.groupby("dimension")["distancia"].mean().to_dict()
79     ref_filtrado = ref_profiles[ref_profiles["prefijo"].isin(grupo_prefijo)]
80
81     for prefijo, df_pref in ref_filtrado.groupby("prefijo"):
82         ref_prom = df_pref.set_index("dimension")["distancia"].to_dict()
83         dims_comunes = sorted(set(nuevo_prom.keys()) & set(ref_prom.keys()))
84         if not dims_comunes:
85             continue
86
87         y_new = np.array([nuevo_prom[d] for d in dims_comunes])
88         y_ref = np.array([ref_prom[d] for d in dims_comunes])
89         mse = mean_squared_error(y_new, y_ref)
90         resultados.append({"archivo": nombre_archivo, "clasificado_como": prefijo, "mse": mse})
91
92     if resultados:
93         mejor = min(resultados, key=lambda x: x["mse"])
94         print(f"{nombre_archivo}: se parece más a '{mejor['clasificado_como']}' (MSE={mejor['mse']:.6f})")
95     else:
96         print(f"{nombre_archivo}: No hubo coincidencias válidas en su grupo.")
97
98     return pd.DataFrame(resultados)
99
100 def clasificacion_global(df_nuevo, ref_profiles, nombre_archivo):
101     """Clasificación global: compara contra todos los prefijos en ref_profiles."""
102     resultados = []
103     nuevo_prom = df_nuevo.groupby("dimension")["distancia"].mean().to_dict()
104
105     for prefijo, df_pref in ref_profiles.groupby("prefijo"):
106         ref_prom = df_pref.set_index("dimension")["distancia"].to_dict()
107         dims = sorted(set(nuevo_prom.keys()) & set(ref_prom.keys()))
108         if not dims:
109             continue
110

```



```

111     y_new = np.array([nuevo_prom[d] for d in dims])
112     y_ref = np.array([ref_prom[d] for d in dims])
113     mse = mean_squared_error(y_new, y_ref)
114     resultados.append({"archivo": nombre_archivo, "prefijo_global": prefijo, "mse_global":
        mse})
115
116     return pd.DataFrame(resultados)
117
118     # -----
119     # Gráficos comparativos: MSE y radar
120     # -----
121     def grafico_mse_comparativo(pdf, res1, res2):
122         """Bar chart comparing MSE (best per archivo) between res1 and res2."""
123         if res1 is None or res2 is None:
124             return
125
126         try:
127             m1 = res1.loc[res1.groupby("archivo")["mse"].idxmin()].copy()
128             m2 = res2.loc[res2.groupby("archivo")["mse"].idxmin()].copy()
129         except Exception:
130             return
131
132         comunes = sorted(set(m1["archivo"]) & set(m2["archivo"]))
133         if not comunes:
134             return
135
136         df_plot = pd.DataFrame({
137             "archivo": comunes,
138             "mse_f1": [m1[m1["archivo"] == a]["mse"].values[0] for a in comunes],
139             "mse_f2": [m2[m2["archivo"] == a]["mse"].values[0] for a in comunes],
140         })
141
142         df_plot = df_plot.sort_values("mse_f1")
143
144         fig, ax = plt.subplots(figsize=(12, 6))
145         width = 0.35
146         x = np.arange(len(df_plot))
147         ax.bar(x - width/2, df_plot["mse_f1"], width, label="Filtro_1", color="#4F81BD")
148         ax.bar(x + width/2, df_plot["mse_f2"], width, label="Filtro_2", color="#9C27B0")
149         ax.set_xticks(x)
150         ax.set_xticklabels(df_plot["archivo"], rotation=45, ha="right")
151         ax.set_ylabel("MSE")
152         ax.set_title("Comparación MSE Filtro_1 vs Filtro_2")
153         ax.legend()
154         ax.grid(True, linestyle="--", alpha=0.4)
155         plt.tight_layout()
156         pdf.savefig(fig)
157         plt.close(fig)
158

```

```

159 def radar_combinado_filtros(pdf, df1, df2):
160     """
161     Radar combinado usando promedios por origen en los dataframes df1 y df2.
162     df1/df2 deben contener columnas 'origen' y 'distancia'.
163     """
164     if df1 is None or df2 is None:
165         return
166
167     a1 = df1.groupby("origen")["distancia"].mean()
168     a2 = df2.groupby("origen")["distancia"].mean()
169
170     comunas = sorted(set(a1.index) & set(a2.index))
171     if not comunas:
172         return
173
174     v1 = a1[comunas].values.tolist()
175     v2 = a2[comunas].values.tolist()
176     v1 += v1[:1]
177     v2 += v2[:1]
178
179     angles = [n / float(len(comunas)) * 2 * np.pi for n in range(len(comunas))]
180     angles += angles[:1]
181
182     fig = plt.figure(figsize=(8, 8))
183     ax = plt.subplot(111, polar=True)
184     ax.set_xticks(angles[:-1])
185     ax.set_xticklabels(comunas)
186
187     ax.plot(angles, v1, linewidth=2, linestyle='solid', color="#4F81BD", label="Filtro_1")
188     ax.fill(angles, v1, color="#4F81BD", alpha=0.25)
189
190     ax.plot(angles, v2, linewidth=2, linestyle='solid', color="#9C27B0", label="Filtro_2")
191     ax.fill(angles, v2, color="#9C27B0", alpha=0.25)
192
193     ax.set_title("Radar combinado Filtro_1 vs Filtro_2", fontsize=14, fontweight="bold")
194     ax.legend(loc="upper right", bbox_to_anchor=(1.2, 1.1))
195
196     pdf.savefig(fig)
197     plt.close(fig)
198
199
200 # -----
201 # Tabla comparativa Filtro1 vs Filtro2
202 # -----
203 def tabla_comparativa_mse(pdf, res1, res2):
204     if res1 is None or res2 is None or res1.empty or res2.empty:
205         return
206
207     # mejores por archivo

```

```

208 m1 = res1.loc[res1.groupby("archivo")["mse"].idxmin()].copy()
209 m2 = res2.loc[res2.groupby("archivo")["mse"].idxmin()].copy()
210
211 archivos_comunes = sorted(set(m1["archivo"]) & set(m2["archivo"]))
212 if not archivos_comunes:
213     return
214
215 filas = []
216 for a in archivos_comunes:
217     mse1 = m1[m1["archivo"] == a]["mse"].values[0]
218     mse2 = m2[m2["archivo"] == a]["mse"].values[0]
219
220     mejor_filtro = "Filtro_1" if mse1 < mse2 else "Filtro_2"
221     filas.append([a, f"{mse1:.6f}", f"{mse2:.6f}", mejor_filtro])
222
223 fig, ax = plt.subplots(figsize=(12, len(filas)*0.5 + 2))
224 ax.axis("off")
225 ax.set_title("Comparativa: ¿Qué filtro tiene menor MSE por archivo?",
226             fontsize=16, fontweight="bold", pad=20)
227
228 tabla_data = [["Archivo", "MSE_Filtro_1", "MSE_Filtro_2", "Mejor_filtro"]] + filas
229
230 tabla = ax.table(
231     cellText=tabla_data,
232     loc="center",
233     cellLoc="center",
234     colWidths=[0.45, 0.20, 0.20, 0.15]
235 )
236 tabla.auto_set_font_size(False)
237 tabla.set_fontsize(10)
238
239 for (r, c), cell in tabla.get_celld().items():
240     if r == 0:
241         cell.set_facecolor("#2E7D32")
242         cell.set_text_props(color="white", weight="bold")
243     else:
244         if tabla_data[r][3] == "Filtro_1":
245             cell.set_facecolor("#E8F5E9")
246         else:
247             cell.set_facecolor("#F3E5F5")
248
249 pdf.savefig(fig)
250 plt.close(fig)
251
252
253 # -----
254 # Función para generar PDF
255 # -----
256 def plot_tendencias_y_clasificacion_expandido(

```

```

257     df_ref_1,
258     resultados_finales_1,
259     df_ref_2=None,
260     resultados_finales_2=None,
261     carpeta_ref=100,
262     out_pdf="tendencias_y_clasificacion.pdf"
263 ):
264     """Genera PDF con:
265     """
266     """gráficos (primer y segundo filtro),
267     tablas estilizadas (primer y segundo filtro),
268     comparación MSE y radar combinado.
269     """
270
271     from math import pi
272
273     # Preparar dataframe (añade num_carpeta_a/b, comp_a/b y limpia)
274     def preparar(df):
275         df = df.copy()
276         df["num_carpeta_a"] = df["carpeta_a"].apply(extraer_num_carpeta)
277         df["num_carpeta_b"] = df["carpeta_b"].apply(extraer_num_carpeta)
278         df["comp_a"] = df["archivo_a"].apply(extraer_num_componente)
279         df["comp_b"] = df["archivo_b"].apply(extraer_num_componente)
280         df["dimension"] = pd.to_numeric(df["dimension"], errors="coerce")
281         df["distancia"] = pd.to_numeric(df["distancia"], errors="coerce")
282         return df.dropna(subset=["num_carpeta_a", "num_carpeta_b", "comp_a", "comp_b", "dimension", "distancia"])
283
284     # helper: dibuja línea + radar por prefijo/dim
285     def graficos_por_prefijo_dim(pdf, df, titulo_bloque=""):
286         for prefijo, df_pref in df.groupby("prefijo"):
287             for dim, df_dim in df_pref.groupby("dimension"):
288                 df_bref = df_dim[df_dim["num_carpeta_b"] == carpeta_ref]
289                 if df_bref.empty:
290                     continue
291
292                 df_mean = df_bref.groupby(["origen", "num_carpeta_a"]).agg(
293                     distancia_mean=("distancia", "mean"),
294                     n_componentes=("comp_a", "nunique")
295                 ).reset_index()
296
297                 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
298                 fig.suptitle(f"{titulo_bloque}_{prefijo}_{dim}_{carpeta_ref}",
299                             fontsize=14, fontweight="bold")
300
301                 sns.lineplot(x="num_carpeta_a", y="distancia_mean", hue="origen", data=df_mean,
302                             marker="o", ax=ax1)
303
304                 for _, row in df_mean.iterrows():
305                     ax1.text(row["num_carpeta_a"], row["distancia_mean"], str(row["n_componentes"]

```

```

    ]),
    ha="center", va="bottom", fontsize=8, color="black")
303
304 ax1.set_title("Tendencia_por_porcentaje")
305 ax1.set_xlabel("Porcentaje")
306 ax1.set_ylabel("Distancia_promedio")
307 ax1.grid(True, linestyle="--", alpha=0.5)
308 ax1.legend(title="Origen")
309
310 # radar
311 df_avg = df_mean.groupby("origen")["distancia_mean"].mean().reset_index()
312 if not df_avg.empty:
313     labels = df_avg["origen"].tolist()
314     values = df_avg["distancia_mean"].tolist()
315     values += values[:1]
316     angles = [n / float(len(labels)) * 2 * pi for n in range(len(labels))]
317     angles += angles[:1]
318
319     ax2 = plt.subplot(1, 2, 2, polar=True)
320     ax2.set_xticks(angles[:-1])
321     ax2.set_xticklabels(labels)
322     ax2.plot(angles, values, linewidth=2, linestyle='solid', color='teal')
323     ax2.fill(angles, values, color='teal', alpha=0.25)
324     ax2.set_title("Radar_de_distancia_promedio")
325
326 plt.tight_layout(rect=[0, 0, 1, 0.95])
327 pdf.savefig(fig)
328 plt.close(fig)
329
330 # helper: tabla estilizada (igual a tu estilo original)
331 def tabla_estilizada(pdf, df_resultados, titulo):
332     if df_resultados is None or df_resultados.empty:
333         return
334     # escoger la columna mse adecuada
335     mse_col = "mse" if "mse" in df_resultados.columns else "mse_global"
336     # mejores por archivo
337     mejores = df_resultados.loc[df_resultados.groupby("archivo")[mse_col].idxmin()].
        reset_index(drop=True)
338
339     fig, ax = plt.subplots(figsize=(10, len(mejores)*0.5 + 2))
340     ax.axis("off")
341     ax.set_title(titulo, fontsize=16, fontweight="bold", pad=20)
342
343     # construir tabla_data dependiendo columnas
344     if "clasificado_como" in mejores.columns:
345         tabla_data = [["Archivo", "Clasificado_como", "MSE"]] + [
346             [row["archivo"], row["clasificado_como"], f"{row['mse']:.6f}"] for _, row in
                mejores.iterrows()
347         ]
348     else:

```

```

349     tabla_data = [["Archivo", "Prefijo_Global", "MSE_Global"]] + [
350         [row["archivo"], row["prefijo_global"], f"{row['mse_global']:.6f}"] for _, row in
            mejores.iterrows()
351     ]
352
353     tabla = ax.table(cellText=tabla_data, loc="center", cellLoc="center", colWidths
        =[0.55,0.25,0.2])
354     tabla.auto_set_font_size(False)
355     tabla.set_fontsize(9)
356
357     for (r, c), cell in tabla.get_celld().items():
358         if r == 0:
359             cell.set_facecolor("#4F81BD")
360             cell.set_text_props(color="white", weight="bold")
361         else:
362             if r % 2 == 0:
363                 cell.set_facecolor("#F2F2F2")
364     pdf.savefig(fig)
365     plt.close(fig)
366
367     # tabla estilizada segunda referencia (morado)
368     def tabla_estilizada_ref2(pdf, df_resultados):
369         if df_resultados is None or df_resultados.empty:
370             return
371         mejores = df_resultados.loc[df_resultados.groupby("archivo")["mse"].idxmin()].reset_index
            (drop=True)
372
373         fig, ax = plt.subplots(figsize=(10, len(mejores)*0.5 + 2))
374         ax.axis("off")
375         ax.set_title("Resumen de clasificación SEGUNDO FILTRO", fontsize=16, fontweight="bold",
            pad=20)
376
377         tabla_data = [["Archivo", "Clasificado como", "MSE"]] + [
378             [row["archivo"], row["clasificado_como"], f"{row['mse']:.6f}"] for _, row in mejores.
                iterrows()
379         ]
380
381         tabla = ax.table(cellText=tabla_data, loc="center", cellLoc="center", colWidths
            =[0.55,0.25,0.2])
382         tabla.auto_set_font_size(False)
383         tabla.set_fontsize(9)
384
385         for (r, c), cell in tabla.get_celld().items():
386             if r == 0:
387                 cell.set_facecolor("#9C27B0")
388                 cell.set_text_props(color="white", weight="bold")
389             else:
390                 if r % 2 == 0:
391                     cell.set_facecolor("#F2E1F2")

```

```

392 pdf.savefig(fig)
393 plt.close(fig)
394
395 # -----
396 # EJECUCIÓN del PDF
397 # -----
398 with PdfPages(out_pdf) as pdf:
399
400     # PRIMER FILTRO: gráficos + tabla
401     if df_ref_1 is not None:
402         df1 = preparar(df_ref_1)
403         graficos_por_prefijo_dim(pdf, df1, titulo_bloque="Primer_filtro")
404         tabla_estilizada(pdf, resultados_finales_1, "Resumen_de_clasificación-PRIMER_FILTRO")
405     else:
406         print("_df_ref_1 es None: no se generan gráficos del primer filtro.")
407
408     # SEGUNDO FILTRO: gráficos + tabla (si existe)
409     df2 = None
410     if df_ref_2 is not None:
411         df2 = preparar(df_ref_2)
412         graficos_por_prefijo_dim(pdf, df2, titulo_bloque="Segundo_filtro")
413         tabla_estilizada_ref2(pdf, resultados_finales_2)
414     else:
415         print("_No hay segundo conjunto de referencia (df_ref_2).")
416
417     # -----
418     # Comparación MSE Filtro1 vs Filtro2
419     # -----
420     if resultados_finales_1 is not None and resultados_finales_2 is not None:
421         try:
422             grafico_mse_comparativo(pdf, resultados_finales_1, resultados_finales_2)
423         except Exception as e:
424             print("_Error generando MSE comparativo:", e)
425
426     # -----
427     # Radar combinado
428     # -----
429     if df_ref_1 is not None and df_ref_2 is not None:
430         try:
431             radar_combinado_filtros(pdf, df1, df2)
432         except Exception as e:
433             print("_Error generando radar combinado:", e)
434
435     # -----
436     # Tabla final
437     # -----
438     if resultados_finales_1 is not None and resultados_finales_2 is not None:
439         try:

```

```

440         tabla_comparativa_mse(pdf, resultados_finales_1, resultados_finales_2)
441     except Exception as e:
442         print("\uError\ugenerando\utabla\ucomparativa\ude\uMSE:", e)
443
444     print(f"PDF\uactualizado\ugenerado:\u{out_pdf}")
445
446
447
448     # -----
449     # Carga, clasificaci3n y ejecuci3n
450     # -----
451     if __name__ == "__main__":
452
453         # -----
454         # Conjunto de referencia principal (archivos_ref)
455         # -----
456         archivos_ref = [
457             "distancias_wasserstein_4m1.csv",
458             "distancias_wasserstein_4m2.csv",
459             "distancias_wasserstein_5m.csv",
460             "distancias_wasserstein_10m.csv",
461             "distancias_wasserstein_10m1.csv",
462             "distancias_wasserstein_10m2.csv",
463             "distancias_wasserstein_10m6.csv",
464             "distancias_wasserstein_10m4.csv",
465             "distancias_wasserstein_10m5.csv",
466             "distancias_wasserstein_10m7.csv",
467         ]
468
469         dfs = []
470         for f in archivos_ref:
471             if os.path.exists(f):
472                 df_temp = pd.read_csv(f)
473                 df_temp["origen"] = os.path.splitext(os.path.basename(f))[0]
474                 dfs.append(df_temp)
475             else:
476                 print(f"Archivo\ude\ureferencia\uno\uencontrado\u(primer\uset):\u{f}")
477
478         if not dfs:
479             raise SystemExit("No\use\u cargaron\uarchivos\ude\ureferencia\u(archivos_ref).")
480
481         df_all = pd.concat(dfs, ignore_index=True)
482         df_ref = preparar_dataframe_ref(df_all)
483         ref_profiles = calcular_promedios_ref(df_ref)
484
485         # -----
486         # Segundo conjunto de referencia (archivos_ref_2)
487         # -----
488         archivos_ref_2 = ["distancias_wasserstein_10mt4.csv",

```



```

489         "distancias_wasserstein_10mt5.csv",
490         "distancias_wasserstein_4mt3.csv",
491         "distancias_wasserstein_5mt1.csv",
492         "distancias_wasserstein_5mt2.csv",
493     ]
494     dfs2 = []
495     for f in archivos_ref_2:
496         if os.path.exists(f):
497             df_temp = pd.read_csv(f)
498             df_temp["origen"] = os.path.splitext(os.path.basename(f))[0]
499             dfs2.append(df_temp)
500         else:
501             print(f"␣Archivo␣de␣referencia␣no␣encontrado␣(segundo␣set):␣{f}")
502
503     df_all_2 = pd.concat(dfs2, ignore_index=True) if dfs2 else None
504     df_ref_2 = preparar_dataframe_ref(df_all_2) if df_all_2 is not None else None
505     ref_profiles_2 = calcular_promedios_ref(df_ref_2) if df_ref_2 is not None else None
506
507     # -----
508     # Nuevos archivos a clasificar
509     # -----
510     carpeta_nuevos = "nuevos_experimentos_10m3"
511     nuevos_csv = [f for f in os.listdir(carpeta_nuevos) if f.endswith(".csv")]
512
513     resultados_totales = []
514     resultados_globales = []
515     resultados_segunda_ref = []
516
517     for nuevo_csv in nuevos_csv:
518         prefijo = extraer_prefijo(nuevo_csv)
519         grupo = obtener_grupo(prefijo)
520
521         if not grupo:
522             print(f"␣{nuevo_csv}:␣␣prefijo␣'␣{prefijo}␣'␣no␣pertenece␣a␣ningún␣grupo.")
523             continue
524
525         ruta = os.path.join(carpeta_nuevos, nuevo_csv)
526         df_nuevo = preparar_dataframe_nuevo(pd.read_csv(ruta))
527         grupo_prefijo = grupos_prefijos[grupo]
528
529         # 1) Clasificación global (todos los prefijos)
530         df_global = clasificacion_global(df_nuevo, ref_profiles, nuevo_csv)
531         if not df_global.empty:
532             resultados_globales.append(df_global)
533
534         # 2) Clasificación por grupo (primer conjunto)
535         resultados = clasificar_nuevo(df_nuevo, ref_profiles, nuevo_csv, grupo_prefijo)
536         if not resultados.empty:
537             resultados_totales.append(resultados)

```

```

538
539     # 3) Clasificación por grupo usando segundo conjunto (si existe)
540     if ref_profiles_2 is not None:
541         resultados_2 = clasificar_nuevo(df_nuevo, ref_profiles_2, nuevo_csv + "_segunda_ref",
542                                         grupo_prefijo)
543         if not resultados_2.empty:
544             resultados_segunda_ref.append(resultados_2)
545
546     # -----
547     # Concatenar resultados y guardar CSV/EXCEL
548     # -----
549     df_final_global = pd.concat(resultados_globales, ignore_index=True) if resultados_globales
550     else None
551     df_final_grupos = pd.concat(resultados_totales, ignore_index=True) if resultados_totales
552     else None
553     df_final_ref2 = pd.concat(resultados_segunda_ref, ignore_index=True) if
554     resultados_segunda_ref else None
555
556     if df_final_grupos is not None:
557         df_final_grupos.to_csv("clasificaciones_grupos.csv", index=False)
558     if df_final_global is not None:
559         df_final_global.to_csv("clasificacion_global.csv", index=False)
560     if df_final_ref2 is not None:
561         df_final_ref2.to_csv("clasificaciones_segunda_ref.csv", index=False)
562
563     # Excel final con hojas separadas
564     with pd.ExcelWriter("clasificaciones_completas.xlsx") as writer:
565         if df_final_global is not None:
566             df_final_global.to_excel(writer, sheet_name="Global", index=False)
567         if df_final_grupos is not None:
568             df_final_grupos.to_excel(writer, sheet_name="Por_Grupo", index=False)
569         if df_final_ref2 is not None:
570             df_final_ref2.to_excel(writer, sheet_name="Segunda_Referencia", index=False)
571
572     print("\u25b6Excel\u25b6generado:\u25b6clasificaciones_completas.xlsx")
573
574     # -----
575     # Generar PDF
576     # -----
577     plot_tendencias_y_clasificacion_expandido(
578         df_ref_1=df_all,
579         resultados_finales_1=df_final_grupos,
580         df_ref_2=df_all_2 if dfs2 else None,
581         resultados_finales_2=df_final_ref2,
582         carpeta_ref=100,
583         out_pdf="tendencias_y_clasificacion.pdf"
584     )
585
586     # -----
587     # Gr\u00e1fico de Dispersi\u00f3n de MSE

```

```
583 # -----
584
585 output_folder = "./resultados_clasificacion/resultados_globales"
586 if df_final_grupos is not None and not df_final_grupos.empty:
587     plt.figure(figsize=(12,6))
588     sns.boxplot(x="clasificado_como", y="mse", data=df_final_grupos, palette="Set3")
589     plt.xticks(rotation=45)
590     plt.xlabel("Ataque_SFI")
591     plt.ylabel("MSE")
592     plt.title("Distribución del MSE por tipo de ataque_SFI")
593     plt.grid(axis="y", linestyle="--", alpha=0.5)
594     plt.tight_layout()
595     #plt.savefig("boxplot_mse_por_prefijo.png", dpi=300)
596     #plt.savefig("metricas_globales_comparadas_mse.svg", bbox_inches="tight")
597     plt.savefig(os.path.join(output_folder, "metricas_globales_comparadas_mse.svg"),
598                 bbox_inches="tight")
599     plt.show()
600 else:
601     print("No hay resultados para generar boxplots de MSE.")
```

Anexo T

Programa que aplica los criterios de clasificación final y muestra falsos positivos

Este programa implementa un procedimiento automatizado para analizar los resultados de clasificación obtenidos en el S a partir de dos filtros diferentes.

En primer lugar, el programa recorre una estructura de directorios que contiene los resultados de distintos experimentos. Para cada subcarpeta se cargan dos archivos CSV: uno correspondiente a las clasificaciones obtenidas mediante el segundo filtro y otro con las clasificaciones generadas a partir de agrupaciones de tipos de ataque SFI. A partir de estos datos se seleccionan para cada uno de ellos, los resultados con el menor valor de error cuadrático medio.

Posteriormente, se realiza una comparación entre ambos conjuntos de resultados, generando una tabla comparativa que incluye el archivo analizado, el tipo de ataque asignado por cada filtro y los valores de MSE correspondientes. Con esta información se determina cuál de los dos métodos presenta el mejor ajuste para cada caso.

Consola T.1: Programa en Jupyter Notebook Resultados.ipynb - Celda 01

```
1 import os
2 import pandas as pd
3 import re
4
5 from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle,
    PageBreak
6 from reportlab.lib import colors
7 from reportlab.lib.styles import getSampleStyleSheet
8 from reportlab.lib.pagesizes import landscape, letter
9
10 # =====
11 # Función para extraer prefijo desde nombre de archivo
12 # =====
13 def extraer_prefijo(archivo):
14     match = re.search(r'_(\w+)\.csv', archivo)
```

```

15     return match.group(1) if match else None
16
17
18 # =====
19 # Limpiar nombres de archivos
20 # =====
21 def limpiar_nombre_archivo(archivo):
22     archivo = re.sub(r"^distancias_wasserstein_\d+m\d*_", "", archivo)
23     archivo = re.sub(r"\.csv.*$", "", archivo)
24     return archivo
25
26
27 # =====
28 # Función para procesar una subcarpeta
29 # =====
30 def procesar_carpeta(path_carpeta):
31     archivo_ref2 = os.path.join(path_carpeta, "clasificaciones_segunda_ref.csv")
32     archivo_grupos = os.path.join(path_carpeta, "clasificaciones_grupos.csv")
33
34     if not os.path.exists(archivo_ref2) or not os.path.exists(archivo_grupos):
35         return None
36
37     df_ref2 = pd.read_csv(archivo_ref2)
38     df_grupos = pd.read_csv(archivo_grupos)
39
40     df_ref2["prefijo"] = df_ref2["archivo"].apply(extraer_prefijo)
41     df_grupos["prefijo"] = df_grupos["archivo"].apply(extraer_prefijo)
42
43     min_ref2 = df_ref2.loc[df_ref2.groupby("prefijo")["mse"].idxmin()]
44     min_grupos = df_grupos.loc[df_grupos.groupby("prefijo")["mse"].idxmin()]
45
46     comparativa = min_grupos.merge(
47         min_ref2,
48         on="prefijo",
49         suffixes=("_grupos", "_ref2")
50     )
51
52     comparativa["mejor"] = comparativa.apply(
53         lambda row: "grupos" if row["mse_grupos"] < row["mse_ref2"] else "ref2",
54         axis=1
55     )
56
57     comparativa["archivo_grupos"] = comparativa["archivo_grupos"].apply(limpiar_nombre_archivo)
58     comparativa["archivo_ref2"] = comparativa["archivo_ref2"].apply(limpiar_nombre_archivo)
59
60     return comparativa
61
62
63 # =====

```

```

64 # Generar PDF horizontal con estilos avanzados
65 # =====
66 def generar_pdf(df_por_carpeta, output="comparativas.pdf"):
67     styles = getSampleStyleSheet()
68     doc = SimpleDocTemplate(
69         output,
70         pagesize=landscape(letter),
71         leftMargin=20,
72         rightMargin=20,
73         topMargin=20,
74         bottomMargin=20
75     )
76
77     story = []
78
79     for carpeta, df in df_por_carpeta.items():
80
81         story.append(Paragraph(f"<b>Resultados comparando con experimento: {carpeta}</b>", styles
82                                ["Title"]))
83
84         story.append(Spacer(1, 12))
85
86         cols = [
87             "Archivo-Nuevo experimento",
88             "Primer Filtro",
89             "mse_1",
90             "Segundo Filtro",
91             "mse_2",
92         ]
93
94         data = [cols]
95
96         for row in df.itertuples():
97             data.append([
98                 row.archivo_grupos,
99                 row.clasificado_como_grupos,
100                 f"{row.mse_grupos:.6f}",
101                 row.clasificado_como_ref2,
102                 f"{row.mse_ref2:.6f}",
103             ])
104
105         colWidths = [doc.width / len(cols)] * len(cols)
106
107         table = Table(data, colWidths=colWidths)
108
109         style = TableStyle([
110             ("BACKGROUND", (0, 0), (-1, 0), colors.gray),
111             ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
112             ("GRID", (0, 0), (-1, -1), 0.5, colors.black),
113             ("ALIGN", (0, 0), (-1, -1), "CENTER"),

```

```

112     ("FONTSIZE", (0, 0), (-1, -1), 8),
113 ]))
114
115 # =====
116 # NUEVO: colorear solo el MSE correcto
117 # =====
118 for i, row in enumerate(df.itertuples(), start=1):
119
120     prefijo_1 = row.clasificado_como_grupos
121     prefijo_2 = row.clasificado_como_ref2
122
123     mse1 = row.mse_grupos
124     mse2 = row.mse_ref2
125
126     # REGLA 1: Si ambos prefijos coinciden usar MSE del segundo filtro
127     if prefijo_1 == prefijo_2:
128         style.add("BACKGROUND", (4, i), (4, i), colors.lightblue)
129
130     else:
131         # REGLA 2: Elegir el MSE más cercano a cero
132         if mse1 < mse2:
133             style.add("BACKGROUND", (2, i), (2, i), colors.lightblue)
134         else:
135             style.add("BACKGROUND", (4, i), (4, i), colors.lightblue)
136
137     table.setStyle(style)
138     story.append(table)
139     story.append(PageBreak())
140
141 doc.build(story)
142 print(f"PDF generado: {output}")
143
144 # =====
145 # PROGRAMA PRINCIPAL
146 # =====
147
148 ruta_base = "../03-Resultados"
149 df_por_carpeta = {}
150
151 for carpeta in os.listdir(ruta_base):
152     path_carpeta = os.path.join(ruta_base, carpeta)
153     if os.path.isdir(path_carpeta):
154         df = procesar_carpeta(path_carpeta)
155         if df is not None:
156             df_por_carpeta[carpeta] = df
157
158 generar_pdf(df_por_carpeta)

```

Cuando ambos filtros coinciden en la clasificación, se prioriza el resultado del segundo filtro; en caso

contrario, se selecciona el valor de MSE más cercano a cero.

Finalmente, se producen gráficos de pastel que muestran la proporción de clasificaciones correctas e incorrectas, o falsos positivos, tanto considerando únicamente el primer filtro como aplicando el criterio combinado de decisión entre ambos filtros.

Consola T.2: Programa en Jupyter Notebook Resultados.ipynb - Celda 02

```

1 import matplotlib.pyplot as plt
2
3 # =====
4 # Cálculo basado ÚNICAMENTE en el PRIMER FILTRO
5 # =====
6
7 aciertos_1 = 0
8 errores_1 = 0
9
10 for carpeta, df in df_por_carpeta.items():
11
12     for row in df.itertuples():
13
14         y_true = row.archivo_grupos
15         y_pred_1 = row.clasificado_como_grupos
16
17         if y_pred_1 == y_true:
18             aciertos_1 += 1
19         else:
20             errores_1 += 1
21
22
23 # =====
24 # GRÁFICA DE PASTEL - PRIMER FILTRO
25 # =====
26 plt.figure(figsize=(7,7))
27
28 plt.pie(
29     [aciertos_1, errores_1],
30     labels=["Acertados_1er_Filtro", "Incorrectos_1er_Filtro"],
31     autopct='%1.1f%%',
32     startangle=90
33 )
34
35 plt.title("Acertados_vs_Incorrecto_Solo_primer_filtro")
36 plt.axis('equal')
37
38 # Guardar en formato vectorial (mejor calidad)
39 ##plt.savefig("aciertos_vs_incorrectos.pdf", bbox_inches="tight")
40 # Alternativa vectorial
41 plt.savefig("ResultadoPrimerFiltro.svg", bbox_inches="tight")
42
43

```



```
44 plt.show()
```

Consola T.3: Programa en Jupyter Notebook Resultados.ipynb - Celda 03

```
1 import matplotlib.pyplot as plt
2
3 # =====
4 # Contadores globales
5 # =====
6 aciertos = 0
7 errores = 0
8
9 for carpeta, df in df_por_carpeta.items():
10     print(carpeta)
11
12     for row in df.itertuples():
13
14         prefijo_real = row.archivo_grupos
15         prefijo_1 = row.clasificado_como_grupos
16         prefijo_2 = row.clasificado_como_ref2
17
18         mse1 = row.mse_grupos
19         mse2 = row.mse_ref2
20
21         # =====
22         # APLICAR LAS REGLAS
23         # =====
24
25         # Regla 1: Ambos filtros clasifican igual
26         if prefijo_1 == prefijo_2:
27             prefijo_elegido = prefijo_2 # elegir el segundo filtro
28
29         else:
30             # Regla 2: elegir MSE más cercano a cero
31             if mse1 < mse2:
32                 prefijo_elegido = prefijo_1
33             else:
34                 prefijo_elegido = prefijo_2
35
36         # =====
37         # COMPARAR CONTRA PREFIJO REAL
38         # =====
39         if prefijo_elegido == prefijo_real:
40             aciertos += 1
41         else:
42             errores += 1
43
44
45 # =====
```

```
46 # GRÁFICA DE PASTEL
47 # =====
48 plt.figure(figsize=(7, 7))
49
50 plt.pie(
51     [aciertos, errores],
52     labels=["Acertados", "Incorrectos"],
53     autopct=' %1.1f%%',
54     startangle=90
55 )
56
57 plt.title("Acertados_vs_Incorrectos_Ambos_filtros_Reglas_de_seleccion")
58 plt.axis("equal")
59
60 # Guardar en formato vectorial (mejor calidad)
61 #plt.savefig("aciertos_vs_incorrectos.pdf", bbox_inches="tight")
62 # Alternativa vectorial
63 plt.savefig("aciertos_vs_incorrectos.svg", bbox_inches="tight")
64
65
66 plt.show()
```

Anexo U

**Muestra de resultados obtenidos del
Anexo T ante un nuevo experimento**

Resultados del nuevo experimento: 10m0

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	WLEC	0.000102	WLEC	0.000290
MIA	MIA	0.003043	MIA	0.001808
MRS	MRS	0.000060	MRS	0.000256
MSFC	MSFC	0.000071	MSFC	0.000059
MVIV	MVIV	0.000035	MVIV	0.000062
WIDS	WIDS	0.000008	WSUT	0.000018
WIDSL	WIDSL	0.000000	WIDSL	0.000000
WLEC	MCEFL	0.000067	MCEFL	0.000051
WSUT	WIDSL	0.000001	WIDSL	0.000002
WVAE	WVAE	0.000012	WVAE	0.000021
WVAL	WVAL	0.000048	WVAL	0.000036
WVIV	WVIV	0.000002	WVIV	0.000001

Resultados del nuevo experimento: 10m1

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	MCEFL	0.000308	MCEFL	0.000006
MIA	MIA	0.001911	MIA	0.001050
MRS	MRS	0.000065	MRS	0.000267
MSFC	MSFC	0.000124	MSFC	0.000098
MVIV	MVIV	0.000011	MVIV	0.000029
WIDS	WIDS	0.000003	WIDS	0.000010
WIDSL	WIDSL	0.000004	WIDSL	0.000006
WLEC	WLEC	0.000041	WLEC	0.000150
WSUT	WSUT	0.000000	WSUT	0.000004
WVAE	WVAE	0.000006	WVAE	0.000012
WVAL	WVAL	0.000039	WVAL	0.000027
WVIV	WVIV	0.000021	WVIV	0.000005

Resultados del nuevo experimento: 10m2

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	WLEC	0.000398	WLEC	0.000682
MIA	MIA	0.005554	MIA	0.003756
MRS	MRS	0.000069	MRS	0.000275
MSC	MSC	0.000033	MSC	0.000002
MSFC	MSFC	0.000096	MSFC	0.000076
MVIV	MVIV	0.000020	MVIV	0.000040
WIDS	WIDS	0.000000	WIDS	0.000004
WIDSL	WIDSL	0.000000	WIDSL	0.000001
WLEC	MCEFL	0.000018	WLEC	0.000080
WSUT	WSUT	0.000001	WSUT	0.000005
WVAE	WVAE	0.000025	MVIV	0.000031
WVAL	MSC	0.000017	MSC	0.000066
WVIV	WVIV	0.000003	WVIV	0.000001

Resultados del nuevo experimento: 10m4

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	MCEFL	0.000014	WLEC	0.000086
MIA	MIA	0.004484	MIA	0.002640
MRS	MRS	0.000066	MRS	0.000268
MSC	MSC	0.000047	MSC	0.000000
MSFC	MSFC	0.000222	MSFC	0.000176
MVIV	MVIV	0.000010	MVIV	0.000027
WIDS	WIDS	0.000000	WIDS	0.000002
WIDSL	WIDSL	0.000001	WIDSL	0.000000
WLEC	MRS	0.000219	MRS	0.000046
WSUT	WSUT	0.000000	WSUT	0.000001
WVAE	WVAE	0.000021	WVAE	0.000031
WVAL	WVAL	0.000038	WVAL	0.000027
WVIV	WVIV	0.000002	WVIV	0.000001

Resultados del nuevo experimento: 10m5

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	MRS	0.000283	MCEFL	0.000024
MIA	MIA	0.003238	MIA	0.001925
MRS	MRS	0.000037	MRS	0.000211
MSFC	MSFC	0.000399	MSFC	0.000317
MVIV	MVIV	0.000026	MVIV	0.000049
WIDS	WIDS	0.000005	WIDS	0.000014
WIDSL	WIDSL	0.000000	WIDSL	0.000000
WLEC	WLEC	0.000181	WLEC	0.000316
WSUT	WSUT	0.000000	WSUT	0.000001
WVAE	WVAE	0.000012	WVAE	0.000021
WVAL	MSC	0.000063	WVAL	0.000085
WVIV	WVIV	0.000006	WVIV	0.000003

Resultados del nuevo experimento: 10m6

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	MCEFL	0.000094	MCEFL	0.000035
MIA	MIA	0.001663	MIA	0.000914
MRS	MRS	0.000042	MRS	0.000223
MSFC	MSFC	0.000154	MSFC	0.000122
MVIV	MVIV	0.000006	MVIV	0.000022
WIDS	WIDS	0.000005	WIDS	0.000014
WIDSL	WIDSL	0.000001	WIDSL	0.000000
WLEC	WLEC	0.000041	WLEC	0.000151
WSUT	WSUT	0.000003	WIDSL	0.000008
WVAE	MVIV	0.000016	MVIV	0.000004
WVAL	WVAL	0.000043	WVAL	0.000031
WVIV	WVIV	0.000025	WVIV	0.000006

Resultados del nuevo experimento: 10m7

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	MCEFL	0.000014	WLEC	0.000078
MIA	MIA	0.004213	MIA	0.002469
MRS	MRS	0.000065	MRS	0.000266
MSC	MSC	0.000047	MSC	0.000000
MSFC	MSFC	0.000215	MSFC	0.000171
MVIV	MVIV	0.000010	MVIV	0.000027
WIDS	WIDS	0.000000	WIDS	0.000002
WIDSL	WIDSL	0.000000	WIDSL	0.000000
WLEC	MRS	0.000263	MCEFL	0.000043
WSUT	WSUT	0.000000	WSUT	0.000002
WVAE	WVAE	0.000021	WVAE	0.000031
WVAL	WVAL	0.000036	WVAL	0.000026
WVIV	WVIV	0.000003	WVIV	0.000000

Resultados del nuevo experimento: 4m1

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	WLEC	0.000173	WLEC	0.000298
MIA	MIA	0.026611	MIA	0.022272
MRS	MRS	0.000067	MRS	0.000009
MSFC	WVIV	0.000579	WVIV	0.000529
MVIV	WVAE	0.000119	WVAE	0.000077
WIDS	WSUT	0.000014	WSUT	0.000008
WIDSL	WIDSL	0.000002	WIDSL	0.000002
WLEC	WLEC	0.000495	WLEC	0.000551
WSUT	WSUT	0.000003	WSUT	0.000000
WVAE	WVAE	0.000019	WVAE	0.000005
WVAL	WVAL	0.000028	WVAL	0.000027
WVIV	MSFC	0.000019	MSFC	0.000072

Resultados del nuevo experimento: 4m2

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	WLEC	0.000087	WLEC	0.000196
MIA	MIA	0.027692	MIA	0.023020
MRS	MRS	0.000072	MRS	0.000007
MSFC	WVIV	0.000568	WVIV	0.000517
MVIV	WVAE	0.000137	WVAE	0.000101
WIDS	WSUT	0.000014	WSUT	0.000008
WIDSL	WIDSL	0.000002	WIDSL	0.000001
WLEC	WLEC	0.000561	WLEC	0.000609
WSUT	WSUT	0.000002	WSUT	0.000000
WVAE	WVAE	0.000002	WVAE	0.000000
WVAL	WVAL	0.000027	WVAL	0.000027
WVIV	MSFC	0.000020	MSFC	0.000072

Resultados del nuevo experimento: 5m

Archivo - Nuevo experimento	Primer Filtro	mse_1	Segundo Filtro	mse_2
MCEFL	MRS	0.000026	MRS	0.000041
MIA	MIA	0.008087	MIA	0.006098
MRS	MCEFL	0.000099	MCEFL	0.000095
MSC	MSC	0.000047	MSC	0.000055
MSFC	WVIV	0.000518	WVIV	0.000442
MVIV	WVAE	0.000012	WVAE	0.000003
WIDS	WIDS	0.000040	WIDS	0.000009
WIDSL	WIDSL	0.000000	WIDSL	0.000001
WLEC	WLEC	0.000053	WLEC	0.000053
WSUT	WIDS	0.000011	WSUT	0.000003
WVAE	WVAE	0.000056	WVAE	0.000030
WVAL	WVAL	0.000022	WVAL	0.000012
WVIV	WVIV	0.000001	WVIV	0.000007

Anexo V

Generación de métricas de evaluación de modelo propuesto

Este programa implementa un procedimiento automatizado para clasificar nuevos experimentos a partir de los perfiles de ataque referencia. En primer lugar, se definen grupos de prefijos que representan distintas categorías de ataques, los cuales se utilizan para restringir la comparación a subconjuntos relevantes. Posteriormente, se cargan múltiples archivos CSV correspondientes a los datos obtenidos de los ataques de referencia (usados en el Anexo S). Con estos datos se generan perfiles promedio por prefijo y dimensión. A continuación, para cada nuevo experimento, se identifica su grupo correspondiente y se calcula un perfil promedio que es comparado con los perfiles de referencia mediante el error cuadrático medio (MSE), seleccionando como clasificación final aquella con el menor valor de error.

Una vez obtenidas las clasificaciones, el programa genera un archivo CSV que contiene el archivo analizado, la clase real, la clase predicha y el valor de MSE asociado. Posteriormente, se evalúa el desempeño del modelo calculando métricas de clasificación como precisión, recall y F1-score para cada clase.

Este proceso se repite variando los valores de referencia y el nuevo experimento analizado por cada experimento, con el fin de seguir un algoritmo de corrimiento a los experimentos utilizados.

Consola V.1: Programa en Jupyter Notebook `metricas_evaluacion.ipynb` - Celda 01

```
1 import os
2 import re
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8 from sklearn.metrics import (
9     mean_squared_error,
10     precision_score,
11     recall_score,
12     f1_score
13 )
14
15 # =====
16 # Grupos de prefijos
```

```

17 # =====
18 grupos_prefijos = {
19     "Grupo1": ["WIDSL", "WIDS", "WSUT"],
20     "Grupo2": ["MVIV", "WVAE"],
21     "Grupo3": ["MSFC", "WVIV"],
22     "Grupo4": ["MSC", "WVAL"],
23     "Grupo5": ["MCEFL", "MRS", "WLEC"],
24     "Grupo6": ["MIA"],
25 }
26
27 # =====
28 # Funciones
29 # =====
30 def extraer_prefijo(nombre_archivo):
31     base = os.path.basename(nombre_archivo)
32     m = re.search(r"distancias_wasserstein_\d+m\d*(\w+)\.csv", base)
33     return m.group(1) if m else None
34
35 def obtener_grupo(prefijo):
36     for grupo, lista in grupos_prefijos.items():
37         if prefijo in lista:
38             return grupo
39     return None
40
41 def preparar_dataframe(df):
42     df = df.copy()
43     df["dimension"] = pd.to_numeric(df["dimension"], errors="coerce")
44     df["distancia"] = pd.to_numeric(df["distancia"], errors="coerce")
45     return df.dropna(subset=["dimension", "distancia"])
46
47 def calcular_promedios_ref(df):
48     return df.groupby(["prefijo", "dimension"])["distancia"].mean().reset_index()
49
50 def clasificar_nuevo(df_nuevo, ref_profiles, nombre_archivo, grupo_prefijo, prefijo_real):
51     resultados = []
52
53     nuevo_prom = df_nuevo.groupby("dimension")["distancia"].mean().to_dict()
54     ref_filtrado = ref_profiles[ref_profiles["prefijo"].isin(grupo_prefijo)]
55
56     for prefijo, df_pref in ref_filtrado.groupby("prefijo"):
57         ref_prom = df_pref.set_index("dimension")["distancia"].to_dict()
58         dims = sorted(set(nuevo_prom.keys()) & set(ref_prom.keys()))
59         if not dims:
60             continue
61
62         y_new = np.array([nuevo_prom[d] for d in dims])
63         y_ref = np.array([ref_prom[d] for d in dims])
64         mse = mean_squared_error(y_new, y_ref)
65

```

```

66     resultados.append({
67         "archivo": nombre_archivo,
68         "real": prefijo_real,
69         "predicho": prefijo,
70         "mse": mse
71     })
72
73     if not resultados:
74         return None
75
76     return min(resultados, key=lambda x: x["mse"])
77
78 # =====
79 # Ejecución
80 # =====
81 if __name__ == "__main__":
82
83     carpeta_resultados = "resultados_clasificacion/R4m1" #Carpeta de guardado
84     os.makedirs(carpeta_resultados, exist_ok=True)
85
86     # =====
87     # Ataques de referencia
88     # =====
89     archivos_ref = [
90         "distancias_wasserstein_4m1.csv",
91         "distancias_wasserstein_4m2.csv",
92         "distancias_wasserstein_5m.csv",
93         "distancias_wasserstein_10m.csv",
94         "distancias_wasserstein_10m1.csv",
95         "distancias_wasserstein_10m2.csv",
96         "distancias_wasserstein_10m4.csv",
97         "distancias_wasserstein_10m5.csv",
98         "distancias_wasserstein_10m6.csv",
99         "distancias_wasserstein_10m7.csv",
100     ]
101
102     dfs = []
103     for f in archivos_ref:
104         if os.path.exists(f):
105             df = pd.read_csv(f)
106             df["prefijo"] = df["archivo_a"].str.extract(r'([A-Z]+)')
107             dfs.append(df)
108
109     df_ref = preparar_dataframe(pd.concat(dfs, ignore_index=True))
110     ref_profiles = calcular_promedios_ref(df_ref)
111
112     # =====
113     # Clasificación
114     # =====

```



```

115 #carpetas generadas en el Anexo S
116 carpeta_nuevos = "nuevos_experimentos_4m1"
117 nuevos_csv = [f for f in os.listdir(carpeta_nuevos) if f.endswith(".csv")]
118
119 resultados_clasificacion = []
120
121 for nuevo_csv in nuevos_csv:
122     prefijo_real = extraer_prefijo(nuevo_csv)
123     grupo = obtener_grupo(prefijo_real)
124     if not grupo:
125         continue
126
127     ruta = os.path.join(carpeta_nuevos, nuevo_csv)
128     df_nuevo = preparar_dataframe(pd.read_csv(ruta))
129
130     resultado = clasificar_nuevo(
131         df_nuevo,
132         ref_profiles,
133         nuevo_csv,
134         grupos_prefijos[grupo],
135         prefijo_real
136     )
137     if resultado:
138         resultados_clasificacion.append(resultado)
139
140 # =====
141 # Guardar resultados
142 # =====
143 df_eval = pd.DataFrame(resultados_clasificacion)
144 df_eval.to_csv(os.path.join(carpeta_resultados, "evaluacion_clasificacion.csv"), index=False)
145
146 labels = sorted(df_eval["real"].unique())
147
148 metricas = []
149 for label in labels:
150     y_true = (df_eval["real"] == label).astype(int)
151     y_pred = (df_eval["predicho"] == label).astype(int)
152
153     metricas.append({
154         "ataque": label,
155         "precision": precision_score(y_true, y_pred, zero_division=0),
156         "recall": recall_score(y_true, y_pred, zero_division=0),
157         "f1": f1_score(y_true, y_pred, zero_division=0)
158     })
159
160 df_metricas = pd.DataFrame(metricas)
161 df_metricas.to_csv(os.path.join(carpeta_resultados, "metricas_por_clase.csv"), index=False)
162

```

```

163 df_error = df_eval.groupby("real")["mse"].mean().reset_index()
164 df_error.to_csv(os.path.join(carpeta_resultados, "error_promedio_por_clase.csv"), index=
    False)
165
166 print(f"Todos los resultados se han guardado en la carpeta '{carpeta_resultados}'")

```

Posteriormente, el programa genera diversos gráficos para facilitar la interpretación de los resultados. Entre ellos se incluyen gráficos de barras comparativos de métricas, una matriz de confusión, un mapa de calor de desempeño por clase y un ranking basado en el F1-score. Todas las visualizaciones, junto con un archivo CSV que contiene las métricas globales promedio, se almacenan en una carpeta específica, permitiendo así un análisis visual y cuantitativo más completo del comportamiento del modelo.

Consola V.2: Programa en Jupyter Notebook `metricas_evaluacion.ipynb` - Celda 02

```

1 import os
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.metrics import precision_score, recall_score, f1_score
6
7
8 # Carpeta donde están las carpetas de experimentos
9 base_path = "./resultados_clasificacion" # Ajusta según tu estructura
10
11 # Nombres de las carpetas con experimentos
12 carpetas_experimentos = [
13     "R4m1", "R4m2", "R5m0",
14     "R10m0", "R10m1", "R10m2", "R10m4", "R10m5", "R10m6", "R10m7"
15 ]
16
17 dfs = []
18
19 for carpeta in carpetas_experimentos:
20     archivo_resultados = os.path.join(base_path, carpeta, "evaluacion_clasificacion.csv")
21     if os.path.exists(archivo_resultados):
22         df = pd.read_csv(archivo_resultados)
23         df["experimento"] = carpeta # Para identificar la fuente
24         dfs.append(df)
25     else:
26         print(f"No se encontró 'evaluacion_clasificacion.csv' en la carpeta '{carpeta}'")
27
28 if not dfs:
29     print("No se encontraron archivos para procesar.")
30     exit()
31
32 # Unir todos los datos
33 df_todos = pd.concat(dfs, ignore_index=True)
34
35
36 # =====

```

```

37 # Calcular métricas globales promedio por clase
38 # =====
39
40 metricas_globales = []
41 clases = sorted(df_todos["real"].unique())
42
43 for clase in clases:
44     y_true = (df_todos["real"] == clase).astype(int)
45     y_pred = (df_todos["predicho"] == clase).astype(int)
46
47     precision = precision_score(y_true, y_pred, zero_division=0)
48     recall = recall_score(y_true, y_pred, zero_division=0)
49     f1 = f1_score(y_true, y_pred, zero_division=0)
50     mse_promedio = df_todos.loc[df_todos["real"] == clase, "mse"].mean()
51
52     metricas_globales.append({
53         "ataque": clase,
54         "precision_promedio": precision,
55         "recall_promedio": recall,
56         "f1_promedio": f1,
57         "mse_promedio": mse_promedio
58     })
59
60 df_metricas_global = pd.DataFrame(metricas_globales)
61
62 # Crear carpeta para resultados globales y gráficos
63 output_folder = "./resultados_clasificacion/resultados_globales"
64 os.makedirs(output_folder, exist_ok=True)
65
66 # Guardar CSV con métricas globales
67 df_metricas_global.to_csv(os.path.join(output_folder, "metricas_globales_promedio.csv"), index=
    False)
68
69 # === Gráficos ===
70
71 # Barras comparativas: Precisión, Recall y F1
72 plt.figure(figsize=(12,7))
73 x = range(len(df_metricas_global))
74 width = 0.25
75
76 plt.bar([i - width for i in x], df_metricas_global["precision_promedio"], width, label="Precisió
    n")
77 plt.bar(x, df_metricas_global["recall_promedio"], width, label="Recall")
78 plt.bar([i + width for i in x], df_metricas_global["f1_promedio"], width, label="F1")
79
80 plt.xticks(x, df_metricas_global["ataque"], rotation=45)
81 plt.ylabel("Valor_promedio")
82 plt.xlabel("Tipo_de_ataque")
83 plt.title("Métricas_promedio_globales_por_tipo_de_ataque")

```

```

84 plt.legend()
85 plt.grid(axis="y", linestyle="--", alpha=0.6)
86 plt.tight_layout()
87 plt.savefig(os.path.join(output_folder, "metricas_globales_comparadas.svg"), bbox_inches="tight"
88 )
89 #plt.savefig(os.path.join(output_folder, "metricas_globales_comparadas.png"), dpi=300)
90 plt.close()
91
92 # =====
93 # Asignación de gráficos
94 # =====
95 df_metricas_detalle = []
96
97 for clase in clases:
98     df_clase = df_todos[df_todos["real"] == clase]
99
100     for _, row in df_clase.iterrows():
101         y_true = [1]
102         y_pred = [1 if row["predicho"] == clase else 0]
103
104         df_metricas_detalle.append({
105             "ataque": clase,
106             "metric": "precision",
107             "valor": precision_score(y_true, y_pred, zero_division=0)
108         })
109         df_metricas_detalle.append({
110             "ataque": clase,
111             "metric": "recall",
112             "valor": recall_score(y_true, y_pred, zero_division=0)
113         })
114         df_metricas_detalle.append({
115             "ataque": clase,
116             "metric": "f1",
117             "valor": f1_score(y_true, y_pred, zero_division=0)
118         })
119
120 df_metricas_detalle = pd.DataFrame(df_metricas_detalle)
121
122 # =====
123 # Matriz de confusión
124 # =====
125 cm = confusion_matrix(df_todos["real"], df_todos["predicho"], labels=clases)
126
127 plt.figure(figsize=(10,8))
128 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
129             xticklabels=clases, yticklabels=clases)
130
131 plt.title("Matriz de confusión global")

```

```
132 plt.xlabel("Predicho")
133 plt.ylabel("Real")
134 plt.tight_layout()
135 plt.savefig(os.path.join(output_folder, "matriz_confusion.svg"))
136 plt.close()
137
138 # =====
139 # Mapa de calor
140 # =====
141 plt.figure(figsize=(8,6))
142 sns.heatmap(df_metricas_global.set_index("ataque")[["precision_promedio", "recall_promedio", "
    f1_promedio"]],
143             annot=True, cmap="YlGnBu")
144
145 plt.title("Heatmap de métricas globales")
146 plt.tight_layout()
147 plt.savefig(os.path.join(output_folder, "heatmap_metrics.svg"))
148 plt.close()
149
150 # =====
151 # Ranking F1
152 # =====
153 df_metricas_sorted = df_metricas_global.sort_values("f1_promedio")
154
155 plt.figure(figsize=(10,6))
156 sns.barplot(x="f1_promedio", y="ataque", data=df_metricas_sorted, palette="viridis")
157
158 plt.title("Ranking de ataques por F1-score")
159 plt.tight_layout()
160 plt.savefig(os.path.join(output_folder, "ranking_f1.svg"))
161 plt.close()
162
163
164 print(f"Todas las gráficas guardadas en: {output_folder}")
```

Bibliografía

- [1] R. Kumar y R. Goyal, «On cloud security requirements, threats, vulnerabilities and countermeasures: A survey», *Computer Science Review*, vol. 33, pp. 1-48, may 2019, doi: 10.1016/j.cosrev.2019.05.002.
- [2] F. Reynaud, F.-X. Aguessy, O. Bettan, M. Bouet, y V. Conan, "Attacks against Network Functions Virtualization and Software-Defined Networking: State-of-the-art", *HAL Open Science*, jun. 2016, doi: 10.1109/netsoft.2016.7502487.
- [3] Marques, H., Laranjeiro, N., & Bernardino, J. (2021). *Injecting software faults in Python applications. Empirical Software Engineering*, 27(1). <https://doi.org/10.1007/s10664-021-10047-9>
- [4] Mell, P. M., & Grance, T. (2011). *The NIST definition of cloud computing*. <https://doi.org/10.6028/nist.sp.800-145>
- [5] Susnjara, S., & Smalley, I. (2025, June 16). *Computación en la nube. IBM-Think*. <https://www.ibm.com/mx-es/think/topics/cloud-computing>
- [6] CSRC Content Editor. (n.d.). *Cyber Attack - Glossary | CSRC*. https://csrc.nist.gov/glossary/term/cyber_attack
- [7] .^{E1} concepto de OpenStack". <https://www.redhat.com/es/topics/openstack>
- [8] "¿Qué es OpenStack? Beneficios y componentes | Fortinet", Fortinet. <https://www.fortinet.com/lat/resources/cyberglossary/openstack>
- [9] A. Rampersad, "Who is using OpenStack?", OpenMetal IaaS, 7 de noviembre de 2025. <https://openmetal.io/resources/blog/who-is-using-openstack/>
- [10] Wolfram Research, Inc. (n.d.). *Homotopy - from Wolfram MathWorld*. <https://mathworld.wolfram.com/Homotopy.html>
- [11] Nadathur, P. (2007, Agosto 16). *An introduction to homology [Manuscrito sin publicar]*. *University of Chicago*. <https://www.math.uchicago.edu/~may/VIGRE/VIGRE2007/REUPapers/INCOMING/Project.pdf>
- [12] Kerber, M., Morozov, D., & Nigmatov, A. (2017). *Geometry helps to compare persistence diagrams*. *ACM Journal of Experimental Algorithmics*, 22, 1–20. <https://doi.org/10.1145/3064175>
- [13] Edelsbrunner, H., & Harer, J. L. (2009). *Computational Topology: An Introduction*.
- [14] Pranav, P., Edelsbrunner, H., Van De Weygaert, R., Vegter, G., Kerber, M., Jones, B. J. T., & Wintraecken, M. (2016). *The topology of the cosmic web in terms of persistent Betti numbers*. *Monthly Notices of the Royal Astronomical Society*, 465(4), 4281–4310. <https://doi.org/10.1093/mnras/stw2862>

- [15] L. Simi, "A scalable approach for mapper via efficient spatial search," OpenReview. <https://openreview.net/forum?id=1TX4bYREAZ>
- [16] G. Singh, F. Memoli, y G. Carlsson, "Topological Methods for the Analysis of High Dimensional Data Sets and 3D Object Recognition", Eurographics, ene. 2007, doi: 10.2312/spbg/spbg07/091-100.
- [17] Chazal, F., Cohen-Steiner, D., Glisse, M., Guibas, L. J., & Oudot, S. Y. (2009). *Proximity of persistence modules and their diagrams*. *ACM Digital Library*, 237–246. <https://doi.org/10.1145/1542362.1542407>
- [18] T. Gowdrige, N. Dervilis, y K. Worden, «On topological data analysis for structural dynamics: an introduction to persistent homology», arXiv (Cornell University), sep. 2022, doi: 10.48550/arxiv.2209.05134.
- [19] Hamilton, W., Borgert, J. E., Hamelryck, T., & Marron, J. S. (2021). *Persistent topology of protein space*. arXiv (Cornell University). <https://doi.org/10.48550/arxiv.2102.06768>
- [20] Instituto Nacional de Ciberseguridad (INCIBE). (n.d.). *Guía de ciberataques*. <https://www.incibe.es/sites/default/files/docs/guia-ciberataques/osi-guia-ciberataques.pdf>
- [21] M. Kosinski y A. Forrest, «Inyección de instrucciones», IBM Think, 26 de noviembre de 2025. <https://www.ibm.com/mx-es/think/topics/prompt-injection>
- [22] Feinbube, L., Pirl, L., & Polze, A. (2018). *Software Fault Injection: A Practical perspective*. In *InTech eBooks*. <https://doi.org/10.5772/intechopen.70427>
- [23] Ibm. (2024, Julio 15). *Sistema de detección de intrusiones*. IBM- Think. <https://www.ibm.com/mx-es/topics/intrusion-detection-system>
- [24] Ho, C., Lai, Y., Chen, I., Wang, F., & Tai, W. (2012). *Statistical analysis of false positives and false negatives from real traffic with intrusion detection/prevention systems*. *IEEE Communications Magazine*, 50(3), 146–154. <https://doi.org/10.1109/mcom.2012.6163595>
- [25] K. Murdock, D. Oswald, F. D. Garcia, J. Van Bulck, D. Gruss, y F. Piessens, "Plundervolt: Software-based Fault Injection Attacks against Intel SGX", 2020 IEEE Symposium On Security And Privacy (SP), pp. 1466-1482, may 2020, doi: 10.1109/sp40000.2020.00057.
- [26] J. M. Voas y A. K. Ghosh, "Software fault injection for survivability", Proceedings DARPA Information Survivability Conference And Exposition. DISCEX'00, vol. 2, pp. 338-346, nov. 2002, doi: 10.1109/discex.2000.821531.
- [27] A. Gangolli, Q. H. Mahmoud, y A. Azim, "A Systematic Review of Fault Injection Attacks on IoT Systems", *Electronics*, vol. 11, n.o 13, p. 2023, jun. 2022, doi: 10.3390/electronics11132023.
- [28] B. Yuce, P. Schaumont, y M. Witteman, "Fault Attacks on Secure Embedded Software: Threats, Design, and Evaluation", *Journal Of Hardware And Systems Security*, vol. 2, n.o 2, pp. 111-130, may 2018, doi: 10.1007/s41635-018-0038-1.
- [29] I. Chillotti, N. Gama, y L. Goubin, "Attacking FHE-based applications by software fault injections," *IACR Cryptology ePrint Archive*, dic. 28, 2016. <https://eprint.iacr.org/2016/1164>

- [30] Velliangiri, S., Alagumuthukrishnan, S., & Joseph, S. I. T. (2019). *A review of Dimensionality Reduction Techniques for Efficient Computation*. *Procedia Computer Science*, 165, 104–111. <https://doi.org/10.1016/j.procs.2020.01.079>
- [31] Trozzi, F., Wang, X., & Tao, P. (2021). *UMAP as a Dimensionality Reduction Tool for Molecular Dynamics Simulations of Biomacromolecules: A comparison study*. *The Journal of Physical Chemistry B*, 125(19), 5022–5034. <https://doi.org/10.1021/acs.jpcc.1c02081>
- [32] Hasan, B. M. S., & Abdulazeez, A. M. (2021). *A review of Principal Component Analysis Algorithm for Dimensionality Reduction*. *Journal of Soft Computing and Data Mining*, 02(01). <https://doi.org/10.30880/jscdm.2021.02.01.003>
- [33] Bhaskar, D., Zhang, W. Y., & Wong, I. Y. (2020). *Topological Data Analysis of Collective and Individual Epithelial Cells using Persistent Homology of Loops*. arXiv (Cornell University). <https://doi.org/10.48550/arxiv.2003.10008>
- [34] Carlsson, G. (2009). *Topology and data*. *Bulletin of the American Mathematical Society*, 46(2), 255–308. <https://doi.org/10.1090/s0273-0979-09-01249-x>
- [35] Tinarrage, R. (2021, Enero 26). *Topological Data Analysis with Persistent Homology*. <https://raphaeltinarrage.github.io/files/EMAP/SummerCourseTDA.pdf>
- [36] Jones, E. (2024, Octubre 28). *6 pillars of data quality and how to improve your data*. IBM. <https://www.ibm.com/products/tutorials/6-pillars-of-data-quality-and-how-to-improve-your-data>
- [37] *MapperGIC Nerve complexes user manual — gudhi v3.11.0 documentation*. (n.d.). https://gudhi.inria.fr/python/latest/cover_complex_sklearn_user.html
- [38] V. Marx, “Seeing data as t-SNE and UMAP do,” *Nature Methods*, vol. 21, no. 6, pp. 930–933, May 2024, doi: 10.1038/s41592-024-02301-x.
- [39] *Rips complex user manual — gudhi v3.11.0 documentation*. (n.d.). https://gudhi.inria.fr/python/latest/rips_complex_user.html
- [40] R. Yacoub y D. Axman, «Probabilistic Extension of Precision, Recall, and F1 Score for More Thorough Evaluation of Classification Models», *ACL Anthology*, pp. 79-91, ene. 2020, doi: 10.18653/v1/2020.eval4nlp-1.9.
- [41] R. Dinga, B. W. J. H. Penninx, D. J. Veltman, L. Schmaal, y A. F. Marquand, «Beyond accuracy: Measures for assessing machine learning models, pitfalls and guidelines», *bioRxiv* (Cold Spring Harbor Laboratory), ago. 2019, doi: 10.1101/743138.